
LassoFlexNet: a Flexible Neural Architecture for Tabular Data

Kry Yik Chau Lui¹ Cheng Chi¹ Kishore Basu² Yanshuai Cao¹

Abstract

Despite their dominance in vision and language, deep neural networks often underperform relative to tree-based models on tabular data. To bridge this gap, we incorporate five key inductive biases into deep tabular learning: robustness to irrelevant features, axis alignment, localized irregularities, feature heterogeneity, and training stability. We propose *LassoFlexNet*, an architecture that evaluates the linear and nonlinear marginal contribution of each input via Per-Feature Embeddings, and sparsely selects relevant variables using a Tied Group Lasso mechanism. Because these components introduce optimization challenges that destabilize standard proximal methods, we analyze stochastic hierarchical proximal dynamics and develop a *Sequential Hierarchical Proximal Adaptive Gradient optimizer with exponential moving averages (EMA)* designed to stabilize training in practice. Across 52 datasets from three benchmarks, LassoFlexNet matches or outperforms leading tree-based models, achieves up to a 10% relative gain, and outperforms TabPFN/TabICL on 5 of 8 TabRed tasks without foundation-model pretraining or quadratic context-length inference. We substantiate these empirical results with ablation studies and theoretical proofs confirming the architecture’s enhanced expressivity, structural breaking of undesired rotational invariance, and Lasso-like interpretability.

1. Introduction

Despite the dominance of deep neural networks in vision and language (He et al., 2016; Vaswani et al., 2017), tree-based methods such as XGBoost and CatBoost (Chen & Guestrin, 2016; Prokhorenkova et al., 2018) remain the default choice for tabular data. Grinsztajn et al. (2022) identified three

primary reasons for this performance gap: (1) robustness to uninformative features, (2) preservation of data orientation, and (3) the ability to learn irregular, localized functions.

We argue that input heterogeneity-the mix of varying abstraction levels-is a fourth, overlooked but equally critical factor. Unlike homogeneous vision or text data, tabular datasets frequently mix raw physical measurements (e.g., headcount) with complex derived metrics (e.g., price-to-equity ratio). Standard neural networks linearly blend these disparate inputs into a shared representation space during the first layer, irreparably obscuring their marginal information. Tree-based models, conversely, recursively partition the input space in an axis-aligned manner, natively isolating heterogeneity and modeling sharp, step-like decision boundaries.

Fusing these four inductive biases into deep learning while ensuring robust optimization requires moving fundamentally beyond architectures amenable to standard stochastic gradient descent. We propose *LassoFlexNet*¹, an architecture engineered to mimic the coordinate-wise localized flexibility of trees, coupled with a novel optimization algorithm to resolve the resulting non-convex non-smooth training dynamics. Our primary contributions are:

1. **Tabular-Native Architecture:** We introduce a Tied Group Lasso mechanism that operates on non-linear Per-Feature Embeddings rather than raw inputs. Furthermore, we rigorously prove that Piecewise Linear Encoding (PLE) fundamentally breaks the undesired rotational invariance of neural networks (Ng, 2004) and amplifies functional expressivity for localized irregularities (Propositions 6.1 and 6.2).
2. **Stable Proximal Optimization:** We derive a *Sequential Hierarchical Proximal Adaptive Gradient optimizer with exponential moving averages (Seq-Hier-Prox-EMA)*. This algorithm decouples the optimization trajectory, ensuring the Lasso selection mechanism and the deep MLP-Mixer module (Tolstikhin et al., 2021) learn synergistically, overcoming the severe training instabilities of prior sparse neural methods like LassoNet (Lemhadri et al., 2021b).

¹RBC Borealis, Toronto, Canada ²RBC, Toronto, Canada. Correspondence to: Kry Yik Chau Lui <yikchau.y.lui@borealisai.com>.

¹Code: <https://github.com/BorealisAI/lassoflexnet>.

3. **State-of-the-Art Empirical Performance:** Across 52 datasets spanning three recent benchmarks (Cherepanova et al., 2023b; Rubachev et al., 2025), *LassoFlexNet* consistently matches or surpasses leading tree-based and deep tabular models, achieving up to a 10% relative improvement. On TabRed, it also outperforms TabPFN/TabICL (Grinsztajn et al., 2025; QU et al., 2025) on 5 of 8 tasks using mini-batch training rather than foundation-model pretraining or quadratic context-length inference, while providing high-fidelity feature importance metrics.

Overall, *LassoFlexNet* bridges the representational and optimization gaps between neural networks and decision trees, offering a principled, robust foundation for deep tabular learning.

2. Related Work

Neural Feature Selection. Beyond LassoNet, several methods have attempted to integrate feature selection into deep learning. Feature Selection Layer (FSL) (Abid et al., 2019) and Stochastic Gates (STG) (Yamada et al., 2020) utilize probabilistic relaxations to sample active features, while LassoLayer (Sudo et al., 2021) applies a soft ℓ_1 penalty to a linear layer. However, these methods often struggle with exact sparsity or fail to capture complex nonlinear dependencies before selection. *LassoFlexNet* diverges from these stochastic relaxations by employing a mathematically principled proximal optimization over a Tied Group Lasso. Crucially, our direct predecessor is LassoNet (Lemhadri et al., 2021b), which introduced the hierarchical proximal constraint. While theoretically sound for inducing feature sparsity, LassoNet frequently underperforms on modern tabular benchmarks (as shown in Sec. 7) and fails to serve as a reliable alternative to tree-based models. *LassoFlexNet* addresses these deficiencies through two critical innovations:

- **Architecture:** Rather than performing selection on raw inputs, we introduce Per-Feature Embeddings (PFE) combined with a Tied Group Lasso mechanism. This enables feature selection on learned, nonlinear representations, capturing marginal predictive contributions that raw inputs might obscure.
- **Optimization:** We replace LassoNet’s joint optimization with a sequential procedure utilizing proximal adaptive gradients stabilized via exponential moving averages (EMA).

Neural Networks on Tabular Data. The broader landscape of deep tabular learning has focused heavily on adapting interaction modules to rival decision trees. Models such as TabNet (Arik & Pfister, 2021) and FT-Transformer (Gorishniy et al., 2021) leverage attention and feature-interaction

mechanisms. Recently, TabM (Gorishniy et al., 2025) enhanced these backbones via parameter-efficient ensembling, improving generalization through diverse predictions.

In-Context and Retrieval-Based Methods. An orthogonal class of modern methods focuses on retrieval and in-context learning. TabR (Gorishniy et al., 2024) utilizes a retrieval-augmented design to query similar training examples at inference. Alternatively, TabPFN (Hollmann et al., 2023) and its successor TabPFN-v2.6 (Grinsztajn et al., 2025) operate as pre-trained in-context learners optimized over diverse synthetic and real datasets, an approach that TabICL (QU et al., 2025) scales further into an alternative foundation model. However, these foundation models suffer from severe structural scalability limitations: their attention mechanisms require $\mathcal{O}(n^2)$ memory with respect to the context length (typically capped around 48K samples). Scaling them to datasets with hundreds of thousands of rows requires expensive ensembling or subsampling heuristics. In contrast, *LassoFlexNet*’s mini-batch training provides $\mathcal{O}(1)$ memory with respect to dataset size, natively supporting massive datasets while remaining highly competitive in performance (see Sec. 7).

Missing Baselines and Transitive Dominance. While our evaluation covers 52 datasets against leading GBDTs and TabM, we omit certain deep baselines such as RealMLP (Blaette et al., 2024), SAINT (Somepalli et al., 2021), TabR (Gorishniy et al., 2024), and Mambular (Thielmann et al., 2024). Recent exhaustive benchmarks (Erickson et al., 2025; Gorishniy et al., 2025) have demonstrated that without ensembling, these models are transitively dominated by TabM and state-of-the-art tree methods, which we do explicitly compare against.

Table 1. LassoFlexNet components addressing tabular data challenges.

Error	Inductive Bias	Solution
Generalization	Robustness to irrelevant features;	Lasso Linear Skip Connection with Tied Group Lasso Sequential Hierarchical Proximal Adaptive Gradient optimizer + EMA PLE; PFE; MLP-Mixer Module
Optimization	axis alignment Stable training	
Approximation	Axis alignment, localized irregularities; heterogeneity; complex interactions	

3. Design Principles for Tabular Inductive Biases

To rival the performance of tree-based models, neural architectures must natively encode five critical inductive biases: robustness to irrelevant features, axis alignment, localized irregularities, feature heterogeneity, and training stability (Section 1). We formalize our design approach by

decomposing the test loss into three distinct error components—generalization, optimization, and approximation:

$$\begin{aligned} \mathcal{L}_{\text{Test}} = & \underbrace{\mathcal{L}_{\text{Test}} - \mathcal{L}_{\text{Train}}}_{\text{Generalization Error}} + \underbrace{\mathcal{L}_{\text{Train}} - \mathcal{L}_{\text{Train, Min}}}_{\text{Optimization Error}} \\ & + \underbrace{\mathcal{L}_{\text{Train, Min}}}_{\text{Approximation Error}}. \end{aligned} \quad (1)$$

LassoFlexNet structurally bounds these errors via targeted architectural and algorithmic interventions (summarized in Table 1):

- **Approximation Error:** To model localized irregularities and manage feature heterogeneity, we utilize **Piecewise Linear Encoding (PLE)** (Gorishniy et al., 2022) and **Per-Feature Embeddings (PFE)**. A subsequent **MLP-Mixer Module** (Tolstikhin et al., 2021) captures complex cross-feature interactions.
- **Generalization Error:** To preserve axis orientation and strictly mitigate overfitting to irrelevant features, we construct a **Linear Skip Connection governed by a Tied Group Lasso** penalty over the learned embeddings.
- **Optimization Error:** To reduce optimization instability across these high-capacity, non-homogeneous modules, we derive a **Seq-Hier-Prox-Adam-EMA** optimizer.

Sections 4 and 5 detail these components mathematically, before Section 6 analyzes how they jointly satisfy the required inductive biases.

4. Architecture

4.1. Preliminaries: LassoNet

LassoNet (Lemhadri et al., 2021b) integrates feature selection by augmenting a standard neural module, $\text{NN}_W(x)$, with a linear skip connection, $\beta^\top x$, yielding the prediction $\hat{y} = \beta^\top x + \text{NN}_W(x)$. The linear coefficients β and neural weights W are jointly optimized via a hierarchical penalty:

$$\min_{\beta, W} L(\beta, W) + \lambda \|\beta\|_1 \quad \text{s.t.} \quad \|W_{i,-}^{(1)}\|_\infty \leq M|\beta_i| \quad (2)$$

where $W_{i,-}^{(1)}$ denotes the outgoing weights from the i -th input. This ℓ_1 penalty promotes strict sparsity on β , while the ℓ_∞ constraint ensures that a feature can only be utilized by the neural network if its corresponding linear counterpart is non-zero. However, LassoNet’s reliance on raw input correlations severely limits its ability to capture nonlinear feature importance. Furthermore, its joint Hier-Prox optimization often suffers from poor, unstable training dynamics (analyzed in Section 6.1).

4.2. LassoFlexNet Overview

LassoFlexNet (Figure 1) circumvents these limitations via a three-component design: (1) independent per-feature neural networks $z_i = \text{NN}_{\phi_i}(x_i)$ utilizing Piecewise Linear Encoding (PLE) and Per-Feature Embedding (PFE); (2) a Tied Group Linear Skip Connection $\beta^\top \bar{z}$, operating on the mean-pooled representation $\bar{z}_i$ of each embedding; and (3) a deep MLP-Mixer (Tolstikhin et al., 2021) module $\tau \cdot \text{NN}_W(z)$, where τ acts as a soft-curriculum scale to control the inductive bias toward simplicity if $\tau < 1$ and towards expressivity if $\tau > 1$ (Eq. 3). Formally, we define:

$$\begin{aligned} z &= [\dots, z_i, \dots], \quad \bar{z} = [\dots, \text{mean}(z_i), \dots] \\ \hat{y} &= \beta^\top \bar{z} + \tau \cdot \text{NN}_W(z) \\ \beta_{\text{Lasso}} &= \arg \min_{\beta} L(\phi, \beta, W) + \lambda \|\beta\|_1 \\ \text{s.t.} \quad & \|W_{i,-}^{(1)}\|_\infty \leq M|\beta_{\text{Lasso};i}| \end{aligned} \quad (3)$$

where i indexes the i -th feature. Note the mean-pooling is a convenient choice for the Tied Group Lasso mechanism, as it allows us to apply the linear skip connection to the entire group of embeddings for a single feature, rather than to each individual embedding dimension. In LassoFlexNet, $W_{i,-}^{(1)}$ denotes the full block of first constrained Mixer feature-mixing weights that receives the non-pooled embedding z_i . Thus, when $\beta_i = 0$, all outgoing connections from feature i ’s PFE embedding into this constrained Mixer bottleneck are forced to zero; the scalar skip coefficient is tied to the full embedding group, not merely to the mean-pooled scalar. Additionally, the expressive neural net module, such as the MLP-Mixer, acts on the *non-pooled embeddings* z , which allows it to capture complex cross-feature and cross-embedding interactions. This paired design ensures that the linear skip connection guides the neural module away from irrelevant features, while the neural module maintains the full expressivity.

4.2.1. PIECEWISE LINEAR ENCODING AND PER-FEATURE EMBEDDING

To inherently adapt to tabular heterogeneity, PLE (Gorishniy et al., 2022) partitions each numerical feature x_i into K_i distinct intervals via decision tree splits $\{b_{i,k}\}$. The resulting encoding $\text{PLE}(x_i) = e_i = [e_{i,1}, \dots, e_{i,K_i}]$ is defined as:

$$e_{i,t} = \max \left(0, \min \left(1, \frac{x_i - b_{i,t-1}}{b_{i,t} - b_{i,t-1}} \right) \right) \quad (4)$$

This encoding exposes K_i fixed bin coordinates per numerical feature, giving downstream linear or shallow nonlinear layers direct access to bin-local spline degrees of freedom. Appendix E formalizes this in the scalar fixed-knot setting by showing that linear-on-PLE is exactly the linear spline class and by comparing its knot complexity with a

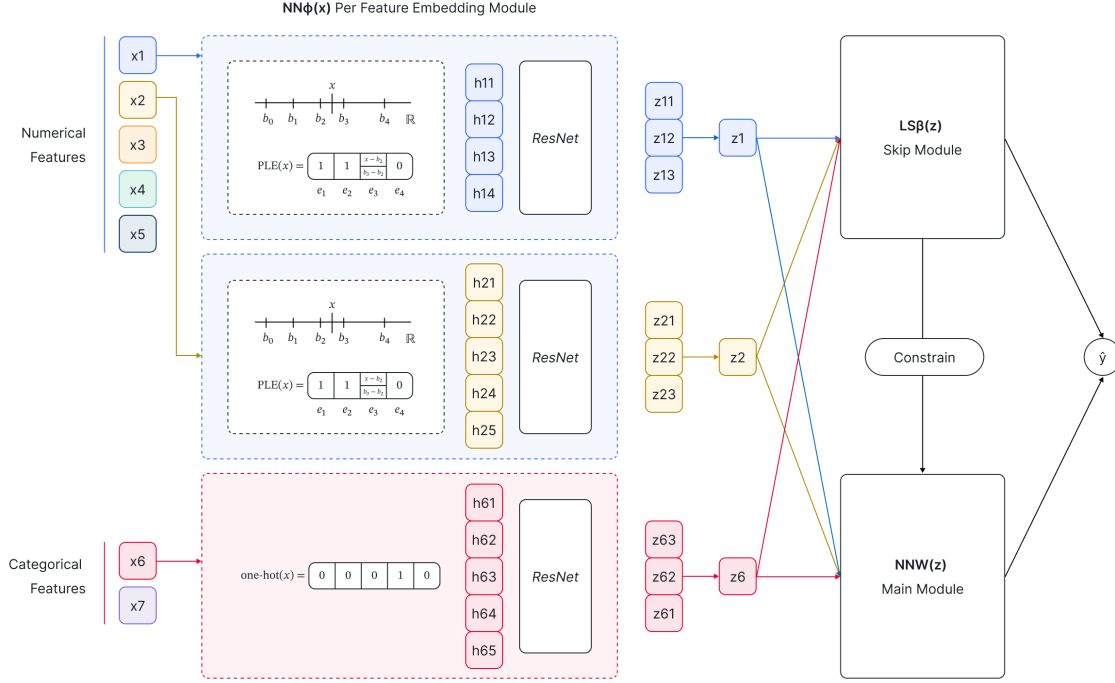


Figure 1. LassoFlexNet architecture: Per-Feature Embedding (PLE for numerical, one-hot for categorical), Linear Skip Connection, and the MLP-Mixer Module.

one-hidden-layer scalar ReLU network. Crucially, these univariate decision-tree splits are computed per variable independently—there are no cross-variable interactions at this stage. This is simply data-driven bin boundary identification, analogous to quantile-based binning. The downstream modules (PFE, Mixer, and Lasso) learn all cross-feature interactions entirely from scratch.

Subsequently, PFE transforms e_i through an independent per-feature ResNet block: $h_{i,l} = h_{i,l-1} + \text{LN}(\text{ReLU}(W_{i,l}h_{i,l-1} + b_{i,l}))$. To improve the comparability of skip-weight magnitudes $|\beta_i|$ across feature embeddings (Hastie et al., 2009), the final embedding $\text{PFE}(e_i)$ is normalized using BatchNorm without learnable affine parameters: within each mini-batch, embedding coordinates are standardized to empirical mean zero and unit variance, while the usual learnable scale and shift are disabled. Unlike existing soft regularization attempts (Sudo et al., 2021), PFE supports arbitrary architectural depth and allows exact feature removal via proximal gradients.

4.2.2. TIED GROUP LASSO MECHANISM

LassoNet’s fundamental reliance on raw inputs ignores non-linear dependencies, risking a scenario where “the blind guide the blind” if the linear skip connection converges slower than the highly-expressive neural module. *Las-*

soFlexNet resolves this tension through two mechanisms:

Selection via Nonlinear Embeddings. We apply the linear skip connection strictly to the mean-pooled embeddings $\bar{z}_i$. This implements a *Tied Group Lasso*: the scalar β_i scores the normalized mean-pooled representation of feature i , and the same β_i bounds the entire outgoing weight block from the non-pooled embedding z_i into the first constrained Mixer feature-mixing layer. This tying severely reduces parameter count and enables feature selection based on rich nonlinear representations, while preserving interpretability because the PFE outputs are standardized without learnable affine rescaling.

Soft Curriculum Learning. To prevent the expressive MLP-Mixer module from instantly dominating the training trajectory and bypassing feature selection, we may scale its output by $\tau < 1$ (Eq. 3). This structurally downscales the Mixer gradients, forcing early convergence of the linear, selection-guiding components. This Lasso-Mixer pairing is vital for tabular generalization; without it, the Mixer rapidly overfits small tabular datasets, while substituting a simpler MLP fatally reduces representational capacity (demonstrated in Sec. 6.1).

Algorithm 1 Update Algorithm for LassoNet / LassoFlexNet

```

1: Input: training data  $X \in \mathbb{R}^{n \times d}$ , labels  $Y$ , Lasso parameter  $\beta$ , neural model  $g_W(\cdot)$ , epochs  $N$ , hierarchy multiplier  $M$ , path multiplier  $\varepsilon$ , learning rate  $\eta$ 
2: Initialize and train the feed-forward network on  $L(\beta, W)$ 
3: Initialize scalar penalty path  $\Lambda = [\lambda_0, \lambda_1, \dots]$ , with each  $\lambda_i \in \mathbb{R}_+$ , and active feature count  $k = d$ 
4: for  $\lambda \in \Lambda$  do ▷ scalar penalty from path
5:   while  $k > 0$  do
6:     for  $b = 1$  to  $N$  do
7:       Compute  $\nabla L(\beta, W)$ 
8:       Update  $(\beta, W)$  via SGD or Adam
9:       if  $\lambda \neq 0$  then
10:        Update  $(\beta, W^{(1)})$  by Hier-Prox or Seq-Hier-Prox
11:        Update  $k$  to the number of non-zero coordinates of  $\beta$ 

```

5. Optimization

LassoNet’s Hier-Prox optimization (Algorithm 3 in App. B.1) enables strict feature sparsity and gives a local optimality characterization of the hierarchical proximal step. *LassoFlexNet* keeps this train/prox template (Algorithm 1) but studies two idealized maps—hard Hier-Prox and a convex relaxation—to isolate stochastic sign-switch instability beyond that result. These analyses motivate, without exactly modeling, the practical Seq-Hier-Prox-Adam-EMA update: joint $\rightarrow$ sequential optimization, proximal gradient $\rightarrow$ proximal adaptive gradient, and EMA stabilization.

Joint Optimization Blinds Lasso Guidance. LassoNet uses linear skip connections to guide the neural module away from irrelevant features, but jointly optimizing θ and $W^{(1)}$ can weaken this guidance. Hier-Prox computes a shared scalar $w_{\tilde{m}}$ to update both parameters:

$$\begin{aligned}\tilde{\theta}_j &= \frac{1}{M} \cdot \text{sign}(\theta_j) \cdot w_{\tilde{m}}, \\ \tilde{W}_j^{(1)} &= \text{sign}(W_j^{(1)}) \cdot \min(w_{\tilde{m}}, |W_j^{(1)}|).\end{aligned}$$

Because $W^{(1)}$ contains many more parameters, $w_{\tilde{m}}$ can bias the trajectory toward the neural module and sideline the skip connection. This does not contradict LassoNet’s proximal lemma (Lemhadri et al., 2021b), which gives local pointwise optimality, not a global learning-dynamics guarantee.

Sequential Optimization: Lasso-First Guidance. *LassoFlexNet* prioritizes the skip connection through a sequential, proximal-style composition: update the Lasso gates first, then enforce the induced first-layer constraint. Let $\mathcal{S}_{\lambda \cdot \eta_t}(\cdot)$ denote the soft-thresholding operator (Parikh et al., 2014). Given the unconstrained update β , we first solve for

Algorithm 2 LassoFlexNet Seq-Hier-Prox-Adam-EMA Update

```

1: procedure SEQ-HIER-PROX-ADAM-EMA
   EMA( $\beta, W^{(1)}, \beta_{\text{EMA}}, W_{\text{EMA}}^{(1)}; \lambda, M$ )
   ▷  $\beta, W^{(1)}$  can come from Adam
2:   for  $j = 1$  to  $d$  do
3:      $\tilde{\beta}_j = \text{Sign}(\beta) \cdot |\mathcal{S}_{\lambda}(\beta_{\text{EMA}, j})|$  ▷ Lasso first
4:     Sort  $|W_{\text{EMA}, j}^{(1)}|$  into  $|W_{\text{EMA}, (j, 1)}^{(1)}| \geq \dots \geq |W_{\text{EMA}, (j, K)}^{(1)}|$ 
5:     for  $m = 0$  to  $K$  do
6:        $w_m = \frac{M}{1+m} \mathcal{S}_{\lambda}(|\tilde{\beta}_j| + M \sum_{i=1}^m |W_{\text{EMA}, (j, i)}^{(1)}|)$ 
7:       Find first  $\tilde{m}$ :  $|W_{\text{EMA}, (j, \tilde{m}+1)}^{(1)}| \leq w_{\tilde{m}} \leq |W_{\text{EMA}, (j, \tilde{m})}^{(1)}|$ 
8:        $\tilde{W}_j^{(1)} = \text{Sign}(W_j^{(1)}) \min(w_{\tilde{m}}, |W_{\text{EMA}, j}^{(1)}|)$ 
9:   return  $(\tilde{\beta}, \tilde{W}^{(1)})$ 

```

Conventions: $W_{(j, K+1)} = 0$, $W_{(j, 0)} = +\infty$; $\min$ and $|\cdot|$ are elementwise.

the independent Lasso parameter $\tilde{\beta}$:

$$\tilde{\beta} = \mathcal{S}_{\lambda \cdot \eta_t}(\beta) = \arg \min_b \left[\frac{1}{2\eta_t} \|\beta - b\|_2^2 + \lambda \|b\|_1 \right] \quad (5)$$

We then shrink $\tilde{W}$ under the boundary induced by $\tilde{\beta}$:

$$\begin{aligned}\tilde{W} &= \arg \min_U \left[\frac{1}{2} \|U - W\|_2^2 + \bar{\lambda} \|U\|_1 \right] \\ \text{s.t. } &\|U\|_\infty \leq M \cdot \|\tilde{\beta}\|_2.\end{aligned}$$

Thus $\tilde{\beta}$ becomes the first-stage feature-selection signal before first-layer shrinkage.

Adaptive Integration: Proximal Adam. We replace LassoNet’s constant-rate SGD with Adam’s adaptive, per-coordinate learning rates $\eta_{i,t}$. The unconstrained update becomes:

$$\beta_{i,t} = \beta_{i,t-1} - \eta_{i,t} \cdot m_t = \beta_{i,t-1} - \eta \frac{m_t}{\sqrt{\nu_t} + \epsilon}$$

The Lasso gate update (Equation 5) remains separable under diagonal preconditioning (Boyd & Vandenberghe, 2004), giving:

$$\begin{aligned}\tilde{\beta}_{i,t+1} &= \arg \min_{b_i} \left[\frac{(b_i - \beta_{i,t})^2}{2\eta_{i,t}} + \lambda |b_i| \right] \\ &= \text{sign}(\beta_{i,t}) \max(|\beta_{i,t}| - \lambda \eta_{i,t}, 0)\end{aligned}$$

Stable EMA-Enhanced Proximal Gradients. Proximal methods often become unstable under stochastic sparse-network training, even with Adam. We apply parameter EMA *inside* the sequential proximal update (Algorithm 2), rather than only post hoc (Izmailov et al., 2018), to stabilize constraint-boundary decisions alongside the τ -induced curriculum. See App. F and App. G for the idealized proximal-dynamics bounds motivating this stabilization.

6. Theoretical Analysis of Tabular Inductive Biases

As formalized in Sections 1 and 3, robust deep tabular models must satisfy five criteria: (1) resilience to irrelevant features, (2) preservation of data orientation, (3) capacity for localized irregularities, (4) processing of heterogeneous inputs, and (5) stable training dynamics. Here, we theoretically and empirically validate how *LassoFlexNet* fulfills these requirements.

(1) Resilience to Irrelevant Features. Ng (2004) (Theorem 3.1) demonstrated that ℓ_1 regularization robustly learns even when irrelevant features exponentially outnumber training samples. By applying a Tied Group Lasso over the nonlinear per-feature embeddings (Sec. 4.2.2), *LassoFlexNet* inherits this strict feature-selection capability, extending the principles of LassoNet (Lemhadri et al., 2021b) into nonlinear representation spaces.

(2) Preservation of Data Orientation. Neural networks traditionally suffer from rotational invariance, whereas decision trees natively respect the original, meaningful coordinate axes—a critical factor in their tabular superiority. While Lasso regularization implicitly breaks this invariance (Ng, 2004), premature dense feature interactions would destroy this property. We rigorously prove that the coordinate-wise encoding alone (PLE) actively breaks rotational invariance (detailed proof in App. D):

Proposition 6.1 (PLE Breaks Rotational Invariance (Informal)). *In the infinite-width Neural Tangent Kernel (NTK) regime, applying a coordinate-wise PLE map before a rotationally invariant downstream kernel learner can break raw-space rotational invariance. We prove this by an explicit kernel-regression counterexample.*

This is consistent with the empirical observations in (Gorishniy et al., 2022; 2025). See App. D Prop. D.4 for the rigorous counterexample and the subsequent discussion for the general encoder-equivariance intuition.

(3) Capacity for Localized Irregularities. Beyond breaking invariance, we mathematically establish that PLE supplies a fixed, axis-aligned spline basis for numerical features, making it well suited to localized univariate irregularities common in tabular data (proof in App. E):

Proposition 6.2 (PLE Expressivity (Informal)). *Allocating T_i segments for feature x_i via PLE turns scalar linear models on the encoded feature into T_i -segment linear splines. The knot locations are fixed before learning, shifting the burden from discovering breakpoints to learning per-bin slopes; downstream parameter counts may still scale with the encoded dimension.*

Sec. 6.1 empirically validates the benefits of PLE. See App. E Thm. E.7, Prop. E.11, and Prop. E.21 for a rigorous

statement and detailed proofs.

(4) Processing Heterogeneous Inputs. Tabular variables often originate from distinct sensing modalities or abstraction levels. While Transformers and MLPs indiscriminately blend these inputs via early linear projections, *LassoFlexNet* utilizes independent PFE modules. Because the downstream Mixer requires a homogeneous vector space, end-to-end backpropagation structurally forces the individual PFE blocks to map highly heterogeneous raw inputs into a homogeneous and normalized latent space.

(5) Stabilizing Training Dynamics. While proximal gradient operators grant robustness to irrelevant features, they can destabilize stochastic optimization. To diagnose this failure mode, App. F analyzes idealized Hier-Prox dynamics (without the full neural net training dynamics with Adam/EMA implementation):

Proposition 6.3 (Hier-Prox Hard-Constraint Stochastic Dynamics Bound (Informal, see App. F Thm. F.31 and Thm. F.34)). *Under standard stochastic optimization regularity assumptions, with $\rho < 1$, LassoNet’s Hier-Prox dynamics satisfy the diagnostic bound:*

$$\begin{aligned} \mathbb{E}[F(y_t) - F(y^*)] &\leq \rho^t [F(y_0) - F(y^*)] \\ &\quad + C_{\text{Noise}} \eta^2 \sigma_{\theta, W_1}^2 \\ &\quad + C_{\text{Sign Flip}} \mathbb{P}[\text{Jump}_t]. \end{aligned}$$

The first two terms are standard contraction and stochastic-noise contributions; the hard constraint adds the $C_{\text{Sign Flip}} \mathbb{P}[\text{Jump}_t]$ jump term. This matters for *LassoFlexNet* because β acts on evolving PFE representations, where high-variance updates can increase sign flips.

We next analyze a tractable *convex relaxation* that removes this instability channel (App. G):

Proposition 6.4 (Hier-Prox Convex Relaxation Stochastic Dynamics Bound (Informal, see App. G Prop. G.9 and Thm. G.20)). *Under analogous assumptions for the convex relaxation, the corresponding stochastic dynamics satisfy:*

$$\begin{aligned} \mathbb{E}[F(y_t) - F(y^*)] &\leq \rho^t [F(y_0) - F(y^*)] \\ &\quad + C_{\text{Noise}} \eta^2 \sigma_{\theta, W_1}^2. \end{aligned}$$

The relaxation removes the jump term through gate-first shrinkage: skip layer gates and rows remain coupled, but row weights no longer set skip layer gate magnitude through the sorted hard projection.

Thus the hard constraint enforces sparsity, while the relaxation gives a cleaner bound. Seq-Hier-Prox-EMA keeps the hard ℓ_∞ constraint but transfers the relaxation’s gate-first structure back as a heuristic, reducing dense first-layer weights’ tendency to wash out the skip signal.

Remark 6.5. These stylized propositions do not prove end-to-end convergence for Seq-Hier-Prox-Adam-EMA; they identify a high-variance sign-flip failure mode beyond LassoNet’s local result and motivate the sequential, EMA-smoothed design which we empirically validate in Section 6.1.

6.1. Ablation Study over Architecture and Optimization Design Choices

We conduct an ablation study on the California Housing dataset to evaluate *LassoFlexNet*’s components, reporting RMSE (lower is better) in Table 2.

Table 2. Ablation study on the California Housing dataset. Ablations done on Linear LassoNet and *LassoFlexNet* regression. Each value is reported as mean $\pm$ std (min, max) over 10 runs with different seeds.

Ablation	Test RMSE
(Linear) LassoNet	
Baseline Linear LassoNet	0.5263 $\pm$ 0.0103 (0.5100, 0.5362)
Adam with HyperGradient Descent (HGD)	0.5294 $\pm$ 0.0088 (0.5073, 0.5352)
Adam	0.5291 $\pm$ 0.0082 (0.5090, 0.6044)
LassoFlexNet	
Baseline LassoFlexNet	0.4047 $\pm$ 0.0069 (0.3954, 0.4180)
$\tau = 1$	0.4004 $\pm$ 0.0095 (0.3812, 0.4136)
Adam Warmup + Nesterov	0.4050 $\pm$ 0.0089 (0.3874, 0.4190)
Adam with HGD	0.4089 $\pm$ 0.0077 (0.3934, 0.4183)
Hybrid Adaptive λ	0.4109 $\pm$ 0.0129 (0.3967, 0.4314)
Linear PFE Layers	0.4145 $\pm$ 0.0068 (0.4064, 0.4241)
Learnable BatchNorm Layers	0.4191 $\pm$ 0.0096 (0.4034, 0.4357)
No EMA	0.4195 $\pm$ 0.0108 (0.4010, 0.4435)
$\tau = 1$, no skip (Mixer only)	0.4221 $\pm$ 0.0117 (0.4027, 0.4325)
Original Proximal Grad. w/ no EMA	0.4320 $\pm$ 0.0097 (0.4208, 0.4532)
Adam w/ no Proximal Gradient	0.4272 $\pm$ 0.0098 (0.4146, 0.4516)
No PLE	0.4387 $\pm$ 0.0124 (0.4086, 0.4538)
Linear LassoNet + PLE	0.5226 $\pm$ 0.0098 (0.5050, 0.5343)
No PFE	0.5371 $\pm$ 0.0166 (0.5144, 0.5770)
MLP 1 (same width/depth) with HGD	0.4768 $\pm$ 0.0079 (0.4649, 0.4859)
MLP 2 (same # params) with HGD	0.4548 $\pm$ 0.0101 (0.4338, 0.4655)
MLP 1 (same width/depth)	0.8453 $\pm$ 0.2016 (0.5815, 1.0330)
MLP 2 (same # params)	0.8406 $\pm$ 0.2009 (0.5815, 1.0329)

The full model achieves an RMSE of 0.4047 (on average). Removing EMA from Seq-Hier-Prox increases RMSE to 0.4195, confirming EMA’s role in stabilizing training. Replacing the Mixer with an MLP backbone of similar depth (MLP 1) or similar numbers of parameters (MLP 2) yields RMSE of 0.8453, and 0.8406, respectively. This instability is remedied substantially by the inclusion of HGD, but is still not competitive with LassoFlex. Replacing Seq-Hier-Prox with LassoNet’s Hier-Prox harms performance (RMSE 0.4320), as the joint optimization undermines the linear skip connection; we visualize this failure mode—and how Seq-Hier-Prox preserves the pre-trained skip-weight separation while driving $W^{(1)}$ to clean sparsity—in App. C.1 (Figs. 10 and 11). Using Adam without proximal gradients yields 0.4272, highlighting the necessity of proximal methods for feature selection. Setting $\tau = 1$ (disabling curriculum learning) marginally improves RMSE to 0.4004, although the variance in this regime is higher. Using learnable Batch-

Norm causes overfitting (0.4191). The Mixer alone (with HGD) performs poorly (0.4221), and the original LassoNet (with Adam or HGD) yields 0.5294 and 0.5291, respectively. Each component—EMA, Seq-Hier-Prox, fixed BatchNorm and the Mixer component contributes significantly to *LassoFlexNet*’s performance.

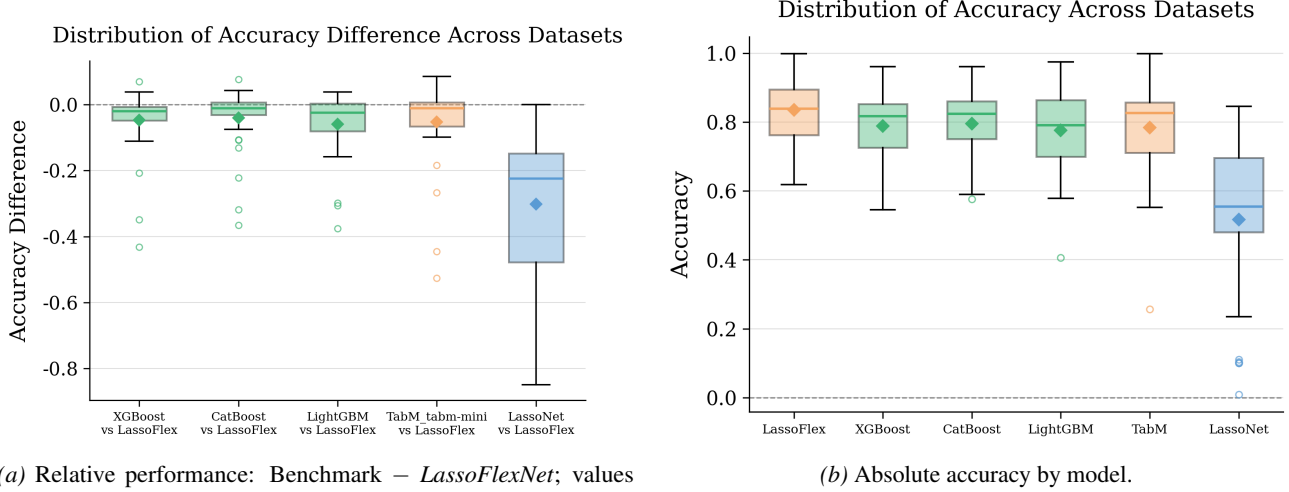
Multi-Dataset Robustness. While Table 2 focuses on California Housing, these architectural benefits generalize broadly. As detailed in Appendix B.3, our hyperparameter sensitivity analysis across 39 benchmarks demonstrates extremely narrow performance distributions (standard deviations typically < 0.02). Furthermore, Appendix B.2 evaluates the λ -training improvements across 300 models on three datasets, confirming that Seq-Hier-Prox consistently improves over pre-training across diverse domains.

7. Comprehensive Benchmarking

We evaluate *LassoFlexNet* across 52 datasets spanning three rigorous benchmarks: a feature selection suite (Cherepanova et al., 2023a), TabZilla (McElfresh et al., 2023), and TabRed (Rubachev et al., 2025). Across these diverse regimes, *LassoFlexNet* consistently outperforms state-of-the-art deep tabular models (e.g., FT-Transformer (Gorishniy et al., 2021), TabM (Gorishniy et al., 2025)) and gradient-boosted trees (e.g., XGBoost (Chen & Guestrin, 2016), CatBoost (Prokhorenkova et al., 2018)). Comprehensive experimental settings are detailed in Appendix C.2. Additional analyses regarding hyperparameter robustness and the generalization impact of Seq-Hier-Prox are provided in Appendices B.3 and B.2, respectively.

7.1. Feature Selection Under Noise

We first assess the capability of *LassoFlexNet* to isolate relevant signals amidst severe noise. We utilize the feature selection benchmark from Cherepanova et al. (2023a), which corrupts 12 standard datasets by injecting either purely random noise features or highly correlated second-order features at extraneous rates of 50% and 75%. Full per-dataset tables (random noise: Table 7; correlated noise: Table 8) are reported in App. C.3. **Baselines and Protocol.** We compare our end-to-end architecture against traditional sequential “two-stage” pipelines. In these baselines, a dedicated feature selection algorithm (e.g., Lasso, LassoNet, XGBoost) first filters the inputs before passing them to a downstream deep model (MLP or FT-Transformer). Crucially, we evaluate these baselines in an *oracle setting*: they are explicitly provided with the ground-truth number of relevant features (k). In stark contrast, *LassoFlexNet* operates blindly, autonomously discovering the sparse support set during training without any prior knowledge of k . **Results.** Despite this severe informational disadvantage, *LassoFlexNet* consistently outperforms the oracle two-stage pipelines. On the



(a) Relative performance: Benchmark — *LassoFlexNet*; values below zero indicate *LassoFlexNet* outperforms the competitor.

(b) Absolute accuracy by model.

Figure 2. Performance distribution of 6 benchmark methods compared to *LassoFlexNet* across 35 TabZilla datasets. Box-plots summarize variance across datasets. In classification tasks, we report accuracy; for regression tasks, we report adjusted R^2 score. In this way, we have a visually clean comparison among the tasks.

random noise benchmark, it achieves top-3 performance in 8/12 datasets at the 50% noise rate and 7/12 at the 75% rate. This confirms that selecting features based on learned, nonlinear representations via a Tied Group Lasso is more robust than linear pre-filtering.

Quantitative Feature Recovery and Stability. To directly validate the interpretability and stability of the learned Lasso gates, we conducted explicit feature-recovery experiments. We augmented the California Housing and House 16H datasets with 50% pure noise features across 5 random seeds. We evaluated whether the magnitude of the learned skip weights ($|\beta|$) could reliably distinguish true features from noise. For House 16H (16 real features), *LassoFlexNet* achieved an AUC-ROC of 0.92 ± 0.045 , Average Precision of 0.94 ± 0.030 , and pairwise rank correlation ρ of 0.73 ± 0.069 . For California Housing (8 real features), it achieved an AUC-ROC of 0.84 ± 0.045 , Average Precision of 0.88 ± 0.032 , and ρ of 0.71 ± 0.103 . These metrics confirm that *LassoFlexNet* provides highly reliable and stable true-feature recovery across different optimization trajectories.

7.2. TabZilla Performance Benchmarks

Benchmark and Baselines. To evaluate robustness across highly diverse data distributions, we utilize the TabZilla benchmark suite (McElfresh et al., 2023). From its collection of the 36 hardest classification tasks, we select the subset of 35 datasets containing numerical or mixed features. The single excluded dataset contains only categorical features, which falls outside the current architectural scope of PLE. We compare *LassoFlexNet* against

five formidable competitors, representing both the gradient-boosted decision tree (GBDT) family (XGBoost (Chen & Guestrin, 2016), CatBoost (Prokhorenkova et al., 2018), LightGBM (Ke et al., 2017)) and state-of-the-art deep tabular model (TabM (Gorishniy et al., 2025)). **Protocol and Results.** We adopt a rigorous two-stage evaluation protocol. For each dataset, we first perform a hyperparameter search on a single fold, selecting the configuration that minimizes validation log-loss on a hold-out split. Using these fixed hyperparameters, we then report the final performance across a separate 5-fold cross-validation. Figure 2a illustrates the distribution of accuracy differentials between the baselines and our method. Because the y -axis represents (Benchmark Accuracy — *LassoFlexNet* Accuracy), values below zero indicate that *LassoFlexNet* outperformed the competitor. The results demonstrate consistent superiority: *LassoFlexNet* outperforms all five baselines in the majority of tasks, exhibiting a robust downward shift in the error distributions. Per-dataset raw performance metrics, the Friedman omnibus test, and pairwise Wilcoxon signed-rank tests (Holm–Bonferroni corrected) are deferred to Appendix C.5.

7.3. TabRed: Robustness to Distribution Shift

Benchmark and Protocol. Standard tabular benchmarks often rely on i.i.d. shuffling, which masks the temporal distribution shifts prevalent in real-world applications. To assess robustness under these challenging conditions, we evaluate *LassoFlexNet* on the TabRed benchmark (Rubachev et al., 2025). Adopting the TabZilla protocol, we aggregate performance across six splits per dataset: three randomized splits and three strict temporal splits (where the training data strictly precedes the test data in time). This rigorously

Table 3. TabRed Benchmark Results against leading non-foundation tabular models.

Dataset \ Model	CatBoost	LightGBM	RF	XGBoost	TabM-mini	LassoFlexNet
ecom-offers (class. AUC)	0.6507 $\pm$ 0.0554 [6]	0.6317 $\pm$ 0.0538 [6]	0.6447 $\pm$ 0.0529 [6]	0.6447 $\pm$ 0.0628 [6]	0.6533 $\pm$ 0.0603 [6]	0.6322 $\pm$ 0.0419 [6]
homecredit-default (class. AUC)	0.8211 $\pm$ 0.0115 [6]	0.8092 $\pm$ 0.0110 [6]	0.7365 $\pm$ 0.0168 [6]	0.8046 $\pm$ 0.0136 [6]	0.8413 $\pm$ 0.0102 [6]	0.8365 $\pm$ 0.0096 [6]
homesite-insurance (class. AUC)	0.9559 $\pm$ 0.0088 [6]	0.9576 $\pm$ 0.0075 [6]	0.8414 $\pm$ 0.0168 [6]	0.9568 $\pm$ 0.0088 [6]	0.9557 $\pm$ 0.0083 [6]	0.9605 $\pm$ 0.0071 [6]
cooking-time (reg. RMSE)	0.4718 $\pm$ 0.0084 [6]	0.4899 $\pm$ 0.0095 [6]	0.5192 $\pm$ 0.0062 [6]	0.4866 $\pm$ 0.0108 [6]	0.5629 $\pm$ 0.0157 [6]	0.4658 $\pm$ 0.0093 [6]
delivery-eta (reg. RMSE)	0.5370 $\pm$ 0.0094 [6]	0.5641 $\pm$ 0.0095 [6]	0.5701 $\pm$ 0.0079 [6]	0.5425 $\pm$ 0.0086 [6]	0.6121 $\pm$ 0.0057 [5]	0.5375 $\pm$ 0.0104 [6]
maps-routing (reg. RMSE)	0.1665 $\pm$ 0.0015 [6]	0.1641 $\pm$ 0.0015 [6]	0.1746 $\pm$ 0.0012 [6]	0.1709 $\pm$ 0.0018 [6]	0.1656 $\pm$ 0.0021 [6]	0.1619 $\pm$ 0.0013 [6]
sberbank-housing (reg. RMSE)	0.2585 $\pm$ 0.0171 [6]	0.2568 $\pm$ 0.0171 [6]	0.2908 $\pm$ 0.0141 [6]	0.2761 $\pm$ 0.0254 [6]	0.2769 $\pm$ 0.0098 [5] ¹	0.2544 $\pm$ 0.0138 [6]
weather (reg. RMSE)	1.5922 $\pm$ 0.0565 [6]	1.6272 $\pm$ 0.0735 [6]	2.4514 $\pm$ 0.0994 [6]	1.5632 $\pm$ 0.0766 [6]	2.2434 $\pm$ 0.1835 [6]	1.5591 $\pm$ 0.0764 [6]

¹ 5 seeds because one seed leads to unstable training.

tests the model’s ability to generalize to future, shifted distributions. **Results.** Table 3 details the performance of *LassoFlexNet* against leading GBDT methods and the state-of-the-art neural model TabM-mini. *LassoFlexNet* establishes a new state-of-the-art, achieving the best performance in 5 out of 8 tasks. In the remaining tasks, it remains highly competitive; for instance, it trails the top performer (CatBoost) by a negligible margin of 0.0005 RMSE in *delivery-eta*, and ranks second in *homecredit-default*. This consistent performance across both randomized and temporal splits highlights *LassoFlexNet*’s superior capacity to handle real-world tabular heterogeneity and non-stationarity.

7.4. Comparison with Tabular Foundation Models

To contextualize *LassoFlexNet* against the emerging paradigm of pre-trained in-context learners, we benchmarked it against the latest tabular foundation models: TabPFN (v2.6) (Grinsztajn et al., 2025) and TabICL (v2) (QU et al., 2025). We evaluated these models on the large-scale TabRed distribution-shift datasets. To ensure a fair evaluation, we followed the official TabPFN guidelines, utilizing their recommended subsampling and ensembling scripts to handle the scale of TabRed (which contains up to $\sim 600\text{K}$ samples and 1000+ features).

As shown in Table 4, *LassoFlexNet* outperforms both TabPFN and TabICL on 5 of the 8 TabRed datasets (*homesite-insurance*, *cooking-time*, *delivery-eta*, *maps-routing*, *weather*); TabPFN takes the lead on the remaining three (*sberbank-housing*, *ecom-offers*, *homecredit-default*, with TabICL matching on *homecredit-default*). Crucially, *LassoFlexNet* achieved these results using only 40 Optuna tuning trials—a tiny fraction of the foundation models’ training and inference costs—and trains with mini-batch SGD, sidestepping the $\mathcal{O}(n^2)$ context-length memory bottleneck that forces TabPFN and TabICL to subsample and ensemble at TabRed scale. These results validate that *LassoFlexNet*’s

architectural inductive biases remain highly competitive with, and often surpass, massive pre-trained foundation models, especially under distribution shifts and large sample sizes.

 Table 4. Performance on TabRed distribution-shift datasets against Tabular Foundation Models (mean $\pm$ std over 6 splits).

Dataset	Metric	LassoFlexNet	TabPFN (v2.6)	TabICL (v2)
Sberbank Housing	RMSE ($\downarrow$)	0.2544 $\pm$ 0.014	0.2424$\pm$0.016	0.2691 $\pm$ 0.046
Ecom Offers	AUC ($\uparrow$)	0.6322 $\pm$ 0.042	0.7069$\pm$0.050	0.6923 $\pm$ 0.040
Homecredit Default	AUC ($\uparrow$)	0.8365 $\pm$ 0.010	0.9670$\pm$0.007	0.9670$\pm$0.007
Homesite Insurance	AUC ($\uparrow$)	0.9605$\pm$0.007	0.9207 $\pm$ 0.004	0.9081 $\pm$ 0.007
Cooking Time	RMSE ($\downarrow$)	0.4658$\pm$0.009	0.4673 $\pm$ 0.008	0.4675 $\pm$ 0.009
Delivery ETA	RMSE ($\downarrow$)	0.5375$\pm$0.010	0.5407 $\pm$ 0.009	0.5405 $\pm$ 0.008
Maps Routing	RMSE ($\downarrow$)	0.1619$\pm$0.001	0.1637 $\pm$ 0.001	0.1658 $\pm$ 0.001
Weather	RMSE ($\downarrow$)	1.5591$\pm$0.076	1.6006 $\pm$ 0.047	1.5941 $\pm$ 0.063

Structural Complexity. Beyond raw performance, *LassoFlexNet* achieves $\mathcal{O}(d)$ feature scaling, $\mathcal{O}(\text{batch})$ memory, and is unique among non-Lasso tabular methods in providing native end-to-end feature selection over nonlinear embeddings. App. C.4 (Table 9) provides a full comparison against representative deep, GBDT, and foundation-model baselines.

8. Conclusion

We introduced *LassoFlexNet*, a novel neural architecture accompanied by a novel optimization algorithm that sets the new state-of-the-art for tabular deep learning. The method is designed specifically to address five challenges: (i) resilience to irrelevant features; (ii) respect for data orientations; (iii) the ability to learn complex and irregular functions; (iv) the capability to model a larger number of heterogeneous inputs; and (v) stability and robustness in training. *LassoFlexNet* bridges the gap between tree-based and neural methods, offering a practical, scalable solution for tabular data.

Acknowledgements

We thank Mahdi Gilany for improving the codebase, Jeff Lee and Guy Morgenshtern for code review, and Raquel Aoki for helpful comments throughout. We are grateful to Tristan Milne for identifying the original loose upper bound on PLE’s expressivity, which motivated the present expressivity treatment. We also thank the broader RBC Borealis community for enthusiastic discussions.

Impact Statement

The native feature interpretability provided by *LassoFlexNet*’s Tied Group Lasso directly aids model auditing in high-stakes settings (e.g., healthcare, finance). However, we caution against over-reliance on any single importance metric without domain expertise.

References

- Abid, A., Baln, M. F., and Zou, J. Concrete autoencoders: Differentiable feature selection and reconstruction. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 444–453. PMLR, 2019.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.
- Arik, S. Ö. and Pfister, T. TabNet: Attentive interpretable tabular learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 6679–6687, 2021.
- Blaette, A. et al. Better by default: Strong pre-tuned mlps and boosted trees on tabular data. In *NeurIPS*, 2024.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Chen, T. and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- Cherepanova, V., Levin, R., Somepalli, G., Geiping, J., Bruss, C., Wilson, A., Goldstein, T., and Goldblum, M. A performance-driven benchmark for feature selection in tabular deep learning. In *NeurIPS 2023 Second Table Representation Learning Workshop*, 2023a. URL <https://openreview.net/forum?id=PxEY7pBb6F>.
- Cherepanova, V., Levin, R., Somepalli, G., Geiping, J., Bruss, C. B., Wilson, A. G., Goldstein, T., and Goldblum, M. A performance-driven benchmark for feature selection in tabular deep learning. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 41956–41979. Curran Associates, Inc., 2023b.
- Erickson, N. et al. Tabarena: A benchmark for tabular data. *arXiv preprint*, 2025.
- Gorishniy, Y., Rubachev, I., Khrulkov, V., and Babenko, A. Revisiting deep learning models for tabular data. *Advances in neural information processing systems*, 34: 18932–18943, 2021.
- Gorishniy, Y., Rubachev, I., and Babenko, A. On embeddings for numerical features in tabular deep learning. In *NeurIPS*, 2022.
- Gorishniy, Y., Rubachev, I., Kartashev, N., Shlenskii, D., Kotelnikov, A., and Babenko, A. Tabr: Tabular deep learning meets nearest neighbors. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=rhgIgTSSxW>.
- Gorishniy, Y., Kotelnikov, A., and Babenko, A. Tabm: Advancing tabular deep learning with parameter-efficient ensembling. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=Sd4wYYOhmY>.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems*, 35:507–520, 2022.
- Grinsztajn, L., Flöge, K., Key, O., Birkel, F., Jund, P., Roof, B., Jäger, B., Safaric, D., Alessi, S., Hayler, A., Manium, M., Yu, R., Jablonski, F., Hoo, S. B., Garg, A., Robertson, J., Bühler, M., Moroshan, V., Purucker, L., Cornu, C., Wehrhahn, L. C., Bonetto, A., Schölkopf, B., Gambhir, S., Hollmann, N., and Hutter, F. TabPFN-2.5: Advancing the state of the art in tabular foundation models, 2025. URL <https://arxiv.org/abs/2511.08667>.
- Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, NY, 2 edition, 2009.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

- Hollmann, N., Müller, S., Eggensperger, K., and Hutter, F. TabPFN: A transformer that solves small tabular classification problems in a second. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- Izmailov, P., Podoprikin, D., Garipov, T., Vetrov, D., and Wilson, A. G. Averaging weights leads to wider optima and better generalization. In *Proceedings of the Thirty-Fourth Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 876–885, 2018.
- Jacot, A., Gabriel, F., and Hongler, C. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pp. 3146–3154, 2017.
- Lemhadri, I., Ruan, F., Abraham, L., and Tibshirani, R. Lassonet: A neural network with feature sparsity. *Journal of Machine Learning Research*, 22(127):1–29, 2021a.
- Lemhadri, I., Ruan, F., Abraham, L., and Tibshirani, R. Lassonet: A neural network with feature sparsity. *Journal of Machine Learning Research*, 22(127):1–29, 2021b.
- McElfresh, D., Khandagale, S., Valverde, J., Prasad C, V., Feuer, B., Hegde, C., Ramakrishnan, G., Goldblum, M., and White, C. When do neural nets outperform boosted trees on tabular data? In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 36, 2023.
- Ng, A. Y. Feature selection, l_1 vs. l_2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.
- Parikh, N., Boyd, S., et al. Proximal algorithms. *Foundations and trends® in Optimization*, 1(3):127–239, 2014.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 2018.
- QU, J., Holzmüller, D., Varoquaux, G., and Morvan, M. L. TabICL: A tabular foundation model for in-context learning on large data. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=0VvD1PmNzM>.
- Rubachev, I., Kartashev, N., Gorishniy, Y., and Babenko, A. Tabred: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=L14sqcrUC3>.
- Somepalli, G. et al. Saint: Improved neural networks for tabular data via row attention and contrastive pre-training. *arXiv preprint arXiv:2106.01342*, 2021.
- Sudo, A., Hou, T. T., Yamaguchi, M., and Tone, Y. Lasso-layer: Nonlinear feature selection by switching one-to-one links. *arXiv preprint arXiv:2108.12165*, 2021.
- Thielmann, A. F. et al. Mambular: A sequential model for tabular deep learning. *arXiv preprint arXiv:2408.06291*, 2024.
- Tolstikhin, I. O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., et al. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems*, 34:24261–24272, 2021.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Yamada, Y., Lindenbaum, O., Negahban, S., and Kluger, Y. Feature selection using stochastic gates. In *International Conference on Machine Learning*, pp. 10648–10659. PMLR, 2020.

A. Limitations

While our evaluation is extensive, we acknowledge several limitations:

- **Wall-Clock Benchmarking:** We do not provide explicit wall-clock training time comparisons against highly engineered libraries like XGBoost. However, *LassoFlexNet* scales linearly in features ($\mathcal{O}(d)$) and operates with constant memory w.r.t. dataset size ($\mathcal{O}(\text{batch})$), natively supporting millions of rows on a single GPU.
- **Architectural Scope:** We restrict the evaluation to numerical or mixed-type datasets due to Piecewise Linear Encoding (PLE). Datasets comprising exclusively categorical features require extra experimentation. Extending the architecture and integrating it with large-scale pre-training paradigms, are natural directions for future work.
- **Limitations of Theoretical Analysis:** Our theoretical propositions (Theorems 6.3 and 6.4) act as rigorous diagnostic tools to isolate the jump-discontinuity variance in the original Hier-Prox method. This goes beyond LassoNet’s local proximal optimality characterization by analyzing stochastic training dynamics, but it does not constitute an end-to-end formal convergence proof for the full Seq-Hier-Prox-Adam-EMA deep learning optimizer, which remains an open problem in the optimization community.

B. Appendix: Background and More Experiments

B.1. LassoNet Training Algorithm

Algorithm 3 LassoNet Hierarchical Proximal Operator (Lemhadri et al., 2021b)

```

1: procedure HIER-PROX( $\theta, W^{(1)}; \lambda, M$ )
2:   for  $j = 1$  to  $d$  do
3:     Sort  $|W_j^{(1)}|$  into  $|W_{(j,1)}^{(1)}| \geq \dots \geq |W_{(j,K)}^{(1)}|$ 
4:     for  $m = 0$  to  $K$  do
5:        $w_m = \frac{M}{1+m M^2} \mathcal{S}_\lambda \left( |\theta_j| + M \sum_{i=1}^m |W_{(j,i)}^{(1)}| \right)$ 
6:       Find first  $\tilde{m} : |W_{(j,\tilde{m}+1)}^{(1)}| \leq w_{\tilde{m}} \leq |W_{(j,\tilde{m})}^{(1)}|$ 
7:        $\tilde{\theta}_j = \frac{1}{M} \text{sign}(\theta_j) w_{\tilde{m}}$ 
8:        $\tilde{W}_j^{(1)} = \text{sign}(W_j^{(1)}) \min(w_{\tilde{m}}, |W_j^{(1)}|)$ 
9:   return  $(\tilde{\theta}, \tilde{W}^{(1)})$ 
    
```

Notation: d is the number of features; K is first-layer width.

B.2. λ -Training Improvements Over Pre-Trained Models

In this section, we evaluate the additional generalization gains provided by the λ -training stage (utilizing our proposed Seq-Hier-Prox-EMA optimizer) compared to the baseline pre-trained models. For each dataset (GE, EY, and CH), we randomly initialized 300 LassoFlexNet models with diverse hyperparameter configurations. We first pre-trained each model for 200 epochs, followed by a λ -training phase of 100 epochs at each λ value (both utilizing early stopping). We then compared the validation metrics of the final λ -trained models against their pre-trained counterparts. The results are visualized in Figure 3.

Our empirical results across both classification and regression tasks demonstrate that the vast majority of data points lie above the $y = x$ line (or to the left, depending on the metric orientation), indicating a consistent, strict improvement in validation metrics over the pre-trained baseline. Points situated directly on the $y = x$ identity line correspond to specific hyperparameter configurations where λ -training yielded no marginal gain; in these instances, the early-stopping mechanism simply reverted the final model to the pre-trained weights.

This consistently positive delta demonstrates the efficacy of our λ -training phase and the stability of the proposed Seq-Hier-Prox-EMA algorithm. Furthermore, because λ -training relies entirely on the learned skip weights β accurately capturing feature importance to constrain the neural module $W^{(1)}$, this experiment implicitly confirms that β is successfully learned during the pre-training stage, validating the end-to-end LassoFlexNet pipeline.

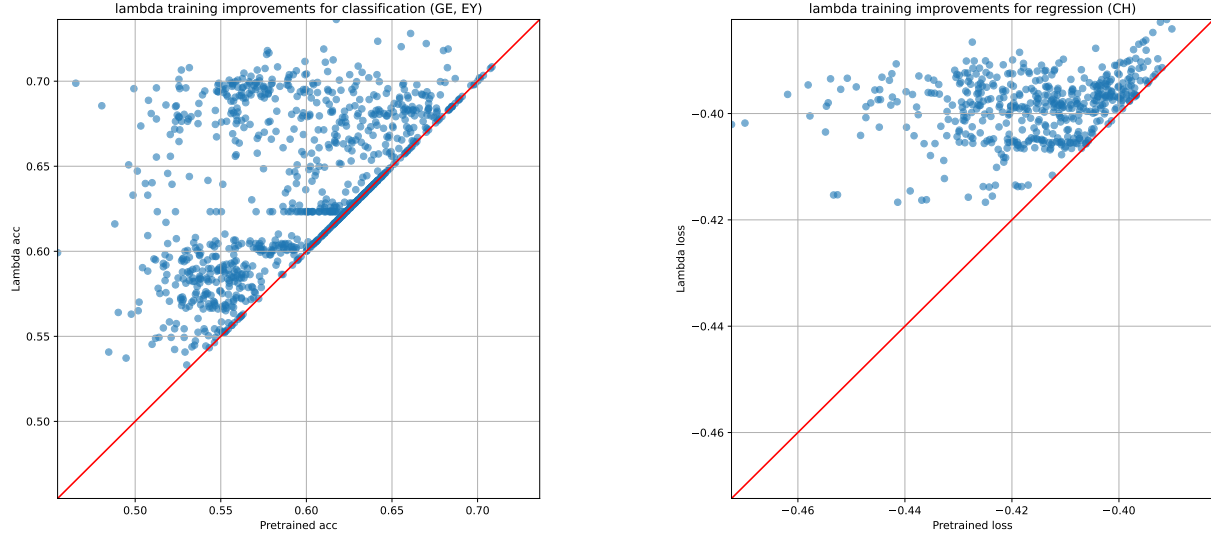


Figure 3. Validation performance comparison between 300 λ -trained models with its corresponding pre-trained model: GE, EY classification tasks (left, higher acc means better, so **left of $y = x$ means λ improves**); CH regression (right, lower the loss magnitude means better, so **left of $y = x$ means λ improves**).

Figure 4 provides a detailed trajectory of LassoFlexNet training on the California Housing dataset. Training is governed by early stopping based on validation loss. The pre-training phase halts at the epoch indicated by the red vertical line, after which the λ -training phase activates and continues to drive the validation loss down, even as the training loss naturally rises due to the increasing ℓ_1 penalty from the λ -training (Seq-Hier-Prox-EMA). The EMA validation loss comes from the EMA of the model weights ensemble. It further lowers the validation loss by smoothing out the noise from the model weights ensemble, similar to EMA in diffusion models (Ho et al., 2020).

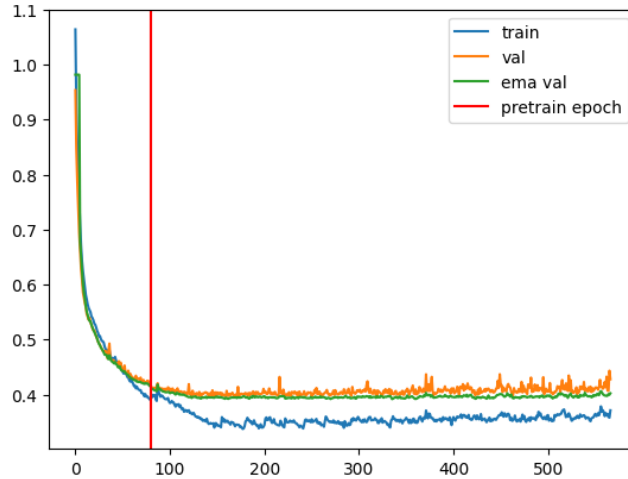


Figure 4. LassoFlexNet model improves generalization gap. Validation continues to go down, as training loss goes up.

B.3. LassoFlexNet Hyperparameter Sensitivity Analysis

To evaluate the sensitivity of LassoFlexNet to hyperparameter configurations, we conducted extensive experiments across 39 distinct feature selection benchmarks (each representing a unique combination of dataset and noise settings). We restricted

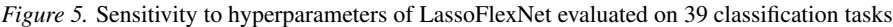


Figure 5. Sensitivity to hyperparameters of LassoFlexNet evaluated on 39 classification tasks

Figure 5 presents the resulting distribution of validation scores via box plots, with red annotations denoting one standard deviation of the validation accuracy for each task. We observe that LassoFlexNet’s performance is highly robust to hyperparameter variations, evidenced by the narrow variance distributions across the vast majority of the benchmarks.

To explicitly test the failure modes of the original LassoNet architecture, we construct an artificial dataset governed by the following generating function:

$$y = \beta_*^\top \mathbf{x} + \alpha \cdot \text{NN}_R(\mathbf{x}) + \gamma \cdot \epsilon \quad (6)$$

where β_* represents the ground-truth linear correlations. Only a sparse subset of elements in β_* are large, while the remainder are exactly or approximately zero. NN_R is a fixed, randomly initialized neural network used to inject non-linearity, and $\epsilon \sim \mathcal{N}(0, 1)$ is standard Gaussian noise. The scalars α and $\gamma \in [0, 1]$ control the strength of the non-linearity and noise, respectively. Notably, by setting $\alpha = 0$, we isolate whether LassoNet can successfully uncover pure linear relations, decoupling the model’s performance from its core theoretical assumption (that linear correlations can adequately constrain nonlinear relations).

The success of LassoNet hinges entirely on the pre-training stage, where the linear skip weights β must converge toward the true sparse support β_* . Only with this accurate foundation can the subsequent λ -training phase effectively utilize β to enforce feature sparsity on the non-linear components.

We trained the original LassoNet model on this artificial dataset with strong linear signals ($\alpha = 0$) and visualized the evolution of the training loss, validation loss, and skip weights over time. The results, averaged over ten random seeds (varying the initialization of β and the neural weights, but keeping β_* fixed), are shown in Figure 6.

As observed in the skip weight evolution plot (Figure 6, right), the linear LassoNet fails to isolate the strong ground-truth linear weights; the trajectories of the important features (black-lines) remain hopelessly entangled with the non-important features (colored lines).

This training dynamic fatally undermines the premise of using skip weights to constrain feature participation. If the skip

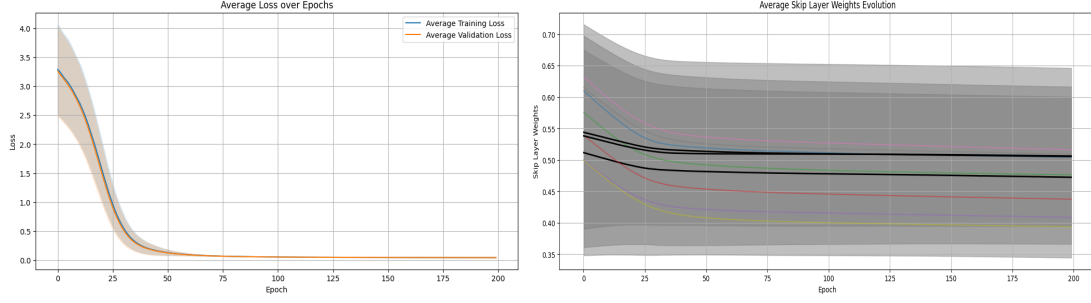


Figure 6. Average loss and skip weights evolution for original LassoNet during training across 10 runs

weights β fail to separate important from unimportant features during pre-training, the subsequent λ -training phase will constrain the hidden layers using corrupted guidance. It is highly unlikely that proximal λ -training can reverse this adverse initialization, rendering the intended end-to-end feature selection ineffective.

Our intuition for this failure is as follows: the linear component of the model, $\beta^\top x$, is a significantly weaker learner than the nonlinear component $\text{NN}_W(x)$. During joint training, the optimization trajectory is dominated by $\text{NN}_W(x)$, preventing $\beta^\top x$ from independently uncovering the ground-truth linear signal. In over-parameterized regimes, the global loss can be minimized even if the skip weights fail to prioritize the correct features. Because there is no structural curriculum enforcing “learn the linear part first, the nonlinear part next”, β ’s ideal gradient share is dispersed into the neural module.

We validate this intuition by modifying the original LassoNet architecture slightly. Instead of $\hat{y} = \beta^\top x + \text{MLP}(x)$, we introduce a small scalar τ such that $\hat{y} = \beta^\top x + \tau \cdot \text{MLP}(x)$. This modification acts as a soft curriculum on the joint learning dynamics, suppressing the MLP’s gradients and encouraging the linear weights to dominate early training. We plot the identical experiment setting $\tau = 0.001$ in Figure 7. Under this soft curriculum, the learned skip weights for the

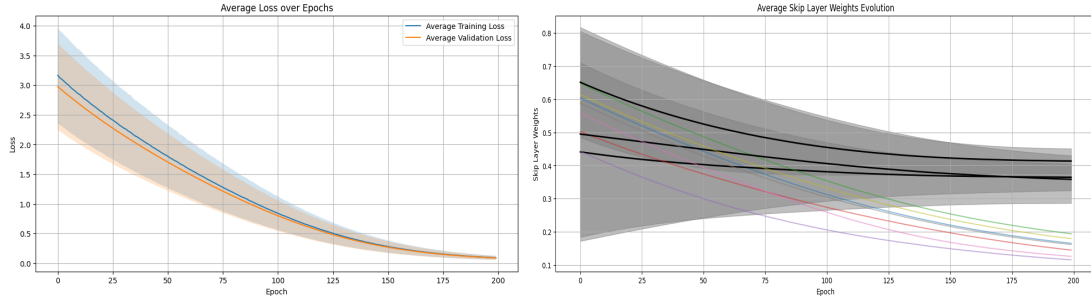


Figure 7. Average loss and skip weights evolution for original LassoNet with small scalar on the MLP part during training across 10 runs

important features cleanly separate from the unimportant ones. Note that the train and validation losses decrease at a slower overall pace with $\tau = 0.001$. This convergence rate difference further corroborates our hypothesis that the unconstrained MLP was previously dominating the optimization dynamics at the expense of the linear skip connection.

C. Appendix: LassoFlexNet Main Module Architecture

MLP-Mixer Modification: Enforcing the Hierarchical Bottleneck. The main neural module of LassoFlexNet utilizes the MLP-Mixer architecture proposed by Tolstikhin et al. (2021). The original MLP-Mixer consists of per-patch linear embeddings, Mixer blocks, and a classifier head. Each Mixer block contains one feature-mixing MLP and one embedding-mixing MLP, each consisting of two fully-connected layers and a GELU nonlinearity, alongside residual connections, dropout, and LayerNorm.

In LassoFlexNet, our Per-Feature Embedding (PFE) module connects directly to the Mixer blocks. Crucially, our Tied Group Lasso skip module constrains the feature-mixing MLP within the *first* Mixer block; the constrained block contains all

outgoing weights from each feature token/PFE embedding into this first feature-mixing MLP. To ensure the skip module strictly gates all information flow—analogous to the first hidden layer in the original LassoNet—we designate the feature-mixing MLP of the first Mixer block as the sole computational bottleneck. Consequently, we explicitly omit the LayerNorm and residual connections in this initial block to prevent any information bypass that would circumvent the hierarchical constraint. This architectural modification is illustrated in Figure 8.

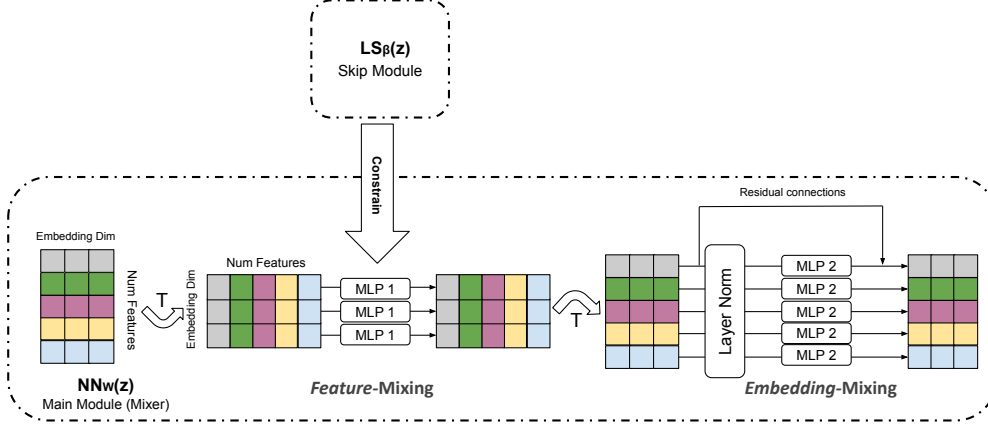


Figure 8. Details of the main module for LassoFlexNet, Mixer architecture is used

C.1. Proximal Gradient Weight Evolution Comparison

In this section, we empirically compare the training dynamics of LassoNet’s original Hier-Prox operator against our proposed Seq-Hier-Prox update. We initialize a pre-trained LassoFlexNet model and iteratively apply either Hier-Prox or Seq-Hier-Prox to update the skip weights β and the first-layer neural weights $W^{(1)}$.

The model is pre-trained on the California Housing dataset augmented with 50% random noise (Cherepanova et al., 2023a). We set the curriculum scalar $\tau = 0.1$ to ensure β is robustly learned during pre-training. For the iterative proximal updates, we use a fixed linear sequence of λ values ranging from 10^{-5} to 10^{-2} . To ensure robust observations, we sample different values of the constraint multiplier M and inject Gaussian noise into the loaded pre-trained weights across multiple random seeds. We record the evolution paths for β and $W^{(1)}$ for each seed and report the average trajectory.

The loaded pre-trained skip weights (β) and first hidden layer weights ($W^{(1)}$) are visualized in Figure 9.

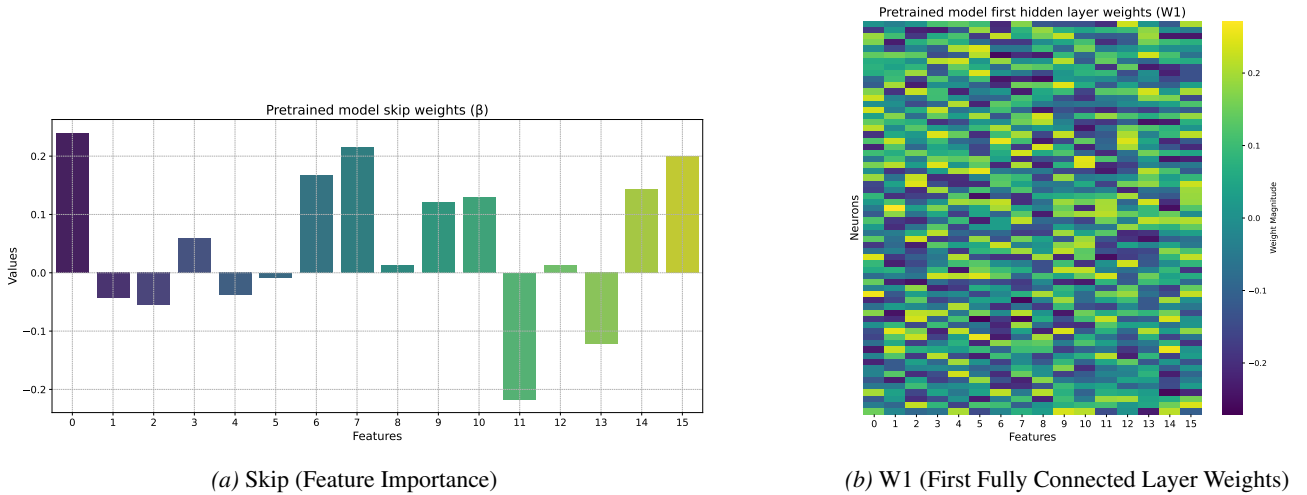


Figure 9. Visualizations of pre-trained LassoFlexNet model skip weights and W1 weights

As shown, the pre-trained skip weights successfully capture feature importance, exhibiting distinct magnitudes (some close

to zero, others large) that separate the true features from the injected noise. Conversely, the distribution of the hidden weights $W^{(1)}$ remains relatively uniform. The objective of the subsequent λ -training is to utilize these informative skip weights to constrain $W^{(1)}$, ensuring that unimportant features do not participate in the neural network’s computations.

Failure of Original Hier-Prox. The average evolution of β and $W^{(1)}$ under the original Hier-Prox operator is visualized in Figure 10. Before any proximal updates (iteration 0), the skip weights are cleanly separated. However, the moment the joint proximal gradient update begins, the skip weights β become heavily influenced by the dense, noisier first hidden layer weights $W^{(1)}$. Consequently, the ability of β to guide $W^{(1)}$ toward sparsity is severely weakened, and the critical feature importance learned during the pre-training stage is rapidly washed away.

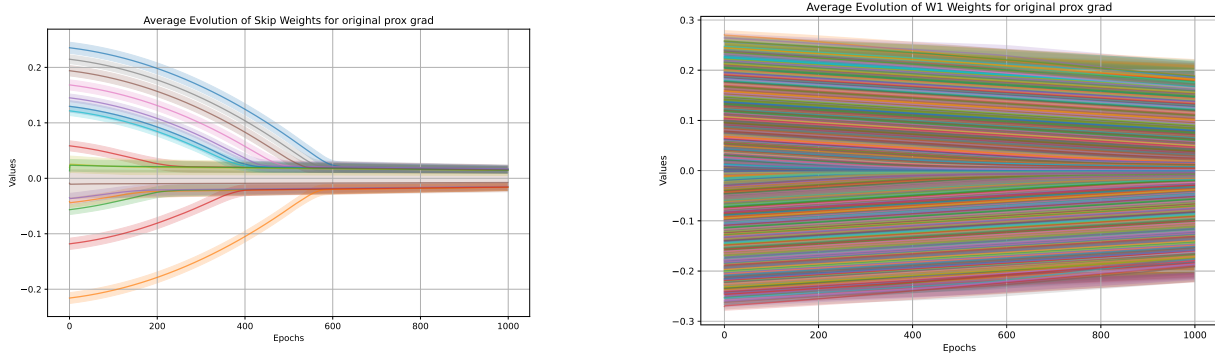


Figure 10. Average evolution of skip weights (left) and W1 hidden layer weights (right) over original Hier-Prox updates

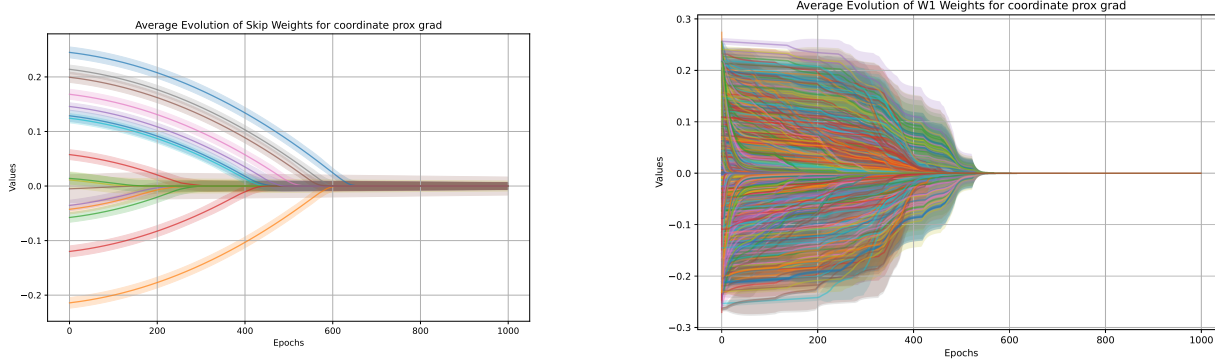


Figure 11. Average evolution of skip weights (left) and W1 hidden layer weights (right) over Seq-Hier-Prox updates

Success of Seq-Hier-Prox. We next examine the evolution under LassoFlexNet’s Seq-Hier-Prox update (Figure 11). Because the sequential update separates the gate update from the first-layer shrinkage step, the skip weights evolve smoothly without any initial jump discontinuities. The λ -training phase effectively leverages the feature importance learned during pre-training. As a result, the first hidden layer weights $W^{(1)}$ become cleanly sparse over time, successfully gating the neural network computations to ignore irrelevant features. Based on these dynamics, we utilize Seq-Hier-Prox-EMA for all experiments in Section 7.

C.2. Experimental Settings for Section 7

In this section, we detail the experimental configurations used in Section 7. For LassoFlexNet, the pre-training stage is set to a maximum of 200 epochs, and the λ -training stage runs for a maximum of 100 epochs per λ value. We employ an early-stopping patience of 20 epochs for both stages. During λ -training, if the validation loss does not improve for 20 epochs at the current λ , the training automatically advances to the next λ value in the sequence.

For **feature selection benchmark** result in Section 7.1, we directly take the result in (Cherepanova et al., 2023a) for all

benchmark methods, and we use exact same setting for getting our LassoFlexNet result: We use the default train, val and test split used in (Cherepanova et al., 2023a), which is [0.65, 0.15, 0.2]; we conduct hyperparameter tuning for LassoFlexNet for 100 iterations using Bayesian optimization engine Optuna (Akiba et al., 2019), and we use the Bayesian sampler’s suggested hyperparameter from iterations to iteration. The objective for this hyperparameters tuning is again the model’s validation metrics. The results presented in Table 7 are model’s test metrics with best hyperparameters, calculated across 10 random model initializations with different seeds. For FT-transformers and MLP, the hyperparameters and their respective ranges remain consistent with those reported in (Cherepanova et al., 2023a). The specific hyperparameters for LassoFlexNet are detailed in Table 5.

For **TabZilla and TabRed benchmark** results in Section 7.2, we again use Bayesian optimization engine Optuna (Akiba et al., 2019) with ASHA (Asynchronous Successive Halving Algorithm). For each algorithm for each dataset, we tune the model only using validation metrics from 1 fold split, and uses this split to tune objective validation loss. For both classification tasks and regression tasks, we use sklearn’s log loss as the validation and test metrics for classification and MSE loss for regression. The box plot results in Figure 2a for each algorithm are best test metrics over 40 hyperparameters tuning trials, averaged across another 5 unseen random splits. For all benchmark methods, we use the same implementations as in (McElfresh et al., 2023).

Hyperparameter	Range	Step/Scale
Mini batch sizes	512–2048	scaled by 2
Number of Mixer blocks	2–5	1
MLP1&2 hidden dimension	64–512	scaled by 2
Feature embedding output dimension	8–32	2
Number of bins used in PLE	4–32	2
τ (scalar multiplied with NN_W)	0.01–2.0	Uniform
ResNet dropout	0.05–0.25	Uniform
Main module dropout	0.05–0.25	Uniform
Number of ResNet layers	2–5	1
ResNet layers dimension	8–32	1
Learning rate (lr)	1×10^{-4} – 1×10^{-3}	Loguniform
AdamW weight decay	1×10^{-6} – 1×10^{-3}	Loguniform

Table 5. Hyperparameters of LassoFlexNet Tuned by Optuna and Their Ranges, in Feature Selection and TabZilla Benchmarks

Hyperparameter	Range	Step/Scale
Mini batch sizes	1024–4096	scaled by 2
Number of Mixer blocks	2–6	1
MLP1&2 hidden dimension	64–512	scaled by 2
Feature embedding output dimension	8–32	2
Number of bins used in PLE	16–64	2
τ (scalar multiplied with NN_W)	0.01–2.0	Uniform
ResNet dropout	0.05–0.5	Uniform
Main module dropout	0.05–0.25	Uniform
Number of ResNet layers	2–8	1
ResNet layers dimension	8–32	1
Learning rate (lr)	1×10^{-5} – 1×10^{-3}	Loguniform
AdamW weight decay	1×10^{-6} – 1×10^{-3}	Loguniform

Table 6. Hyperparameters of LassoFlexNet Tuned by Optuna and Their Ranges, in TabRed Benchmark

C.3. Feature Selection Benchmark: Full Per-Dataset Results

We report full per-dataset performance for both versions of the feature-selection benchmark from Cherepanova et al. (2023a): Table 7 (random noise extras) and Table 8 (correlated second-order extras). The qualitative summary is given in Sec. 7.1 of

the main paper.

%	FS Method	AL	CH	CO	EY	GE	HE	HI	HO	JA	MI	OT	YE
50	No FS + MLP	0.941	-0.480	0.961	0.538	0.466	0.366	0.798	-0.622	0.703	-0.911	0.773	-0.801
	Univariate + MLP	0.960	-0.447	0.970	0.575	0.515	0.379	0.811	-0.549	0.715	-0.891	0.808	-0.776
	Lasso + MLP	0.949	-0.454	0.969	0.547	0.458	0.378	0.812	-0.575	0.715	-0.907	0.805	-0.774
	1L Lasso + MLP	0.952	-0.453	0.969	0.564	0.473	0.380	0.811	-0.568	0.715	-0.897	0.796	-0.773
	AGL + MLP	0.958	-0.512	0.969	0.578	0.495	0.386	0.813	-0.557	0.718	-0.898	0.791	-0.783
	LassoNet + MLP	0.954	-0.445	0.969	0.495	0.383	0.386	0.811	-0.557	0.705	-0.907	0.783	-0.776
	AM + MLP	0.953	-0.444	0.968	0.554	0.498	0.382	0.813	-0.566	0.722	-0.904	0.801	-0.777
	RF + MLP	0.955	-0.444	0.969	0.554	0.498	0.382	0.813	-0.566	0.722	-0.904	0.801	-0.777
	XGBoost + MLP	0.956	-0.448	0.969	0.590	0.502	0.385	0.812	-0.560	0.720	-0.893	0.800	-0.774
	Deep Lasso + MLP	0.959	-0.443	0.968	0.573	0.485	0.383	0.814	-0.549	0.720	-0.894	0.802	-0.776
	No FS + FT	0.959	-0.432	0.966	0.673	0.500	0.384	0.817	-0.577	0.730	-0.902	0.813	-0.792
	Univariate + FT	0.963	-0.424	0.970	0.700	0.519	0.389	0.819	-0.554	0.733	-0.897	0.819	-0.788
	Lasso + FT	0.952	-0.419	0.969	0.703	0.504	0.392	0.817	-0.594	0.728	-0.999	0.820	-0.998
	1L Lasso + FT	0.963	-0.423	0.969	0.722	0.489	0.389	0.819	-0.577	0.732	-0.904	0.819	-0.775
	AGL + FT	0.899	-0.419	0.969	0.701	0.480	0.386	0.822	-0.586	0.733	-0.915	0.814	-0.586
	LassoNet + FT	0.963	-0.423	0.970	0.690	0.481	0.392	0.818	-0.559	0.733	-0.904	0.808	-0.789
	AM + FT	0.962	-0.432	0.969	0.703	0.504	0.392	0.817	-0.568	0.735	-0.903	0.820	-0.789
	RF + FT	0.962	-0.432	0.970	0.715	0.591	0.395	0.820	-0.558	0.737	-0.900	0.820	-0.791
	XGBoost + FT	0.963	-0.420	0.970	0.725	0.572	0.400	0.819	-0.568	0.734	-0.898	0.820	-0.789
	Deep Lasso + FT	0.962	-0.419	0.969	0.703	0.504	0.392	0.817	-0.560	0.733	-0.900	0.817	-0.788
Ours: LassoFlexNet		0.962	-0.409	0.969	0.720	0.609	0.396	0.819	-0.567	0.730	-0.897	0.815	-0.775
75	No FS + MLP	0.925	-0.527	0.955	0.502	0.417	0.348	0.778	-0.674	0.671	-0.917	0.749	-0.812
	Univariate + MLP	0.960	-0.447	0.970	0.575	0.502	0.381	0.810	-0.549	0.713	-0.890	0.806	-0.776
	Lasso + MLP	0.959	-0.454	0.967	0.543	0.491	0.381	0.811	-0.612	0.716	-0.907	0.802	-0.789
	1L Lasso + MLP	0.957	-0.448	0.968	0.555	0.432	0.380	0.809	-0.572	0.717	-0.903	0.799	-0.775
	AGL + MLP	0.954	-0.447	0.968	0.561	0.429	0.386	0.809	-0.571	0.719	-0.901	0.762	-0.777
	LassoNet + MLP	0.958	-0.452	0.966	0.528	0.475	0.383	0.809	-0.555	0.705	-0.913	0.768	-0.794
	RF + MLP	0.949	-0.453	0.968	0.584	0.610	0.386	0.814	-0.585	0.718	-0.902	0.808	-0.784
	XGBoost + MLP	0.958	-0.451	0.969	0.576	0.583	0.382	0.810	-0.568	0.720	-0.892	0.804	-0.774
	Deep Lasso + MLP	0.957	-0.446	0.969	0.569	0.479	0.387	0.814	-0.559	0.721	-0.893	0.800	-0.774
Ours: LassoFlexNet		0.958	-0.411	0.968	0.665	0.602	0.392	0.810	-0.572	0.721	-0.895	0.805	-0.776

Table 7. Feature selection benchmark on datasets with random extra features. Reports accuracy (classification) and negative RMSE (regression). Bold = top-3 per dataset at 50% and 75%. “No FS” indicates training without an FS pre-filter, as done for *LassoFlexNet*.

C.4. Structural Complexity Comparison

Beyond raw performance, *LassoFlexNet* enjoys structural advantages over competing tabular methods that are inherent to the architecture rather than to engineering optimization. Table 9 contrasts feature- and sample-wise scaling for representative methods. *LassoFlexNet* achieves $\mathcal{O}(d)$ scaling in the number of features through per-feature embeddings and the MLP-Mixer’s token/channel mixing, whereas FT-Transformer and tabular foundation models incur $\mathcal{O}(d^2)$ cost via attention. Mini-batch training keeps memory $\mathcal{O}(\text{batch})$, in contrast to exact-split GBDTs (which maintain $\mathcal{O}(n)$ pre-sorted indices) and foundation models (which scale $\mathcal{O}(\text{ctx}^2)$ in context length, typically capped at $\sim 48\text{K}$). Among these, only LassoNet and *LassoFlexNet* provide native end-to-end feature selection, with *LassoFlexNet* uniquely operating on nonlinear embeddings.

C.5. TabZilla Comprehensive Results

To provide full details, we present the raw performance metrics for all evaluated TabZilla datasets in Table 10.

To rigorously validate the aggregated gains shown in the main text boxplots (Figure 2a), we conducted a Friedman omnibus test followed by pairwise Wilcoxon signed-rank tests (Holm–Bonferroni corrected) comparing *LassoFlexNet* against each baseline across the 35 datasets. Results are summarized in Tables 11 and 12.

Computing environment: We conduct our experiments in a Slurm managed environment for batched parallel jobs, and for each individual run or experiment we use a single GPU (A100 or V100), depending on dataset sizes.

λ Path: In *LassoFlexNet*, the λ path controls the regularization strength schedule. In this section, we describe our simple scheduling.

Concretely, we construct the λ path using polynomial spacing. $p = 0.95$ is our experiment, the other hyperparameters are searched by Optuna.

%	FS method	AL	CH	CO	EY	GE	HE	HI	HO	JA	MI	OT	YE
50	No FS + MLP	0.946	-0.475	0.965	0.557	0.525	0.370	0.802	-0.607	0.703	-0.909	0.778	-0.797
	Univariate + MLP	0.955	-0.451	0.966	0.556	0.514	0.346	0.810	-0.620	0.717	-0.920	0.795	-0.828
	Lasso + MLP	0.955	-0.449	0.968	0.548	0.512	0.382	0.813	-0.602	0.713	-0.903	0.796	-0.795
	1L Lasso + MLP	0.955	-0.447	0.968	0.566	0.515	0.382	0.812	-0.581	0.718	-0.902	0.795	-0.780
	AGL + MLP	0.953	-0.450	0.968	0.588	0.538	0.386	0.813	-0.561	0.722	-0.902	0.796	-0.780
	LassoNet + MLP	0.955	-0.452	0.969	0.570	0.556	0.382	0.811	-0.551	0.719	-0.905	0.795	-0.777
	AM + MLP	0.955	-0.449	0.967	0.583	0.527	0.381	0.814	-0.555	0.722	-0.905	0.797	-0.780
	RF + MLP	0.951	-0.453	0.967	0.574	0.568	0.383	0.810	-0.565	0.724	-0.904	0.788	-0.786
	XGBoost + MLP	0.954	-0.454	0.969	0.583	0.510	0.385	0.815	-0.553	0.722	-0.892	0.803	-0.779
	Deep Lasso + MLP	0.955	-0.447	0.968	0.577	0.525	0.388	0.815	-0.567	0.721	-0.895	0.801	-0.776
	No FS + FT	0.960	-0.430	0.967	0.686	0.576	0.345	0.812	-0.574	0.733	-0.920	0.809	-0.826
	Univariate + FT	0.963	-0.422	0.965	0.681	0.574	0.387	0.820	-0.586	0.732	-0.937	0.812	-0.915
	Lasso + FT	0.952	-0.422	0.936	0.697	0.556	0.389	0.820	-0.570	0.731	-0.899	0.816	-0.795
	1L Lasso + FT	0.962	-0.419	0.969	0.718	0.571	0.392	0.820	-0.552	0.735	-0.914	0.816	-0.830
	AGL + FT	0.906	-0.426	0.969	0.697	0.591	0.393	0.820	-0.572	0.736	-0.903	0.813	-0.790
	LassoNet + FT	0.962	-0.426	0.970	0.679	0.578	0.392	0.820	-0.549	0.734	-0.901	0.817	-0.790
	AM + FT	0.962	-0.424	0.969	0.680	0.572	0.392	0.820	-0.557	0.735	-0.898	0.806	-0.793
	RF + FT	0.962	-0.422	0.969	0.711	0.600	0.387	0.819	-0.548	0.735	-0.897	0.816	-0.790
	XGBoost + FT	0.963	-0.422	0.970	0.706	0.564	0.392	0.821	-0.561	0.736	-0.898	0.809	-0.788
	Deep Lasso + FT	0.961	-0.422	0.968	0.725	0.577	0.393	0.821	-0.598	0.736	-0.898	0.801	-0.776
	Ours: LassoFlexNet	0.960	-0.415	0.970	0.712	0.599	0.390	0.817	-0.590	0.731	-0.903	0.809	-0.780
	No FS + MLP	0.921	-0.516	0.956	0.518	0.503	0.356	0.788	-0.632	0.686	-0.913	0.762	-0.808
	Univariate + MLP	0.955	-0.569	0.941	0.510	0.495	0.347	0.742	-0.620	0.686	-0.921	0.779	-0.838
	Lasso + MLP	0.948	-0.454	0.963	0.565	0.490	0.373	0.810	-0.593	0.717	-0.903	0.795	-0.791
	1L Lasso + MLP	0.955	-0.444	0.967	0.549	0.495	0.380	0.811	-0.576	0.715	-0.903	0.797	-0.779
	AGL + MLP	0.928	-0.566	0.967	0.548	0.490	0.382	0.811	-0.574	0.714	-0.904	0.788	-0.780
	LassoNet + MLP	0.947	-0.452	0.969	0.539	0.533	0.383	0.805	-0.572	0.708	-0.908	0.791	-0.785
	RF + MLP	0.952	-0.450	0.963	0.547	0.533	0.372	0.805	-0.573	0.716	-0.903	0.765	-0.788
	XGBoost + MLP	0.954	-0.515	0.968	0.571	0.530	0.381	0.811	-0.571	0.721	-0.895	0.800	-0.784
	Deep Lasso + MLP	0.959	-0.441	0.968	0.554	0.517	0.386	0.813	-0.563	0.718	-0.898	0.804	-0.778
	Ours: LassoFlexNet	0.955	-0.422	0.963	0.675	0.579	0.382	0.811	-0.588	0.718	-0.905	0.801	-0.785

Table 8. Benchmarking feature-selection methods for MLP and FT-Transformer downstream models on datasets **with corrupted features**. We report accuracy for classification and negative RMSE for regression. % denotes the percent of extra second-order features (50% or 75%). Bold = top-3 within a (dataset, noise-level) cell.

Table 9. Structural complexity of representative tabular methods. d = features, n = training samples, batch = mini-batch size, ctx = in-context size for foundation models (typically capped at $\sim 48K$). “Native FS” indicates end-to-end feature selection without an external pipeline.

Model	Feature scaling	Memory scaling	Native FS	Mechanism
LassoFlexNet (Ours)	$\mathcal{O}(d)$	$\mathcal{O}(\text{batch})$	✓ (nonlinear)	Per-feature embeddings + Mixer
LassoNet	$\mathcal{O}(d)$	$\mathcal{O}(\text{batch})$	✓ (linear)	Linear skip + MLP
TabM	$\mathcal{O}(d)$	$\mathcal{O}(\text{batch})$	—	Parameter-efficient MLP ensemble
FT-Transformer	$\mathcal{O}(d^2)$	$\mathcal{O}(\text{batch})$	—	Attention over d feature tokens
GBDTs (exact-split)	$\mathcal{O}(d \log n)$	$\mathcal{O}(n)$	implicit	Pre-sorted indices per feature
TabPFN / TabICL	$\mathcal{O}(d^2)$	$\mathcal{O}(\text{ctx}^2)$	—	Attention over in-context examples

$$\lambda_0 := \text{lambda_start}, \quad S := \text{lambda_seq_size}, \quad p := \text{power},$$

$$\lambda_E := \begin{cases} \text{lambda_end}, & \text{if provided} \\ \lambda_0 \cdot \text{lambda_multiplier}, & \text{otherwise} \end{cases}$$

$$b_i := \lambda_0^{1/p} + \frac{i}{S-1} \left(\lambda_E^{1/p} - \lambda_0^{1/p} \right), \quad i = 0, \dots, S-1,$$

$$\lambda_i := b_i^p, \quad i = 0, \dots, S-1.$$

We generate the λ sequence using the exponent-scaled schedule defined above. This design is motivated by empirical observations: under a simple linear schedule, we found that λ -training primarily reduces validation loss during the early, smaller λ values, whereas larger λ values later in the sequence rarely yield improvements. This indicates that a linear sequence is overly fine-grained at the beginning and too coarse at the end. By applying an exponent $p < 1$, we skew the distribution of λ values to grow more rapidly at the beginning and more slowly at the end, thereby uniformizing the marginal impact of each step in the sequence.

In our hyperparameter search, the starting value λ_0 is sampled from the interval $[10^{-8}, 10^{-6}]$, and the `lambda_multiplier` is sampled from $[10^3, 10^5]$.

Table 10. Per-dataset raw test performance on the 35 TabZilla benchmark datasets (mean $\pm$ standard deviation across seeds; higher is better). For classification tasks, we report accuracy; for regression tasks, we report adjusted R^2 score. In the latter case, the score can become negative. Bold indicates the best model on each dataset. The final row reports the number of datasets on which each model wins.

Dataset	LassoFlex	XGBoost	CatBoost	LightGBM	TabM	LassoNet
Australian	0.8696 $\pm$ 0.0290	0.8522 $\pm$ 0.0000	0.8681 $\pm$ 0.0000	0.7116 $\pm$ 0.0000	0.8522 $\pm$ 0.0052	0.5551 $\pm$ 0.0000
Bioresponse	0.7819 $\pm$ 0.0053	0.7977 $\pm$ 0.0000	0.7758 $\pm$ 0.0000	0.7987 $\pm$ 0.0000	0.7939 $\pm$ 0.0039	0.5423 $\pm$ 0.0000
GesturePhaseSeg.	0.6186 $\pm$ 0.0107	0.5706 $\pm$ 0.0000	0.5897 $\pm$ 0.0000	0.6565 $\pm$ 0.0000	0.6136 $\pm$ 0.0028	0.2988 $\pm$ 0.0000
MiniBooNE	0.9433 $\pm$ 0.0008	0.9471 $\pm$ 0.0000	0.9267 $\pm$ 0.0000	0.9474 $\pm$ 0.0000	0.9518 $\pm$ 0.0003	0.7194 $\pm$ 0.0000
SpeedDating	0.8788 $\pm$ 0.0073	0.8676 $\pm$ 0.0000	0.8638 $\pm$ 0.0000	0.8694 $\pm$ 0.0000	0.8676 $\pm$ 0.0020	0.8353 $\pm$ 0.0000
ada-agnostic	0.8385 $\pm$ 0.0072	0.8187 $\pm$ 0.0000	0.8549 $\pm$ 0.0000	0.7902 $\pm$ 0.0000	0.8515 $\pm$ 0.0024	0.7519 $\pm$ 0.0000
airlines	0.6631 $\pm$ 0.0007	0.6547 $\pm$ 0.0000	0.6518 $\pm$ 0.0000	0.6689 $\pm$ 0.0000	0.6518 $\pm$ 0.0016	0.5546 $\pm$ 0.0000
albert	0.6937 $\pm$ 0.0015	0.6724 $\pm$ 0.0000	0.7019 $\pm$ 0.0001	0.6855 $\pm$ 0.0000	0.6790 $\pm$ 0.0017	0.5000 $\pm$ 0.0000
artificial-characters	0.8941 $\pm$ 0.0100	0.5451 $\pm$ 0.0000	0.5758 $\pm$ 0.0000	0.8301 $\pm$ 0.0000	0.7105 $\pm$ 0.0015	0.1038 $\pm$ 0.0042
balance-scale	0.9937 $\pm$ 0.0087	0.8832 $\pm$ 0.0000	0.8864 $\pm$ 0.0000	0.9008 $\pm$ 0.0000	0.8956 $\pm$ 0.0052	0.4602 $\pm$ 0.0009
california	0.8621 $\pm$ 0.0038	0.8303 $\pm$ 0.0002	0.7552 $\pm$ 0.0000	0.8361 $\pm$ 0.0006	0.5952 $\pm$ 0.0692	-0.0010 $\pm$ 0.0020
cnae-9	0.9606 $\pm$ 0.0046	0.9102 $\pm$ 0.0000	0.8296 $\pm$ 0.0000	0.8204 $\pm$ 0.0000	0.9176 $\pm$ 0.0016	0.1111 $\pm$ 0.0000
colic	0.8108 $\pm$ 0.0427	0.8125 $\pm$ 0.0000	0.8532 $\pm$ 0.0000	0.6959 $\pm$ 0.0000	0.7444 $\pm$ 0.0109	0.6631 $\pm$ 0.0000
credit-approval	0.8841 $\pm$ 0.0205	0.8362 $\pm$ 0.0000	0.8768 $\pm$ 0.0000	0.5783 $\pm$ 0.0000	0.8496 $\pm$ 0.0056	0.5530 $\pm$ 0.0045
credit-g	0.7940 $\pm$ 0.0297	0.7540 $\pm$ 0.0000	0.7440 $\pm$ 0.0000	0.7410 $\pm$ 0.0000	0.7202 $\pm$ 0.0116	0.7000 $\pm$ 0.0000
electricity	0.9008 $\pm$ 0.0080	0.8315 $\pm$ 0.0000	0.8259 $\pm$ 0.0000	0.9032 $\pm$ 0.0000	0.8574 $\pm$ 0.0012	0.5755 $\pm$ 0.0000
elevators	0.8945 $\pm$ 0.0040	0.8884 $\pm$ 0.0000	0.8661 $\pm$ 0.0000	0.8889 $\pm$ 0.0000	0.8149 $\pm$ 0.0246	0.6909 $\pm$ 0.0000
heart-h	0.7467 $\pm$ 0.0447	0.7078 $\pm$ 0.0000	0.8233 $\pm$ 0.0000	0.6664 $\pm$ 0.0000	0.8258 $\pm$ 0.0099	0.5544 $\pm$ 0.0943
higgs	0.7294 $\pm$ 0.0025	0.7187 $\pm$ 0.0000	0.7156 $\pm$ 0.0002	0.7269 $\pm$ 0.0000	0.7362 $\pm$ 0.0002	0.5286 $\pm$ 0.0000
jasmine	0.8000 $\pm$ 0.0104	0.8043 $\pm$ 0.0000	0.8046 $\pm$ 0.0000	0.8100 $\pm$ 0.0000	0.7899 $\pm$ 0.0029	0.5001 $\pm$ 0.0003
jungle-chess	0.9922 $\pm$ 0.0014	0.7842 $\pm$ 0.0000	0.7697 $\pm$ 0.0000	0.8633 $\pm$ 0.0000	0.9980 $\pm$ 0.0005	0.5146 $\pm$ 0.0000
kcl	0.8692 $\pm$ 0.0054	0.8554 $\pm$ 0.0000	0.8568 $\pm$ 0.0000	0.8450 $\pm$ 0.0000	0.8455 $\pm$ 0.0002	0.8454 $\pm$ 0.0000
lymph	0.7467 $\pm$ 0.0989	0.8167 $\pm$ 0.0000	0.7619 $\pm$ 0.0000	0.6967 $\pm$ 0.0000	0.8325 $\pm$ 0.0057	0.5333 $\pm$ 0.0000
mfeat-fourier	0.8040 $\pm$ 0.0198	0.8425 $\pm$ 0.0000	0.8285 $\pm$ 0.0000	0.8170 $\pm$ 0.0000	0.8553 $\pm$ 0.0032	0.1000 $\pm$ 0.0000
mfeat-zernike	0.7620 $\pm$ 0.0217	0.7840 $\pm$ 0.0000	0.7685 $\pm$ 0.0000	0.7710 $\pm$ 0.0000	0.8066 $\pm$ 0.0029	0.1000 $\pm$ 0.0000
nomao	0.9684 $\pm$ 0.0016	0.9602 $\pm$ 0.0000	0.9613 $\pm$ 0.0000	0.9741 $\pm$ 0.0000	0.9689 $\pm$ 0.0001	0.7144 $\pm$ 0.0000
one-hundred-plants-texture	0.7825 $\pm$ 0.0195	0.7255 $\pm$ 0.0000	0.7511 $\pm$ 0.0000	0.4065 $\pm$ 0.0000	0.2568 $\pm$ 0.0476	0.0088 $\pm$ 0.0004
phoneme	0.8847 $\pm$ 0.0084	0.8390 $\pm$ 0.0000	0.8605 $\pm$ 0.0006	0.8749 $\pm$ 0.0000	0.8674 $\pm$ 0.0102	0.7065 $\pm$ 0.0000
poker-hand	0.9980 $\pm$ 0.0000	0.5664 $\pm$ 0.0000	0.6326 $\pm$ 0.0000	0.6994 $\pm$ 0.0000	0.5527 $\pm$ 0.0222	0.5012 $\pm$ 0.0000
profb	0.6853 $\pm$ 0.0494	0.6548 $\pm$ 0.0000	0.7157 $\pm$ 0.0000	0.7217 $\pm$ 0.0000	0.6759 $\pm$ 0.0075	0.6667 $\pm$ 0.0000
qsar-biodeg	0.8170 $\pm$ 0.0084	0.8398 $\pm$ 0.0000	0.8569 $\pm$ 0.0000	0.7706 $\pm$ 0.0000	0.8386 $\pm$ 0.0226	0.6559 $\pm$ 0.0149
socmob	0.9483 $\pm$ 0.0136	0.8946 $\pm$ 0.0000	0.9464 $\pm$ 0.0000	0.9300 $\pm$ 0.0000	0.9453 $\pm$ 0.0041	0.7786 $\pm$ 0.0000
vehicle	0.7600 $\pm$ 0.0271	0.7458 $\pm$ 0.0000	0.7530 $\pm$ 0.0000	0.7199 $\pm$ 0.0000	0.6903 $\pm$ 0.0082	0.2352 $\pm$ 0.0000
Wins	15	0	4	7	7	0

Table 11. Friedman test for comparing multiple classifiers over multiple datasets. The null hypothesis states that all models perform equivalently, i.e., their average ranks are equal. A significant result ($p < 0.05$) indicates that at least one model differs from the others.

Statistic	Value
Friedman χ^2	72.8413
p -value	2.62e-14
Significant ($\alpha = 0.05$)	Yes

Average ranks across datasets (rank 1 = best).

Model	Average Rank
LassoFlexNet	2.161
LightGBM	3.032
TabM	3.113
CatBoost	3.161
XGBoost	3.597
LassoNet	5.935

Table 12. Pairwise Wilcoxon signed-rank tests: LassoFlexNet vs. each competitor (one-sided, H_1 : LassoFlex > competitor). The Holm-Bonferroni correction controls the family-wise error rate across 5 comparisons. A model is significantly worse than LassoFlex if its adjusted p -value falls below $\alpha = 0.05$.

Competitor	W -statistic	p -value (adjusted)	Significant
LassoNet	496	2.33e-09	✓
XGBoost	430	3.21e-04	✓
LightGBM	404	2.38e-03	✓
TabM	372	1.40e-02	✓
CatBoost	357	1.61e-02	✓

D. Appendix: PLE Breaks Rotational Invariance

Piecewise Linear Encoding (PLE) for tabular data is intentionally *axis-aligned*: it encodes each numerical feature with a coordinate-wise piecewise linear map defined by feature-wise breakpoints. This section explains, in two phases, why this design breaks rotational invariance in the sense of (Ng, 2004), and why adding a powerful downstream model such as an neural nets trained by SGD, does not restore rotational invariance. Throughout the counterexample, PLE breakpoints are treated as fixed after being chosen on the original coordinate system. This matches the intended tabular use case, where columns are semantic and PLE is an axis-aligned preprocessing step tied to those columns.

Phase 1 (NTK approach). In the wide/lazy regime, SGD-trained MLPs behave like kernel regression with an orthogonally invariant neural tangent kernel (NTK) (Jacot et al., 2018). The induced kernel on raw inputs is the NTK applied to encoded vectors. If the encoder is not rotation-equivariant, the induced kernel is not rotation-invariant, which yields an explicit counterexample.

Phase 2 (general obstruction). Even beyond NTK, if the downstream learning rule is itself symmetric to orthogonal changes of coordinates in feature space, then the only way the *end-to-end* raw-space algorithm can be rotationally invariant is if the encoder intertwines raw rotations with feature-space orthogonal maps. PLE generally does not.

D.1. Setup and Definitions

D.1.1. ROTATIONAL INVARIANCE

Let $O(n) = \{M \in \mathbb{R}^{n \times n} : M^\top M = I\}$ denote the orthogonal group (Ng, 2004). A labeled dataset of size m is $S = \{(x^{(i)}, y^{(i)})\}_{i=1}^m$ with $x^{(i)} \in \mathbb{R}^n$. For $M \in O(n)$, define the rotated dataset

$$MS := \{(Mx^{(i)}, y^{(i)})\}_{i=1}^m.$$

Definition D.1 (Rotational invariance of a learning algorithm). Let L be a (possibly randomized) learning algorithm. We write $L[S](x)$ for the prediction (a random variable if L is randomized) obtained by training L on S and evaluating at x . We say:

- L is *rotationally invariant (deterministic)* if for all $S, M \in O(n)$, and x ,

$$L[S](x) = L[MS](Mx).$$

- L is *rotationally invariant (randomized)* if for all $S, M \in O(n)$, and x , the random variables $L[S](x)$ and $L[MS](Mx)$ have the same distribution.

D.1.2. A CONCRETE PLE MAP (COORDINATE-WISE, PIECEWISE LINEAR)

For a single scalar feature $x \in [b_0, b_T]$, fix breakpoints $b_0 < b_1 < \dots < b_T$ and segment lengths $\Delta_t = b_t - b_{t-1} > 0$. Define the clipping operator $\text{clip}(u) = \min\{1, \max\{0, u\}\}$.

Definition D.2 (PLE for one feature). The piecewise linear encoding (PLE) map $\phi : [b_0, b_T] \rightarrow \mathbb{R}^T$ is

$$\phi(x) := (\phi_1(x), \dots, \phi_T(x)), \quad \phi_t(x) := \text{clip}\left(\frac{x - b_{t-1}}{\Delta_t}\right).$$

For an n -dimensional input $x = (x_1, \dots, x_n)$, we use a coordinate-wise encoder (with possibly different T_j per coordinate):

$$\Phi(x) := (\phi^{(1)}(x_1), \phi^{(2)}(x_2), \dots, \phi^{(n)}(x_n)) \in \mathbb{R}^D, \quad D := \sum_{j=1}^n T_j.$$

In practical PLE for tabular data, the breakpoints are often *data-dependent* (e.g., learned from axis-aligned trees), but not learned end to end with the neural nets. The counterexamples below condition on such fitted breakpoints and then analyze the induced fixed coordinate-wise encoder. They do not cover a hypothetical rotate-and-refit pipeline in which breakpoints are recomputed after every raw-space rotation.

D.2. Phase 1: Counterexample via the NTK/Kernel-Regression View of SGD-Trained MLPs

The main observation in this section is that we can explicitly construct a counterexample for a simple MLP. Because NTK theory approximately reduces the analysis of wide neural networks to kernel regression, demonstrating a symmetry break in the induced kernel is sufficient to prove that the network breaks rotational invariance.

D.2.1. FROM WIDE MLP + SGD TO KERNEL REGRESSION

Consider an MLP $f_\theta : \mathbb{R}^D \rightarrow \mathbb{R}$ trained on encoded data $\{(z_i, y_i)\}_{i=1}^m$ with $z_i \in \mathbb{R}^D$ by (stochastic) gradient descent on squared loss. In the wide-network, lazy-training regime (weights move little from initialization), the training dynamics linearize around the initialization θ_0 and are governed by the neural tangent kernel (NTK),

$$K_{\text{NTK}}(z, z') := \langle \nabla_\theta f_{\theta_0}(z), \nabla_\theta f_{\theta_0}(z') \rangle.$$

In this regime, the learned predictor is (approximately, and exactly in the infinite-width gradient-flow limit) the kernel regression / kernel ridge solution with kernel K_{NTK} (Jacot et al., 2018).

Key symmetry property. For standard isotropic random initialization of fully-connected networks, the limiting NTK is *orthogonally invariant* in input space:

$$K_{\text{NTK}}(Qz, Qz') = K_{\text{NTK}}(z, z') \quad \forall Q \in O(D). \quad (7)$$

Intuitively: there is no preferred coordinate system in feature space.

D.2.2. APPLYING THE NTK TO PLE + MLP

Fix a raw dataset $S = \{(x_i, y_i)\}_{i=1}^m \subset \mathbb{R}^n \times \mathbb{R}$. Define encoded points

$$z_i := \Phi(x_i) \in \mathbb{R}^D.$$

The NTK view says that training the MLP on PLE features is (approximately) kernel regression with kernel K_{NTK} applied to $\{z_i\}$.

After the breakpoints are fixed, this induces a *raw-space* kernel:

$$K_S(x, x') := K_{\text{NTK}}(\Phi(x), \Phi(x')). \quad (8)$$

What rotational invariance would force. If the end-to-end pipeline $x \mapsto \Phi(x) \mapsto (\text{SGD-trained MLP})$ were rotationally invariant, then for every $M \in O(n)$, the induced kernels would satisfy

$$K_S(x_i, x_j) = K_{MS}(Mx_i, Mx_j) \quad \text{for all training pairs } (i, j), \quad (9)$$

because kernel regression predictions are determined by the Gram matrix on the training set (and its interaction with test points).

We now show equation 9 fails for PLE via a fully explicit computation.

D.2.3. AN EXPLICIT COUNTEREXAMPLE: ROTATION LEADS TO DIFFERENT INNER PRODUCTS

We work in raw dimension $n = 2$. Use the same PLE map on each coordinate with breakpoints $-1, 0, 1$ (so $T = 2$, $\Delta_1 = \Delta_2 = 1$). For a scalar $x \in [-1, 1]$, the 2-dimensional PLE is

$$\phi(x) = (\phi_1(x), \phi_2(x)), \quad \phi_1(x) = \text{clip}(x + 1), \quad \phi_2(x) = \text{clip}(x),$$

and the full encoder is $\Phi(x_1, x_2) = (\phi(x_1), \phi(x_2)) \in \mathbb{R}^4$.

Let the two raw training points be

$$x^{(1)} := (0.5, 0), \quad x^{(2)} := (0, 0.5).$$

Compute their encodings:

$$\phi(0.5) = (1, 0.5), \quad \phi(0) = (1, 0),$$

so

$$z_1 = \Phi(x^{(1)}) = (1, 0.5, 1, 0), \quad z_2 = \Phi(x^{(2)}) = (1, 0, 1, 0.5).$$

Hence

$$\langle z_1, z_1 \rangle = 2.25, \quad \langle z_2, z_2 \rangle = 2.25, \quad \langle z_1, z_2 \rangle = 2.0.$$

Now rotate raw space by 45° :

$$M := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \in O(2).$$

Then

$$Mx^{(1)} = \left(\frac{1}{2\sqrt{2}}, \frac{1}{2\sqrt{2}} \right) \approx (0.3536, 0.3536), \quad Mx^{(2)} = \left(-\frac{1}{2\sqrt{2}}, \frac{1}{2\sqrt{2}} \right) \approx (-0.3536, 0.3536).$$

Encode these rotated points:

$$\phi(0.3536) = (1, 0.3536), \quad \phi(-0.3536) = (0.6464, 0),$$

so

$$z'_1 = \Phi(Mx^{(1)}) = (1, 0.3536, 1, 0.3536), \quad z'_2 = \Phi(Mx^{(2)}) = (0.6464, 0, 1, 0.3536).$$

A direct dot-product computation gives

$$\langle z'_1, z'_1 \rangle = 2.25, \quad \langle z'_2, z'_2 \rangle \approx 1.5429, \quad \langle z'_1, z'_2 \rangle \approx 1.7714.$$

In particular, the feature-space geometry has changed: for example $\langle z_1, z_2 \rangle = 2.0$ while $\langle z'_1, z'_2 \rangle \approx 1.7714$. The rotation leads to changes on the NTK component. We summarize these as a lemma:

Lemma D.3 (PLE breaks Gram-matrix matching for dot-product kernels). *Let $k : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ be any kernel that is a nonconstant function of the inner product $\langle z, z' \rangle$ and the norms $\|z\|, \|z'\|$. For the PLE encoder Φ , there exists a Gram matrix $(k(z_i, z_j))_{i,j \in 1,2}$ that changes under the rotation M .*

D.2.4. TURNING KERNEL MISMATCH INTO A PREDICTION MISMATCH

A kernel regressor (e.g., kernel ridge regression) trained on a dataset is determined by the Gram matrix on the training set. Therefore, once the Gram matrix differs between S and MS , we can force a change in predictions by an appropriate choice of labels.

To keep the algebra transparent, we illustrate this using kernel ridge regression with the *linear* kernel $k(z, z') = \langle z, z' \rangle$ and a ridge penalty of $\lambda = 0.1$. Let $y = (1, -1)$ be the label vector for $(x^{(1)}, x^{(2)})$. Substituting any fixed, non-degenerate dot-product kernel would yield the same conclusion.

Example 1 (Explicit prediction mismatch for kernel ridge regression). *With the setup above, kernel ridge regression trained on $\{(z_1, y_1), (z_2, y_2)\}$ and evaluated at $x^{(1)}$ yields a different prediction than kernel ridge regression trained on the rotated dataset and evaluated at $Mx^{(1)}$.*

Proof. Kernel ridge regression has the closed form

$$f(x) = k_x^\top (K + \lambda I)^{-1} y, \quad (k_x)_j := k(z, z_j), \quad z := \Phi(x),$$

where $K_{ij} = k(z_i, z_j)$. For the unrotated dataset,

$$K = \begin{pmatrix} 2.25 & 2.0 \\ 2.0 & 2.25 \end{pmatrix}, \quad k_{x^{(1)}} = (2.25, 2.0).$$

A direct computation gives $f(x^{(1)}) \approx 0.7143$.

For the rotated dataset,

$$K' = \begin{pmatrix} 2.25 & 1.7714 \\ 1.7714 & 1.5429 \end{pmatrix}, \quad k_{Mx^{(1)}} = (2.25, 1.7714),$$

and solving the same linear system gives $f'(Mx^{(1)}) \approx 0.5276$. Therefore $f(x^{(1)}) \neq f'(Mx^{(1)})$, so the learning rule is not rotationally invariant. $\square$

D.2.5. THE NTK CONCLUSION

Example 1 used the linear kernel only to keep arithmetic transparent. Lemma D.3 shows that for a broad class of dot-product kernels, including standard fully-connected NTKs, the Gram matrix changes under rotations once we insert a PLE map. We summarize the counter-example discussion into a proposition.

Proposition D.4 (Phase 1 conclusion: not rotationally invariant in the NTK/lazy regime). *Consider the end-to-end pipeline: raw input $x \in \mathbb{R}^n$ is encoded by a coordinate-wise PLE map Φ , then a fully-connected MLP is trained on $\Phi(x)$ by SGD in the wide/lazy regime. Assume the limiting NTK satisfies the orthogonal invariance property equation 7 and is nonconstant in $\langle z, z' \rangle$ and norms. Then the resulting learning algorithm is not rotationally invariant.*

Proof. By the NTK theory, the learned predictor is kernel regression with kernel K_{NTK} on encoded points, i.e., the induced kernel equation 8 on raw inputs. Lemma D.3 provides a dataset S and rotation M for which the Gram matrix on the training set differs between S and MS . Since kernel regression predictions depend on the Gram matrix, there exists a label vector for which predictions differ at corresponding test points (Example 1). $\square$

Remark D.5 (Finite width: why the message typically persists). Theorem D.4 is a statement about the lazy/wide regime where SGD is well-approximated by kernel regression. At finite width, SGD no longer exactly equals kernel regression. However, the *source* of symmetry breaking remains: Φ is axis-aligned and generally not rotation-equivariant. Thus, unless training uses additional mechanisms that explicitly enforce rotational invariance in raw space, one should not expect invariance to hold in general.

D.3. Phase 2: A General Obstruction — Invariance Cannot Be Recovered Without Encoder Equivariance

Phase 1 used the NTK story (SGD behaves like a kernel method) to make the failure concrete. We now present a more general, kernel-free viewpoint that applies to non-lazy regimes.

D.3.1. TWO SYMMETRY NOTIONS (RAW SPACE VS FEATURE SPACE)

Let $S = \{(x_i, y_i)\}_{i=1}^m$ be a raw dataset. Let $\Phi_S : \mathbb{R}^n \rightarrow \mathbb{R}^D$ be a (possibly data-dependent) encoder and define the encoded dataset

$$Z_S := \{(z_i, y_i)\}_{i=1}^m, \quad z_i := \Phi_S(x_i).$$

A downstream learning algorithm A maps an encoded dataset Z to a predictor $A(Z) : \mathbb{R}^D \rightarrow \mathcal{Y}$. The end-to-end learner is

$$L[S](x) := A(Z_S)(\Phi_S(x)).$$

Definition D.6 (Orthogonal invariance of the downstream learner). For $Q \in O(D)$, define $QZ := \{(Qz_i, y_i)\}_{i=1}^m$. We say A is *orthogonally invariant* if for all datasets Z , all $Q \in O(D)$, and all feature vectors $z \in \mathbb{R}^D$,

$$A(QZ)(Qz) = A(Z)(z).$$

This captures the idea that the downstream training has no preferred coordinate system in feature space. (For example, standard fully-connected networks with isotropic initialization and SGD dynamics satisfy this kind of invariance in an idealized sense.)

D.3.2. A SUFFICIENT CONDITION FOR END-TO-END ROTATIONAL INVARIANCE

The next definition isolates the *only* way a feature-space orthogonal symmetry can translate back to the original space rotational symmetry.

Definition D.7 (Encoder equivariance up to a feature-space orthogonal map). The encoder family $\{\Phi_S\}$ is *rotation-equivariant up to feature orthogonals* if for every raw dataset S and every $M \in O(n)$, there exists $Q_{S,M} \in O(D)$ such that for all raw inputs x ,

$$\Phi_{MS}(Mx) = Q_{S,M} \Phi_S(x).$$

(Here $Q_{S,M}$ may depend on S and M , but must be a single global orthogonal map, independent of x .)

Proposition D.8 (Sufficiency: encoder equivariance + feature-space invariance implies raw rotational invariance). *If A is orthogonally invariant (Definition D.6) and Φ is rotation-equivariant up to a feature-space orthogonal (Definition D.7), then the composite learner $L[S](x) = A(Z_S)(\Phi_S(x))$ is rotationally invariant.*

Proof. Fix S, M, x . Let $Q := Q_{S,M}$ be from Definition D.7. For each training point x_i ,

$$\Phi_{MS}(Mx_i) = Q\Phi_S(x_i),$$

so $Z_{MS} = QZ_S$. Also $\Phi_{MS}(Mx) = Q\Phi_S(x)$. Therefore

$$L[MS](Mx) = A(Z_{MS})(\Phi_{MS}(Mx)) = A(QZ_S)(Q\Phi_S(x)) = A(Z_S)(\Phi_S(x)) = L[S](x),$$

where we used orthogonal invariance of A in the third equality. $\square$

D.3.3. THE OBSTRUCTION INTUITION: WITHOUT ENCODER EQUIVARIANCE, INVARIANCE HAS NO SYMMETRY REASON TO HOLD

Proposition D.8 is only a sufficient condition: if the encoder provides a global orthogonal alignment $Q_{S,M}$ between the original and rotated encoded problems, then any orthogonally invariant downstream learner inherits raw-space rotational invariance.

The converse is not automatic. A degenerate downstream learner could ignore the encoded data entirely, and a non-equivariant encoder could still accidentally produce identical predictions for a particular dataset, label vector, or query set. Thus, non-equivariance of Φ alone should not be read as a universal impossibility theorem for every downstream algorithm.

Nevertheless, encoder equivariance identifies the symmetry mechanism that would make raw rotational invariance structurally automatic. When PLE fails this equivariance, the rotated encoded problem is generally not a mere orthogonal reparameterization of the original encoded problem. A symmetric downstream learner therefore has no group-theoretic reason to recover raw-space rotational invariance after the coordinate-wise PLE map. Phase 1 makes this obstruction concrete by giving an explicit kernel-regression counterexample in the NTK/lazy regime.

D.3.4. WHY DOWNSTREAM MLPs SHOULD NOT BE EXPECTED TO UNLEARN THE ROTATION UNDER PLE

The preceding counterexample and encoder-equivariance discussion clarify a few subtle but important points.

- **Expressivity is not the issue.** An MLP is a universal approximator; in principle, with enough data, it can learn complicated functions that may correlate with a rotated target concept.
- **Algorithmic rotational invariance is the issue.** Rotational invariance is a statement about the *learning procedure*: for every dataset and every rotation, training on S and testing at x must agree with training on MS and testing at Mx .
- **PLE breaks the symmetry before SGD begins.** Coordinate-wise PLE creates axis-aligned “kinks” (breakpoints) and saturations. A generic raw rotation changes how points fall relative to these kinks. Unless the encoder is rotation-equivariant up to a global orthogonal map in feature space, a symmetric training rule (such as standard SGD on an MLP with isotropic initialization) has no canonical mechanism that forces it to output a rotationally invariant predictor.

D.4. Discussion: Why Breaking Rotational Invariance Is Often the Point in Tabular Learning

Rotational invariance is attractive when the coordinate system is arbitrary (e.g., images after whitening). Tabular data is often different: coordinates correspond to semantically meaningful features, and feature selection or axis-aligned splits can be beneficial. Specifically, tabular datasets frequently contain noisy, irrelevant features; a rotationally invariant model would inextricably mix these with informative features, degrading performance. In fact, (Ng, 2004) gives lower bounds showing that *any* rotationally invariant learner can be forced to require $\Omega(n)$ samples on an “easy” one-feature threshold problem after an adversarial rotation. From that perspective, PLE’s axis alignment is not a bug: it is a deliberate inductive bias toward feature-wise heterogeneity.

E. Appendix: PLE Provides Fixed-Knot Localized Expressivity

In the scalar fixed-knot setting, PLE supplies an axis-aligned spline basis: it fixes breakpoint locations before learning and exposes **bin-local degrees of freedom** through local slopes. This is a more restrictive hypothesis class than unconstrained breakpoint learning, but it is often well aligned with tabular data, where localized univariate effects and axis-aligned thresholds are natural inductive biases. We also analyze the induced value and slope kernels to clarify how this fixed basis changes the geometry seen by downstream linear models.

Scope. We work in the simplest setting of a *single numerical feature* $x \in [b_0, b_T]$ and compare: (i) a linear model on top of PLE (“linear-on-PLE”); and (ii) a one-hidden-layer scalar ReLU MLP on the raw scalar (“MLP-without-PLE”). In other words, we investigate the benefits of PLE for a single numerical feature. These results should be read as scalar fixed-knot expressivity statements, not as a parameter-free expressivity theorem for the full LassoFlexNet architecture.

E.1. Setup and Notation

Definition E.1 (Knots, bins, and bin index). Fix knots

$$b_0 < b_1 < \dots < b_T, \quad I := [b_0, b_T], \quad \Delta_t := b_t - b_{t-1} > 0.$$

For $x \in (b_0, b_T)$, define the *bin index* $s(x) \in \{1, \dots, T\}$ by

$$s(x) = t \iff x \in (b_{t-1}, b_t).$$

Let the boundary set be $\mathcal{B} := \{b_0, \dots, b_T\}$.

Definition E.2 (Clipping operator). For $u \in \mathbb{R}$, define $\text{clip}(u) := \min\{1, \max\{0, u\}\} \in [0, 1]$.

Definition E.3 (Piecewise linear encoding (PLE) map for one feature). Define $\phi : I \rightarrow \mathbb{R}^T$ by $\phi(x) = (\phi_1(x), \dots, \phi_T(x))$, where for $t \in \{1, \dots, T\}$,

$$\phi_t(x) := \text{clip}\left(\frac{x - b_{t-1}}{\Delta_t}\right). \quad (10)$$

Equivalently,

$$\phi_t(x) = \begin{cases} 0, & x \leq b_{t-1}, \\ \frac{x - b_{t-1}}{\Delta_t}, & b_{t-1} \leq x \leq b_t, \\ 1, & x \geq b_t. \end{cases}$$

Remark E.4 (“Almost everywhere” (a.e.) differentiability convention). All derivatives with respect to x are understood on $I \setminus \mathcal{B}$ (and similarly for MLPs, away from their kink set). At points where a piecewise-linear function is not differentiable, any selection of subgradient can be used; this affects only a measure-zero set in x and does not change the kernel identities stated “a.e.”.

E.2. Phase 1: Linear-on-PLE Is Exactly the Linear Spline Class (Full Derivation)

Definition E.5 (Linear-on-PLE model). Given $(w, c) \in \mathbb{R}^T \times \mathbb{R}$, define

$$f_{w,c}(x) := c + w^\top \phi(x), \quad x \in I.$$

Definition E.6 (Linear spline space with knots $\{b_t\}$). Let $\mathcal{S}(b_{0:T})$ denote the set of continuous functions $s : I \rightarrow \mathbb{R}$ that are affine on each bin $[b_{t-1}, b_t]$. Equivalently, $s \in \mathcal{S}(b_{0:T})$ iff there exist slopes $(\alpha_1, \dots, \alpha_T) \in \mathbb{R}^T$ and a base value $s(b_0) \in \mathbb{R}$ such that, for $x \in [b_{t-1}, b_t]$,

$$s(x) = s(b_0) + \sum_{j=1}^{t-1} \alpha_j \Delta_j + \alpha_t (x - b_{t-1}). \quad (11)$$

Theorem E.7 (Spline equivalence with explicit forward formula and converse). *The class $\{f_{w,c} : (w, c) \in \mathbb{R}^T \times \mathbb{R}\}$ equals $\mathcal{S}(b_{0:T})$. Moreover:*

1. **(Forward / explicit spline form).** Fix (w, c) . For $x \in (b_{t-1}, b_t)$ (so $t = s(x)$),

$$f_{w,c}(x) = c + \sum_{j=1}^{t-1} w_j + w_t \frac{x - b_{t-1}}{\Delta_t}. \quad (12)$$

Hence $f_{w,c}$ is a continuous linear spline with per-bin slope

$$\partial_x f_{w,c}(x) = \frac{w_{s(x)}}{\Delta_{s(x)}} \quad \text{for } x \in I \setminus \mathcal{B}. \quad (13)$$

2. **(Converse / explicit weight construction).** Given any spline $s \in \mathcal{S}(b_{0:T})$ with representation equation 11, define

$$c := s(b_0), \quad w_t := \alpha_t \Delta_t \quad (t = 1, \dots, T). \quad (14)$$

Then $f_{w,c}(x) = s(x)$ for all $x \in I$.

Proof. We proceed in explicit steps.

Step 1: Evaluate $\phi(x)$ inside a bin. Fix $x \in (b_{t-1}, b_t)$ with $t = s(x)$. From Definition E.3:

$$\phi_j(x) = \begin{cases} 1, & j \leq t-1 \quad (\text{since } x \geq b_j), \\ \frac{x - b_{t-1}}{\Delta_t}, & j = t, \\ 0, & j \geq t+1 \quad (\text{since } x \leq b_{j-1}). \end{cases}$$

Step 2: Expand $w^\top \phi(x)$. Using the binwise values above,

$$w^\top \phi(x) = \sum_{j=1}^{t-1} w_j \cdot 1 + w_t \cdot \frac{x - b_{t-1}}{\Delta_t} + \sum_{j=t+1}^T w_j \cdot 0 = \sum_{j=1}^{t-1} w_j + w_t \frac{x - b_{t-1}}{\Delta_t}.$$

Adding c gives equation 12. This shows $f_{w,c}$ is affine on each bin and continuous at the knots (the formula matches at endpoints), hence $f_{w,c} \in \mathcal{S}(b_{0:T})$.

Step 3: Compute the per-bin slope. For $x \in (b_{t-1}, b_t)$, differentiating equation 12 yields

$$\partial_x f_{w,c}(x) = \frac{w_t}{\Delta_t},$$

which is equation 13.

Step 4: Converse construction. Let $s \in \mathcal{S}(b_{0:T})$ be written as equation 11. Define (w, c) by equation 14. Fix $x \in (b_{t-1}, b_t)$. Then using equation 12,

$$f_{w,c}(x) = c + \sum_{j=1}^{t-1} w_j + w_t \frac{x - b_{t-1}}{\Delta_t} = s(b_0) + \sum_{j=1}^{t-1} \alpha_j \Delta_j + \alpha_t \Delta_t \frac{x - b_{t-1}}{\Delta_t}.$$

Canceling Δ_t in the last term gives exactly equation 11:

$$f_{w,c}(x) = s(b_0) + \sum_{j=1}^{t-1} \alpha_j \Delta_j + \alpha_t (x - b_{t-1}) = s(x).$$

By continuity, equality holds for all $x \in I$. $\square$

Corollary E.8 (Kinks at knots and the “generic kink” condition). *Let $f_{w,c}$ be linear-on-PLE. For an interior knot b_t ($t = 1, \dots, T-1$), the spline is differentiable at b_t iff the left and right slopes match:*

$$\frac{w_t}{\Delta_t} = \frac{w_{t+1}}{\Delta_{t+1}}.$$

Equivalently, b_t is a kink iff $\frac{w_t}{\Delta_t} \neq \frac{w_{t+1}}{\Delta_{t+1}}$. In particular, if w is drawn from a distribution with a density on $\mathbb{R}^T$, then with probability 1 all interior knots are kinks.

Proof. The left derivative at b_t equals the slope on (b_{t-1}, b_t) , i.e. w_t/Δ_t , and the right derivative equals the slope on (b_t, b_{t+1}) , i.e. w_{t+1}/Δ_{t+1} , by equation 13. Equality of one-sided derivatives is equivalent to differentiability at b_t . The probability-one statement follows because $\{w : w_t/\Delta_t = w_{t+1}/\Delta_{t+1}\}$ is a codimension-1 hyperplane in $\mathbb{R}^T$. $\square$

Remark E.9. Theorem E.7 formally identifies linear-on-PLE with the linear spline function class. Next, we bound the number of hidden units a standard ReLU MLP requires to match this knot complexity, before quantifying PLE’s *localized* learning behavior via NTK analysis.

E.3. Phase 2: Degrees of Freedom vs. a 1-Hidden-Layer Scalar ReLU MLP (Knot Counting)

Definition E.10 (MLP-without-PLE: one-hidden-layer scalar ReLU net). Fix width $m \in \mathbb{N}$. Consider

$$f_\theta(x) = \sum_{r=1}^m a_r \text{ReLU}(w_r x + b_r) + c, \quad \text{ReLU}(u) = \max\{0, u\}, \quad (15)$$

with parameters $\theta = (a, w, b, c) \in \mathbb{R}^m \times \mathbb{R}^m \times \mathbb{R}^m \times \mathbb{R}$.

Proposition E.11 (Knot counting for one-hidden-layer ReLU). *For any fixed parameters θ , the function $x \mapsto f_\theta(x)$ is continuous piecewise linear on $\mathbb{R}$ with at most m interior breakpoints (kinks). Consequently, to represent a generic spline in $\mathcal{S}(b_{0:T})$ having $T-1$ distinct interior knots, a one-hidden-layer ReLU net must have width*

$$m \geq T-1.$$

Proof. For each neuron r , the map $x \mapsto \text{ReLU}(w_r x + b_r)$ is piecewise linear with a single possible kink at the point where $w_r x + b_r = 0$ (if $w_r \neq 0$), i.e.

$$x_r^* = -\frac{b_r}{w_r}.$$

If $w_r = 0$, then $\text{ReLU}(w_r x + b_r) = \text{ReLU}(b_r)$ is constant and contributes no kink. Therefore, the kink set of the sum equation 15 is contained in the union of the individual kink locations $\{x_r^* : w_r \neq 0\}$, so it has at most m distinct points. This proves the first claim.

For the consequence, note that a generic spline with knots exactly at $b_1, \dots, b_{T-1}$ has $T-1$ distinct interior kink locations. Since a width- m network can have at most m distinct kink locations, representing such a spline requires $m \geq T-1$. $\square$

Remark E.12 (Why this comparison is the right one). Deep ReLU nets can create many breakpoints, but the key point here is structural: *without PLE, breakpoint locations must be learned by the network; with PLE, the knot grid is built into the representation and learning can focus on per-bin slopes*. This does not mean PLE has no parameter cost; rather, the breakpoints are fixed by preprocessing, and the learned parameters control slopes over those fixed bins. Next, we discuss the *localized slope geometry* given by PLE.

E.4. Phase 3: Kernel Geometry — Value NTK vs. Slope NTK

The preceding sections show that PLE builds explicit breakpoints into the representation. We now formalize another structural benefit: PLE provides more localized geometry for *input sensitivities*. In tabular data, the slope with respect to an input feature is often meaningful, and a raw-input MLP ties first-layer parameters directly to the input in a comparatively global way. By contrast, PLE exposes bin-specific slope degrees of freedom. We make this precise using value and slope Neural Tangent Kernel (NTK) calculations.

E.4.1. STANDARD (VALUE) NTK: DEFINITIONS AND EXPLICIT FORMULAS

Definition E.13 (Value Jacobian and value NTK). For a parameterized model $f_\theta(x)$, define the value Jacobian

$$J_{\text{val}}(x) := \nabla_\theta f_\theta(x),$$

and the (finite-width) value NTK

$$K_{\text{val}}(x, x') := \langle J_{\text{val}}(x), J_{\text{val}}(x') \rangle.$$

Further background can be found in [Jacot et al. \(2018\)](#).

Proposition E.14 (Value NTK for linear-on-PLE is explicit and nonlocal across bins). *For linear-on-PLE (Definition E.5) with parameters $\theta = (w, c)$,*

$$J_{\text{val}}(x) = (\phi(x) \mid 1) \in \mathbb{R}^{T+1}, \quad K_{\text{val}}^{\text{PLE}}(x, x') = 1 + \langle \phi(x), \phi(x') \rangle.$$

Moreover, for $x \in (b_{s-1}, b_s)$ and $x' \in (b_{t-1}, b_t)$ with $s = s(x)$ and $t = s(x')$, letting

$$\alpha(x) := \frac{x - b_{s-1}}{\Delta_s} \in (0, 1), \quad \alpha(x') := \frac{x' - b_{t-1}}{\Delta_t} \in (0, 1),$$

we have the closed form

$$\langle \phi(x), \phi(x') \rangle = (\min\{s, t\} - 1) + \begin{cases} \alpha(x), & s < t, \\ \alpha(x)\alpha(x'), & s = t, \\ \alpha(x'), & s > t. \end{cases}$$

In particular, $K_{\text{val}}^{\text{PLE}}(x, x')$ is typically nonzero even when $s(x) \neq s(x')$.

Proof. For $\theta = (w, c)$, we have $\nabla_w f_{w,c}(x) = \phi(x)$ and $\partial f_{w,c}(x)/\partial c = 1$, hence $J_{\text{val}}(x) = (\phi(x) \mid 1)$ and the kernel formula follows by dot product.

For the binwise closed form, use the explicit structure of ϕ on a bin: if $x \in (b_{s-1}, b_s)$ then

$$\phi_j(x) = \begin{cases} 1, & j \leq s-1, \\ \alpha(x), & j = s, \\ 0, & j \geq s+1, \end{cases}$$

and similarly for x' . Taking the dot product, only coordinates $j \leq \min\{s, t\}$ contribute. If $s < t$, then $\phi_s(x') = 1$ and $\phi_j(x') = 1$ for $j \leq s-1$, giving $\langle \phi(x), \phi(x') \rangle = (s-1) + \alpha(x)$. If $s = t$, the only fractional coordinate is $j = s$, giving $(s-1) + \alpha(x)\alpha(x')$. The case $s > t$ is symmetric. $\square$

Proposition E.15 (Exact value NTK for a one-hidden-layer ReLU MLP (no PLE)). *For the MLP equation 15, define $z_r(x) := w_r x + b_r$ and $\mathbb{I}_r(x) := \mathbf{1}\{z_r(x) > 0\}$. For x away from the kink set $\{z_r(x) = 0\}$, the value Jacobian has entries*

$$\frac{\partial f_\theta(x)}{\partial a_r} = \text{ReLU}(z_r(x)), \quad \frac{\partial f_\theta(x)}{\partial w_r} = a_r x \mathbb{I}_r(x), \quad \frac{\partial f_\theta(x)}{\partial b_r} = a_r \mathbb{I}_r(x), \quad \frac{\partial f_\theta(x)}{\partial c} = 1.$$

Consequently, for such x, x' ,

$$K_{\text{val}}^{\text{MLP}}(x, x') = 1 + \sum_{r=1}^m \left[\text{ReLU}(z_r(x)) \text{ReLU}(z_r(x')) + a_r^2 (1 + x x') \mathbb{I}_r(x) \mathbb{I}_r(x') \right]. \quad (16)$$

Proof. We compute derivatives neuron-by-neuron. First, $\partial f / \partial a_r = \text{ReLU}(z_r(x))$ is immediate. Next, $\partial \text{ReLU}(z) / \partial z = \mathbb{I}_r(x)$ away from $z = 0$, so by chain rule:

$$\frac{\partial f}{\partial w_r} = a_r \frac{\partial \text{ReLU}(z_r(x))}{\partial z_r} \cdot \frac{\partial z_r(x)}{\partial w_r} = a_r \mathbb{I}_r(x) \cdot x, \quad \frac{\partial f}{\partial b_r} = a_r \mathbb{I}_r(x) \cdot 1.$$

Also $\partial f / \partial c = 1$.

Now take the dot product of value Jacobians at x and x' and sum contributions blockwise:

- (a)-block:

$$\sum_{r=1}^m \text{ReLU}(z_r(x)) \text{ReLU}(z_r(x')).$$

- (w)-block:

$$\sum_{r=1}^m (a_r x \mathbb{I}_r(x)) (a_r x' \mathbb{I}_r(x')) = \sum_{r=1}^m a_r^2 x x' \mathbb{I}_r(x) \mathbb{I}_r(x').$$

- (b)-block:

$$\sum_{r=1}^m a_r^2 \mathbb{I}_r(x) \mathbb{I}_r(x').$$

- c-block: 1.

Combining these terms gives equation 16. □

Remark E.16 (Effective dimension / rank: PLE increases value-kernel rank). Consider training on a finite dataset $\{x_i\}_{i=1}^n \subset (b_0, b_T)$. The Gram matrix of the value kernel for linear-on-PILE is

$$\left(K_{\text{val}}^{\text{PLE}}(x_i, x_j) \right)_{i,j} = \mathbf{1}\mathbf{1}^\top + \Phi\Phi^\top, \quad \Phi \in \mathbb{R}^{n \times T}, \quad \Phi_{i,:} = \phi(x_i)^\top.$$

Hence $\text{rank}(K_{\text{val}}^{\text{PLE}}) \leq T + 1$, and for generic data spanning many bins, the rank is typically close to $T + 1$.

By contrast, for the raw linear model $f(x) = \beta x + c$ (no PLE), we have $J_{\text{val}}(x) = (x, 1)$ and thus $K_{\text{val}}(x, x') = xx' + 1$, whose Gram matrix has rank at most 2. This formalizes that PLE increases the number of independent “directions” in which function values can be fitted in the kernel (lazy-training) picture.

Remark E.17 (Refinement under local input changes: PLE’s value kernel is cumulative and continuous). Both $K_{\text{val}}^{\text{PLE}}$ and $K_{\text{val}}^{\text{MLP}}$ can be nonlocal. However, $K_{\text{val}}^{\text{PLE}}(x, x')$ varies *continuously* with x within each bin and changes only at the fixed knots $\{b_t\}$, giving a cumulative “similarity” structure along the scalar feature (Proposition E.14). In contrast, for fixed MLP parameters, $K_{\text{val}}^{\text{MLP}}(x, x')$ changes its algebraic form whenever x crosses one of the learned thresholds $-b_r/w_r$ (activation pattern changes), which can produce more abrupt changes in similarity as a function of x . This is a qualitative comparison (not a nonlocality claim): it describes how PLE induces a stable, pre-specified partition of x -space.

E.4.2. SLOPE NTK: DEFINITIONS AND EXACT FINITE-WIDTH FORMULAS

Definition E.18 (Slope function, slope Jacobian, and slope NTK). For a model $f_\theta(x)$ that is differentiable in x on $I \setminus \mathcal{B}$, define the *slope function*

$$g_\theta(x) := \partial_x f_\theta(x) \quad (x \in I \setminus \mathcal{B}),$$

the *slope Jacobian*

$$J_{\text{slope}}(x) := \nabla_\theta g_\theta(x),$$

and the *slope NTK*

$$K_{\text{slope}}(x, x') := \langle J_{\text{slope}}(x), J_{\text{slope}}(x') \rangle, \quad x, x' \in I \setminus \mathcal{B}.$$

Remark E.19 (Why slope NTK is meaningful). The standard NTK describes learning dynamics of function values $f_\theta(x)$. The slope NTK describes learning dynamics of the derived scalar quantity $\partial_x f_\theta(x)$, i.e. a (coordinate) partial derivative. This makes “local shape” geometry explicit: it quantifies how parameter gradients for slopes at different inputs align. It does not, by itself, imply that ordinary supervised training on function values is governed by the slope NTK; value-loss training affects slopes through a value–slope cross kernel, discussed below.

Proposition E.20 (Exact slope NTK for a one-hidden-layer ReLU MLP (no PLE)). *For the MLP equation 15, for x away from the kink set $\{z_r(x) = 0\}$,*

$$g_\theta(x) = \partial_x f_\theta(x) = \sum_{r=1}^m a_r w_r \mathbb{I}_r(x).$$

Moreover, for such x (in the “a.e.” sense of Remark E.4),

$$\frac{\partial g_\theta(x)}{\partial a_r} = w_r \mathbb{I}_r(x), \quad \frac{\partial g_\theta(x)}{\partial w_r} = a_r \mathbb{I}_r(x), \quad \frac{\partial g_\theta(x)}{\partial b_r} = 0, \quad \frac{\partial g_\theta(x)}{\partial c} = 0.$$

Consequently, for x, x' away from the kink set,

$$K_{\text{slope}}^{\text{MLP}}(x, x') = \sum_{r=1}^m (w_r^2 + a_r^2) \mathbb{I}_r(x) \mathbb{I}_r(x'). \quad (17)$$

Proof. For x with $z_r(x) \neq 0$, we have $\partial_x \text{ReLU}(z_r(x)) = \mathbb{I}_r(x) \cdot \partial_x z_r(x) = \mathbb{I}_r(x) w_r$. Thus,

$$g_\theta(x) = \partial_x f_\theta(x) = \sum_{r=1}^m a_r \partial_x \text{ReLU}(z_r(x)) = \sum_{r=1}^m a_r w_r \mathbb{I}_r(x).$$

Treating $\mathbb{I}_r(x)$ as constant in parameters for this fixed x (valid a.e.), differentiate:

$$\frac{\partial g}{\partial a_r} = w_r \mathbb{I}_r(x), \quad \frac{\partial g}{\partial w_r} = a_r \mathbb{I}_r(x),$$

and $\partial g / \partial b_r = \partial g / \partial c = 0$ a.e. (the dependence on b_r enters only through the gate, whose derivative is supported on the measure-zero set $z_r(x) = 0$).

Finally, take the dot product of slope Jacobians at x and x' . Only the (a) and (w) blocks contribute, giving

$$K_{\text{slope}}^{\text{MLP}}(x, x') = \sum_{r=1}^m \left[(w_r \mathbb{I}_r(x))(w_r \mathbb{I}_r(x')) + (a_r \mathbb{I}_r(x))(a_r \mathbb{I}_r(x')) \right] = \sum_{r=1}^m (w_r^2 + a_r^2) \mathbb{I}_r(x) \mathbb{I}_r(x'),$$

which is equation 17. $\square$

Proposition E.21 (Exact slope NTK for linear-on-PLE is diagonal across bins). *For linear-on-PLE with parameters $\theta = (w, c)$, for $x \in (b_{t-1}, b_t)$,*

$$g_{w,c}(x) = \partial_x f_{w,c}(x) = \frac{w_t}{\Delta_t}, \quad \nabla_w g_{w,c}(x) = \frac{1}{\Delta_t} e_t, \quad \frac{\partial g_{w,c}(x)}{\partial c} = 0.$$

Therefore, for $x, x' \in I \setminus \mathcal{B}$,

$$K_{\text{slope}}^{\text{PLE}}(x, x') = \frac{\mathbf{1}\{s(x) = s(x')\}}{\Delta_{s(x)}^2}. \quad (18)$$

In particular, if $s(x) \neq s(x')$ then $K_{\text{slope}}^{\text{PLE}}(x, x') = 0$ (no cross-bin interference for slope learning).

Proof. For $x \in (b_{t-1}, b_t)$, Theorem E.7 gives $\partial_x f_{w,c}(x) = w_t / \Delta_t$. Differentiating $g(x) = w_t / \Delta_t$ with respect to w yields $\nabla_w g(x) = (1 / \Delta_t) e_t$, and $\partial g / \partial c = 0$. Taking dot products between the one-hot vectors gives equation 18. $\square$

Remark E.22 (Slope NTKs reveal separated local sensitivity degrees of freedom). Comparing the slope NTKs $K_{\text{slope}}^{\text{MLP}}(x, x')$ and $K_{\text{slope}}^{\text{PLE}}(x, x')$ provides a diagnostic view of PLE’s local sensitivity geometry. For linear-on-PLE, the parameter gradients of slopes in different bins are orthogonal: two slope queries interact through $K_{\text{slope}}^{\text{PLE}}$ if and only if they fall within the same bin. This shows that PLE separates local derivative degrees of freedom, rather than tying each first-layer parameter globally to the raw input.

Remark E.23 (Ordinary value-loss training is not strictly bin-local). For standard supervised losses, gradient flow is driven by value residuals, not by slope labels. In squared-loss gradient flow, for example,

$$\dot{\theta} = - \sum_i (f_{\theta}(x_i) - y_i) \nabla_{\theta} f_{\theta}(x_i),$$

and therefore the slope evolves as

$$\frac{d}{dt} \partial_x f_{\theta}(x) = - \sum_i (f_{\theta}(x_i) - y_i) \langle \nabla_{\theta} \partial_x f_{\theta}(x), \nabla_{\theta} f_{\theta}(x_i) \rangle.$$

Thus the relevant kernel for how value-loss training changes slopes is the value–slope cross kernel

$$K_{\text{slope, val}}(x, x_i) := \langle J_{\text{slope}}(x), J_{\text{val}}(x_i) \rangle,$$

not only the slope–slope kernel. For linear-on-PLE this cross kernel inherits the cumulative structure of the value representation, so ordinary value training is not strictly bin-local. The slope NTK result should therefore be read as a structural statement: PLE gives separate local slope degrees of freedom, while value fitting still propagates through the cumulative spline basis.

Remark E.24 (Extension: one-hidden-layer MLP on top of PLE (why diagonality can fail)). If a ReLU MLP is applied to $\phi(x)$, e.g.

$$h_{\Theta}(x) = \sum_{r=1}^m a_r \text{ReLU}(u_r^{\top} \phi(x) + b_r),$$

then (a.e.) $\partial_x h_{\Theta}(x) = \sum_r a_r \mathbf{1}\{u_r^{\top} \phi(x) + b_r > 0\} (u_r^{\top} \phi'(x))$. Since $\phi'(x)$ is one-hot by bin, $u_r^{\top} \phi'(x)$ selects a bin-dependent coordinate of u_r . However, the gate $\mathbf{1}\{u_r^{\top} \phi(x) + b_r > 0\}$ can still couple different bins, so exact bin-diagonality like equation 18 is not guaranteed in general.

F. Appendix: Hier-Prox Convergence to a Noisier Ball and its Local Optimality

The practical LassoNet update applies a stochastic gradient step with stepsize $\eta > 0$, then applies the hierarchical proximal operator with *fixed* (λ, M) for many epochs.

Algorithm 4 Stochastic Hier-Prox update with fixed (η, λ, M)

Require: initial $y_0 = (\theta_0, W_0)$, stepsize $\eta > 0$, prox weight $\lambda > 0$, hierarchy parameter $M > 0$

- 1: **for** $t = 0, 1, 2, \dots$ **do**
 - 2: obtain stochastic gradient g_t (e.g. minibatch gradient) of f at y_t
 - 3: $z_t \leftarrow y_t - \eta g_t$
 - 4: $y_{t+1} \leftarrow \Psi_{\lambda, M}(z_t)$ ▷ LassoNet Hier-Prox operator (blockwise sorting per feature)
-

We analyze the constant-hyperparameter regime used in practice (learning rate η and regularization λ held independent and constant over many epochs), which naturally leads to a Lyapunov analysis in *function values* for the composite potential $F(y) = \eta f(y) + \Omega(y)$. Under (i) exact global solution of each proximal subproblem by HIER-PROX, (ii) boundedness of iterates on a visited domain, (iii) L -smoothness of f , and (iv) a local *proximal-PL* inequality stated for the exact hierarchical prox-gradient mapping, we prove a master recursion

$$\Delta_{t+1} \leq (1 - c) \Delta_t + C_{\text{noise}} \eta^2 \sigma^2 + C_{\text{sign}} R^2 q_t,$$

where $\Delta_t = \mathbb{E}[F(y_t) - F(y^*)]$ and q_t is the probability that stochastic gradient noise flips the active sign/branch between the noiseless and noisy pre-prox points. We then give *SNR/margin* bounds on q_t (Chebyshev), making explicit how weak signals (small margin) lead to large branch-switch variance, causing training instability.

F.1. Practical Algorithmic Scaling and the Lyapunov Function

F.1.1. THE STOCHASTIC UPDATE

We focus on the block of parameters to which HIER-PROX is applied:

$$y = (\theta, W) \in \mathbb{R}^d \times \mathbb{R}^{d \times K},$$

where $\theta \in \mathbb{R}^d$ act as feature “gates” and $W \in \mathbb{R}^{d \times K}$ are the first-layer weights (row j is $W_{j,\cdot}$) of the MLP NN. Let $f : \mathbb{R}^{d+dK} \rightarrow \mathbb{R}$ be the (possibly nonconvex) smooth training loss as a function of this block.²

Training uses a stochastic gradient g_t (e.g. a minibatch gradient) and then applies the hierarchical proximal map:

$$z_t = y_t - \eta g_t, \quad y_{t+1} = \Psi(z_t), \quad (19)$$

where the proximal mapping is

$$\Psi(z) \in \arg \min_y \left\{ \Omega(y) + \frac{1}{2} \|y - z\|^2 \right\}. \quad (20)$$

Where $\Omega(y)$ is defined in the subsequent section.

F.1.2. THE HIERARCHICAL REGULARIZER/CONSTRAINT

The LassoNet hierarchical structure couples each $(\theta_j, W_{j,\cdot})$ through

$$\|W_{j,\cdot}\|_\infty \leq M|\theta_j| \quad (j = 1, \dots, d),$$

and adds an ℓ_1 penalty $\lambda\|\theta\|_1$. We package these into an extended-valued function $\Omega : \mathbb{R}^{d+dK} \rightarrow \mathbb{R} \cup \{+\infty\}$:

$$\Omega(\theta, W) := \lambda\|\theta\|_1 + \sum_{j=1}^d \mathbf{1}\{\|W_{j,\cdot}\|_\infty \leq M|\theta_j|\}, \quad (21)$$

where $\mathbf{1}\{\cdot\}$ is 0 if the condition holds and $+\infty$ otherwise.

Remark F.1 (Why this scaling differs from standard proximal-gradient textbooks). Classical proximal-gradient methods typically use $\arg \min_y \{\Omega(y) + \frac{1}{2\eta}\|y - z\|^2\}$, so that Ω is scaled by η . In the original LassoNet training regime, η , λ , and M are typically treated as *independent constants* over many epochs. The proximal step equation 20 therefore has a fixed quadratic weight $1/2$. To obtain a clean descent identity, we analyze the Lyapunov function F in the next section, rather than $f + \Omega$.

F.1.3. THE LYAPUNOV FUNCTION

Define the potential

$$F(y) := \eta f(y) + \Omega(y). \quad (22)$$

The key intuition is that equation 19 is precisely a stochastic descent method for F when η is small enough, up to additional terms coming from noise and branch switching.

F.1.4. ASSUMPTIONS

Assumption F.2 (Bounded iterates / bounded prox outputs). There exists $R > 0$ such that $\|y_t\| \leq R$ almost surely for all t . Equivalently (and slightly more robustly), we may assume $\|\Psi(z_t)\| \leq R$ almost surely.

Assumption F.3 (Unbiased gradients and bounded conditional variance). The stochastic gradients satisfy $\mathbb{E}[g_t \mid y_t] = \nabla f(y_t)$ and

$$\mathbb{E}[\|g_t\|^2 \mid y_t] \leq \sigma^2, \quad \xi_t := g_t - \nabla f(y_t).$$

Assumption F.4 (Coordinate-wise noise variances). For each t and j , the conditional variance of the gate-noise coordinate is bounded:

$$\mathbb{E}[\xi_{\theta,j,t}^2 \mid y_t] \leq \nu_{j,t}^2.$$

²In the full neural network, other parameters may also be present; one can view them as frozen for our analysis.

Assumption F.5 (L -smoothness of the loss). The function f has L -Lipschitz gradient on a neighborhood containing $\{y_t\}$:

$$f(y') \leq f(y) + \langle \nabla f(y), y' - y \rangle + \frac{L}{2} \|y' - y\|^2.$$

Assumption F.6 (Proximal-PL inequality for F). There exists $\mu_{\text{PL}} > 0$ and a reference value F^* such that for all iterates y_t ,

$$\frac{1}{2} \|G(y_t)\|^2 \geq \mu_{\text{PL}} (F(y_t) - F^*).$$

F.2. Geometry of HIER-PROX: Union of Cones and Branchwise Convex Prox

The mapping Ψ in equation 20 is separable across features j because Ω in equation 21 is a sum of featurewise terms and the squared norm decomposes into a sum. Thus it suffices to study the one-feature problem and then lift the results to the full map.

F.2.1. ONE FEATURE: THE FEASIBLE SET IS A UNION OF TWO CONVEX CONES

Fix one feature and drop index j . Write the input to the proximal map as $x = (v, u) \in \mathbb{R} \times \mathbb{R}^K$, and the proximal variable as $(b, w) \in \mathbb{R} \times \mathbb{R}^K$, corresponding to $(\theta_j, W_{j,\cdot})$.

Define the feasible set

$$\mathcal{C} := \{(b, w) \in \mathbb{R} \times \mathbb{R}^K : \|w\|_\infty \leq M|b|\}.$$

Introduce the two sign cones

$$\mathcal{C}^+ := \{(b, w) : b \geq 0, \|w\|_\infty \leq Mb\}, \quad \mathcal{C}^- := \{(b, w) : b \leq 0, \|w\|_\infty \leq -Mb\},$$

so that $\mathcal{C} = \mathcal{C}^+ \cup \mathcal{C}^-$. Each $\mathcal{C}^\pm$ is a closed convex cone, but their union is nonconvex.

F.2.2. BRANCHWISE PROXIMAL MAPS AND FIRM NONEXPANSIVENESS

Define the one-feature branch objective

$$G_s(b, w) := \lambda s b + \mathbf{1}_{\mathcal{C}^s}(b, w), \quad s \in \{+1, -1\}.$$

Since $\mathcal{C}^s$ is convex and $b \mapsto \lambda s b$ is linear, G_s is proper, closed, and convex.

Definition F.7 (One-feature branch optimization problem). For $s \in \{\pm 1\}$ define

$$\psi^{(s)}(x) := \arg \min_{(b, w) \in \mathbb{R} \times \mathbb{R}^K} \left\{ G_s(b, w) + \frac{1}{2} \|(b, w) - x\|^2 \right\}. \quad (23)$$

We will show the branch problem's optimum is well defined.

Lemma F.8 (Two-branch convexification and uniqueness on each branch). *For each $s \in \{+1, -1\}$, the set $\mathcal{C}^s$ is a nonempty closed convex set. For each (v, u) , the branch problem $\min \psi^{(s)}$ has a unique minimizer $\psi^{(s)}(v, u)$.*

Proof. **Step 1 (Closed convexity).** Consider $s = +1$; the case $s = -1$ follows by the change of variables $(b, W) \mapsto (-b, W)$. We can write

$$\mathcal{C}^+ = \{(b, W) : b \geq 0\} \cap \bigcap_{i=1}^K \{(b, W) : -Mb \leq W_i \leq Mb\}.$$

Each set on the right is closed and convex; intersections preserve both. Nonemptiness holds since $(0, 0)$ is feasible.

Step 2 (Strong convexity). The quadratic term $(b, W) \mapsto \frac{1}{2}(v - b)^2 + \frac{1}{2}\|u - W\|^2$ is 1-strongly convex. Adding the linear term $\lambda s b$ preserves strong convexity. Restricting to a closed convex set by adding $\iota_{\mathcal{C}^s}$ retains strong convexity on the domain.

Step 3 (Existence and uniqueness). Coercivity of the quadratic ensures existence of a minimizer; strong convexity implies uniqueness. $\square$

Definition F.9 (Global minimizer set). Let $\psi_{\text{set}}(v, u)$ be the (possibly set-valued) global minimizer set of equation 23:

$$\psi_{\text{set}}(v, u) := \arg \min_{(b, W) \in \mathcal{C}} \frac{1}{2}(v - b)^2 + \frac{1}{2} \|u - W\|^2 + \lambda|b|.$$

Lemma F.10 (Global minimizers lie on one of the two branches). *For any (v, u) ,*

$$\psi_{\text{set}}(v, u) \subseteq \{\psi^{(+1)}(v, u), \psi^{(-1)}(v, u)\}.$$

Moreover, if the two branch optimal values differ, then the global minimizer is unique.

Proof. Since $\mathcal{C} = \mathcal{C}^+ \cup \mathcal{C}^-$, the global optimum equals the minimum of the two branch optima. On $\mathcal{C}^+$, $|b| = b$ and the objective equals $G_{+1}(b, w) + \frac{1}{2} \|(b, w) - x\|^2$; on $\mathcal{C}^-$, $|b| = -b$ and the objective equals $G_{-1}(b, w) + \frac{1}{2} \|(b, w) - x\|^2$. Any global minimizer must lie in one branch and therefore must coincide with that branch's unique minimizer (Lemma F.8). If branch values differ, only one branch attains the minimum, yielding uniqueness. $\square$

Lemma F.11 (Branch map is a convex proximal operator). *For each $s \in \{\pm 1\}$, the map $\psi^{(s)}$ is the proximal operator of the convex function G_s :*

$$\psi^{(s)}(x) = \text{prox}_{G_s}(x).$$

In particular, $\psi^{(s)}$ is firmly nonexpansive and thus 1-Lipschitz:

$$\left\| \psi^{(s)}(x) - \psi^{(s)}(x') \right\|^2 \leq \langle \psi^{(s)}(x) - \psi^{(s)}(x'), x - x' \rangle \leq \|x - x'\|^2.$$

Proof. Fix the sign s . By definition equation 23 and lemma F.10, $\psi^{(s)}(x)$ is exactly the minimizer of $G_s(\cdot) + \frac{1}{2} \|\cdot - x\|^2$, which is the definition of $\text{prox}_{G_s}(x)$. Firm nonexpansiveness is a standard property of proximal operators of proper closed convex functions. $\square$

F.2.3. THE GLOBAL ONE-FEATURE MAP AND SIGN SELECTION

The HIER-PROX rule selects a sign branch based on the input v . We define $\text{sign}(0) := +1$ as a deterministic tie-breaking convention.

Definition F.12 (Global one-feature map). Define $\psi : \mathbb{R} \times \mathbb{R}^K \rightarrow \mathbb{R} \times \mathbb{R}^K$ by

$$\psi(v, u) := \psi^{(\text{sign}(v))}(v, u).$$

Lemma F.13 (Sign selection is globally optimal except at ties). *Let $x = (v, u)$ and let (b^*, w^*) be any minimizer of the one-feature proximal problem*

$$\min_{(b, w) \in \mathcal{C}} \left\{ \lambda|b| + \frac{1}{2}(b - v)^2 + \frac{1}{2} \|w - u\|^2 \right\}.$$

If $v \neq 0$, then every minimizer satisfies $\text{sign}(b^) = \text{sign}(v)$. When $v = 0$, both signs can be optimal, and we select the $+$ branch by convention.*

Proof. Fix any feasible pair $(b, w) \in \mathcal{C}$. Because the constraint depends on $|b|$, the pair $(-b, w)$ is also feasible and has the same $\lambda|b|$ and $\|w - u\|^2$ terms. The difference in objective values is entirely in the quadratic:

$$\frac{1}{2}(b - v)^2 - \frac{1}{2}(-b - v)^2 = \frac{1}{2}(b^2 - 2bv + v^2 - (b^2 + 2bv + v^2)) = -2bv.$$

If $v > 0$ and $b > 0$, this difference is negative, so (b, w) is strictly better than $(-b, w)$. If $v > 0$ and $b < 0$, then $(-b, w)$ is strictly better. Thus for $v > 0$ any minimizer must have $b \geq 0$. An analogous argument gives $b \leq 0$ when $v < 0$. When $v = 0$, the two objectives are equal for $\pm b$, so ties are possible. $\square$

F.2.4. LIFTING TO d FEATURES

Define the full mapping Ψ in equation 20 by applying the one-feature map ψ featurewise. Equivalently, define the sign pattern of a point $z = (\theta, W)$ by

$$s(z) := (\text{sign}(\theta_1), \dots, \text{sign}(\theta_d)) \in \{\pm 1\}^d,$$

and let $\Psi^{(s)}$ denote the map obtained by applying the branch maps $\psi^{(s_j)}$ to each feature j . Then

$$\Psi(z) = \Psi^{(s(z))}(z), \quad \text{and for each fixed } s, \Psi^{(s)} \text{ is firmly nonexpansive (hence 1-Lipschitz).}$$

F.2.5. A NEGATIVE RESULT: THE GLOBAL MAP NEED NOT BE LIPSCHITZ

The next proposition makes precise what *cannot* be extended from standard proximal mappings.

Proposition F.14 (Discontinuity at the sign boundary; not globally Lipschitz). *Consider the one-feature case $K = 1$ with parameters $M > 0$ and $\lambda \geq 0$. Fix an input $u > 0$ and define the one-feature HIER-PROX map $v \mapsto \psi(v, u) = (b(v), w(v))$. If $Mu > \lambda$, then*

$$\lim_{v \downarrow 0} b(v) = +\frac{Mu - \lambda}{1 + M^2}, \quad \lim_{v \uparrow 0} b(v) = -\frac{Mu - \lambda}{1 + M^2},$$

while $b(0) = +\frac{Mu - \lambda}{1 + M^2}$ under the convention $\text{sign}(0) = +1$. In particular, the map $v \mapsto \psi(v, u)$ is discontinuous at $v = 0$ and therefore not Lipschitz.

Proof. With $K = 1$, the constraint is $|w| \leq M|b|$ and the one-feature prox problem is

$$\min_{b, w} \left\{ \lambda|b| + \frac{1}{2}(b - v)^2 + \frac{1}{2}(w - u)^2 : |w| \leq M|b| \right\}.$$

For any fixed b , the minimizer over w is the Euclidean projection of u onto $[-M|b|, M|b|]$, hence

$$w^*(b) = \min\{u, M|b|\} \quad (\text{since } u > 0),$$

and the resulting reduced objective is

$$\phi_v(b) = \lambda|b| + \frac{1}{2}(b - v)^2 + \frac{1}{2}(u - M|b|)_+^2,$$

where $(t)_+ = \max\{t, 0\}$. The reduced objective is generally nonconvex in b because of the term $|b|$ inside the hinge. We therefore analyze the two sign branches separately, where the reduced objective is convex.

We first analyze $v \geq 0$ and minimize over $b \geq 0$ (Lemma F.13 shows the minimizer has $b \geq 0$). On $b \in [0, u/M]$, we have $(u - Mb)_+ = u - Mb$ and

$$\phi_v(b) = \lambda b + \frac{1}{2}(b - v)^2 + \frac{1}{2}(u - Mb)^2.$$

Differentiating gives

$$\phi'_v(b) = \lambda + (b - v) - M(u - Mb) = (1 + M^2)b + \lambda - v - Mu.$$

Setting $\phi'_v(b) = 0$ yields the unique critical point

$$b_+(v) = \frac{Mu + v - \lambda}{1 + M^2}.$$

When $Mu > \lambda$ and $v \geq 0$ is sufficiently small, $b_+(v) > 0$. Moreover, for all sufficiently small $v \geq 0$, $b_+(v) < u/M$ (a direct rearrangement using $Mu > \lambda$), so this critical point lies in the region where the expression is valid. Since the $b \geq 0$ branch is convex, this $b_+(v)$ is the unique minimizer on the positive branch in this regime.

By symmetry of ϕ_v in the pair (b, v) through $|b|$ and $|v|$ (and again Lemma F.13), for $v \leq 0$ the minimizer is $b(v) = -b_+(|v|)$. Therefore, as $v \downarrow 0$,

$$b(v) \rightarrow \frac{Mu - \lambda}{1 + M^2}, \quad \text{and as } v \uparrow 0, \quad b(v) \rightarrow -\frac{Mu - \lambda}{1 + M^2}.$$

Under the convention $\text{sign}(0) = +1$, $b(0) = b_+(0) = \frac{Mu - \lambda}{1 + M^2}$, so the map is discontinuous at 0. $\square$

Remark F.15 (Interpretation). The example shows that if the first-layer weight input u is large, the constraint $|w| \leq M|b|$ can force the optimal gate magnitude $|b|$ to remain bounded away from 0, even when $v \approx 0$. Then an infinitesimal change of sign in v can flip the sign of b while keeping its magnitude almost constant, creating a jump. This is exactly the “branch-switch” phenomenon that our stochastic analysis must track.

F.3. Piecewise Lipschitz + Jump: The Instability Decomposition

We now state the key inequality that replaces global Lipschitz continuity. The proof is short because it reduces to “nonexpansive on each branch” plus “bounded jump across branches.”

Remark F.16 (How boundedness is justified in practice). In practical deep-learning pipelines, boundedness is often enforced implicitly or explicitly: (i) warm-start training brings parameters into a stable region before enabling HIER-PROX; (ii) gradient clipping or weight decay prevents divergence; (iii) early stopping prevents excursions to extreme regions. For a theoretical note, we state boundedness as an assumption because it is exactly what controls the jump size.

Lemma F.17 (Piecewise nonexpansive with a jump term). *Let Ψ be the full HIER-PROX map and assume Assumption F.2. Then for all $x, x' \in \mathbb{R}^{d+dK}$,*

$$\|\Psi(x) - \Psi(x')\|^2 \leq \|x - x'\|^2 + J^2 \mathbf{1}\{s(x) \neq s(x')\}, \quad \text{with } J := 2R.$$

Proof. If $s(x) = s(x')$, then $\Psi(x) = \Psi^{(s)}(x)$ and $\Psi(x') = \Psi^{(s)}(x')$ for the same sign pattern s . Since each branch map is firmly nonexpansive and the full map is a product of branch maps, $\|\Psi(x) - \Psi(x')\| \leq \|x - x'\|$, hence the squared inequality holds with no J term.

If $s(x) \neq s(x')$, then by Assumption F.2,

$$\|\Psi(x) - \Psi(x')\| \leq \|\Psi(x)\| + \|\Psi(x')\| \leq 2R,$$

so $\|\Psi(x) - \Psi(x')\|^2 \leq 4R^2 = J^2$. Combining the two cases yields the claim. $\square$

Corollary F.18 (Expectation form with conditional branch-switch probability). *Fix a σ -field $\mathcal{F}$ and random variables X, X' measurable w.r.t. $\mathcal{F}$. Let $B := \{s(X) \neq s(X')\}$ and define $q := \mathbb{P}(B \mid \mathcal{F})$. Then*

$$\mathbb{E}[\|\Psi(X) - \Psi(X')\|^2 \mid \mathcal{F}] \leq \mathbb{E}[\|X - X'\|^2 \mid \mathcal{F}] + J^2 q.$$

Proof. Apply Lemma F.17 pointwise and take conditional expectation. $\square$

F.3.1. A SIMPLE SIGNAL-TO-NOISE BOUND FOR BRANCH SWITCHING (CHEBYSHEV)

We now connect q_t to a familiar ML notion: *margin* versus *noise*.

Definition F.19 (Noise and margin for the gate coordinates). Let $\xi_t := g_t - \nabla f(y_t)$ be the stochastic gradient noise, and define the “noiseless” pre-prox point

$$\bar{z}_t := y_t - \eta \nabla f(y_t), \quad z_t = \bar{z}_t - \eta \xi_t.$$

Write $\bar{z}_t = (\bar{\theta}_t, \bar{W}_t)$ and define the coordinate-wise margins

$$m_{j,t} := |(\bar{\theta}_t)_j| \quad (j = 1, \dots, d).$$

Define the branch mismatch event

$$B_t := \{s(z_t) \neq s(\bar{z}_t)\}, \quad q_t(y_t) := \mathbb{P}(B_t \mid y_t).$$

Lemma F.20 (Chebyshev SNR bound for branch switches). *Under Definition F.19 and Assumption F.4, if $m_{j,t} > 0$ for all j , then*

$$q_t(y_t) \leq \sum_{j=1}^d \frac{\eta^2 \nu_{j,t}^2}{m_{j,t}^2}.$$

If some $m_{j,t} = 0$, the Chebyshev bound is vacuous for that coordinate; this is precisely the margin-collapse regime discussed below.

Proof. A branch switch occurs if at least one coordinate changes sign:

$$B_t \subseteq \bigcup_{j=1}^d \left\{ \text{sign}((\bar{\theta}_t)_j - \eta \xi_{\theta,j,t}) \neq \text{sign}((\bar{\theta}_t)_j) \right\}.$$

For a fixed j with $m_{j,t} > 0$, a sign change implies $|\eta \xi_{\theta,j,t}| \geq m_{j,t}$. Therefore, by the union bound and Chebyshev's inequality,

$$q_t(y_t) \leq \sum_{j=1}^d \mathbb{P} \left(|\xi_{\theta,j,t}| \geq \frac{m_{j,t}}{\eta} \mid y_t \right) \leq \sum_{j=1}^d \frac{\mathbb{E}[\xi_{\theta,j,t}^2 \mid y_t]}{(m_{j,t}/\eta)^2} \leq \sum_{j=1}^d \frac{\eta^2 \nu_{j,t}^2}{m_{j,t}^2}.$$

If $m_{j,t} = 0$, the implication “sign change $\Rightarrow |\eta \xi_{\theta,j,t}| \geq m_{j,t}$ ” gives no useful lower threshold, so the finite SNR bound above should not be applied to that coordinate. $\square$

Remark F.21 (Margin collapse is exactly when instability can appear). When some margin $m_{j,t}$ becomes small (feature enters/exits, or gate hovers near 0), even moderate noise can create frequent sign flips, making q_t large. This is where the extra sign-switch penalty can dominate.

F.4. Deterministic Descent in Function Values (No Convexity Needed)

F.4.1. THE DETERMINISTIC PROX-GRADIENT MAP

Define the deterministic map

$$T(y) := \Psi(y - \eta \nabla f(y)), \quad G(y) := y - T(y).$$

The vector $G(y)$ is the *prox-gradient mapping*; it vanishes exactly at fixed points of T .

Lemma F.22 (Prox optimality inequality). *Let $y^+ = \Psi(z)$ for some z . Then for all y ,*

$$\Omega(y^+) + \frac{1}{2} \|y^+ - z\|^2 \leq \Omega(y) + \frac{1}{2} \|y - z\|^2.$$

Proof. This is immediate from the definition equation 20 of Ψ as an argmin. $\square$

Lemma F.23 (Deterministic descent for F). *Under Assumption F.5, if $\eta \leq 1/L$, then for all y ,*

$$F(T(y)) \leq F(y) - \frac{1 - \eta L}{2} \|G(y)\|^2.$$

Proof. Let $z = y - \eta \nabla f(y)$ and $y^+ = T(y) = \Psi(z)$. Apply Lemma F.22 with y as the comparator:

$$\Omega(y^+) + \frac{1}{2} \|y^+ - z\|^2 \leq \Omega(y) + \frac{1}{2} \|y - z\|^2.$$

Expanding $z = y - \eta \nabla f(y)$ gives

$$\|y^+ - z\|^2 = \|(y^+ - y) + \eta \nabla f(y)\|^2 = \|y^+ - y\|^2 + 2\eta \langle \nabla f(y), y^+ - y \rangle + \eta^2 \|\nabla f(y)\|^2,$$

and $\|y - z\|^2 = \eta^2 \|\nabla f(y)\|^2$. Canceling the common $\eta^2 \|\nabla f(y)\|^2 / 2$ term yields

$$\Omega(y^+) + \frac{1}{2} \|y^+ - y\|^2 + \eta \langle \nabla f(y), y^+ - y \rangle \leq \Omega(y).$$

By L -smoothness,

$$f(y^+) \leq f(y) + \langle \nabla f(y), y^+ - y \rangle + \frac{L}{2} \|y^+ - y\|^2.$$

Multiply by η and add to the previous inequality to obtain

$$F(y^+) \leq F(y) - \frac{1 - \eta L}{2} \|y^+ - y\|^2.$$

Finally, $\|y^+ - y\| = \|G(y)\|$ by definition, completing the proof. $\square$

Remark F.24 (No convexity is used here). Lemma F.23 uses only L -smoothness of f and exact optimality of the proximal subproblem. It does *not* assume convexity of f or Ω . The role of proximal-PL in the next section is to turn “descent” into a *linear rate*.

F.5. A Proximal-PL Condition and How to Certify It Locally

Remark F.25 (What F^* represents). If F attains a minimum on the relevant region, one can take $F^* = \min_y F(y)$. More generally (in nonconvex settings), proximal-PL can be stated relative to a set of global minimizers or even relative to a best-achievable value along the trajectory. For simplicity, we keep the scalar F^* .

F.5.1. LOCAL CERTIFICATION HEURISTICS

The proximal-PL condition is an assumption, but it is not arbitrary. Below are two practically relevant routes that can justify it *locally*.

- **Stable sign pattern + margin.** If the sign pattern $s(y - \eta \nabla f(y))$ remains fixed on a region and the margins are bounded away from 0, then Ψ reduces to a *single convex proximal operator* $\Psi^{(s)}$ on that region. In that case, standard arguments for composite problems on convex sets can be used to derive PL-type inequalities.
- **Empirical local regularity.** In deep learning, it is common that training dynamics enter a well-conditioned region after warm-start. Proximal-PL can be viewed as a formalization of “loss landscape is locally sharp” for the relevant trajectory, after the hier-prox operation.

Remark F.26 (We do not require global PL). Nothing in the stochastic analysis requires proximal-PL to hold for all $y \in \mathbb{R}^{d+K}$. It only needs to hold along (or near) the iterates that the algorithm visits.

F.6. Stochastic Recursion: Noise + Sign-Switch Penalty

F.6.1. A CONDITIONAL DESCENT INEQUALITY

Lemma F.27 (Stochastic descent with a noise cross term). *Assume Assumptions F.5 and F.3. If $\eta \leq 1/L$, then almost surely*

$$F(y_{t+1}) \leq F(y_t) - \frac{1 - \eta L}{2} \|y_{t+1} - y_t\|^2 - \eta \langle \xi_t, y_{t+1} - y_t \rangle.$$

Proof. Apply Lemma F.22 to $y_{t+1} = \Psi(z_t)$ with comparator $y = y_t$ and $z_t = y_t - \eta g_t$:

$$\Omega(y_{t+1}) + \frac{1}{2} \|y_{t+1} - z_t\|^2 \leq \Omega(y_t) + \frac{1}{2} \|y_t - z_t\|^2.$$

Expand as in Lemma F.23:

$$\Omega(y_{t+1}) + \frac{1}{2} \|y_{t+1} - y_t\|^2 + \eta \langle g_t, y_{t+1} - y_t \rangle \leq \Omega(y_t).$$

By L -smoothness,

$$\eta f(y_{t+1}) \leq \eta f(y_t) + \eta \langle \nabla f(y_t), y_{t+1} - y_t \rangle + \frac{\eta L}{2} \|y_{t+1} - y_t\|^2.$$

Add the two inequalities, use $F = \eta f + \Omega$, and substitute $g_t = \nabla f(y_t) + \xi_t$ to get the claim. $\square$

Lemma F.28 (Young inequality removes the cross term). *Fix any $\alpha > 0$. Then*

$$-\eta \langle \xi_t, y_{t+1} - y_t \rangle \leq \frac{\eta^2}{2\alpha} \|\xi_t\|^2 + \frac{\alpha}{2} \|y_{t+1} - y_t\|^2.$$

Proof. This is the inequality $-\langle a, b \rangle \leq \frac{1}{2\alpha} \|a\|^2 + \frac{\alpha}{2} \|b\|^2$ with $a = \eta \xi_t$ and $b = y_{t+1} - y_t$. $\square$

Corollary F.29 (Conditional descent in terms of $\mathbb{E} \|y_{t+1} - y_t\|^2$). *Under Assumptions F.5 and F.3, if $\eta \leq 1/L$ and $\alpha = (1 - \eta L)/2$, then*

$$\mathbb{E}[F(y_{t+1}) \mid y_t] \leq F(y_t) - \frac{1 - \eta L}{4} \mathbb{E}[\|y_{t+1} - y_t\|^2 \mid y_t] + \frac{\eta^2}{1 - \eta L} \mathbb{E}[\|\xi_t\|^2 \mid y_t].$$

Proof. Combine Lemma F.27 and Lemma F.28 and take conditional expectation. $\square$

F.6.2. RELATING $\mathbb{E} \|y_{t+1} - y_t\|^2$ TO THE DETERMINISTIC PROX-GRADIENT MAPPING

Let $\bar{z}_t = y_t - \eta \nabla f(y_t)$ and $\bar{y}_{t+1} := T(y_t) = \Psi(\bar{z}_t)$. Then $G(y_t) = y_t - \bar{y}_{t+1}$.

Lemma F.30 (Lower bound with a branch-switch penalty). *Assume Assumptions F.2 and F.3. Then*

$$\mathbb{E} [\|y_{t+1} - y_t\|^2 \mid y_t] \geq \frac{1}{2} \|G(y_t)\|^2 - \eta^2 \sigma^2 - J^2 q_t(y_t), \quad J = 2R,$$

where $q_t(y_t) = \mathbb{P}(s(z_t) \neq s(\bar{z}_t) \mid y_t)$.

Proof. Write

$$y_{t+1} - y_t = (\bar{y}_{t+1} - y_t) + (y_{t+1} - \bar{y}_{t+1}) = -G(y_t) + (y_{t+1} - \bar{y}_{t+1}).$$

Using the inequality $\|a + b\|^2 \geq \frac{1}{2} \|a\|^2 - \|b\|^2$ (valid for all vectors a, b), we obtain

$$\|y_{t+1} - y_t\|^2 \geq \frac{1}{2} \|G(y_t)\|^2 - \|y_{t+1} - \bar{y}_{t+1}\|^2.$$

Now apply Corollary F.18 with $\mathcal{F} = \sigma(y_t)$, $X = z_t$ and $X' = \bar{z}_t$:

$$\mathbb{E} [\|y_{t+1} - \bar{y}_{t+1}\|^2 \mid y_t] = \mathbb{E} [\|\Psi(z_t) - \Psi(\bar{z}_t)\|^2 \mid y_t] \leq \mathbb{E} [\|z_t - \bar{z}_t\|^2 \mid y_t] + J^2 q_t(y_t).$$

Finally, $z_t - \bar{z}_t = -\eta \xi_t$, so $\mathbb{E} [\|z_t - \bar{z}_t\|^2 \mid y_t] = \eta^2 \mathbb{E} [\|\xi_t\|^2 \mid y_t] \leq \eta^2 \sigma^2$. Substituting into the previous inequality completes the proof. $\square$

F.6.3. THE MASTER RECURSION

Theorem F.31 (One-step recursion: noise + sign-switch penalty). *Assume Assumptions F.5, F.2, F.3, and F.6, and let $\eta \leq 1/L$. Define $\Delta_t := \mathbb{E}[F(y_t) - F^*]$. Then*

$$\mathbb{E}[F(y_{t+1}) \mid y_t] - F^* \leq \left(1 - \frac{1 - \eta L}{4} \mu_{\text{PL}}\right) (F(y_t) - F^*) + C_{\text{noise}} \eta^2 \sigma^2 + C_{\text{sign}} J^2 q_t(y_t),$$

where $J = 2R$ and

$$C_{\text{noise}} = \frac{1 - \eta L}{4} + \frac{1}{1 - \eta L}, \quad C_{\text{sign}} = \frac{1 - \eta L}{4}.$$

Proof. Start from Corollary F.29 and plug in Lemma F.30:

$$\begin{aligned} \mathbb{E}[F(y_{t+1}) \mid y_t] &\leq F(y_t) - \frac{1 - \eta L}{4} \left(\frac{1}{2} \|G(y_t)\|^2 - \eta^2 \sigma^2 - J^2 q_t(y_t) \right) + \frac{\eta^2}{1 - \eta L} \sigma^2 \\ &= F(y_t) - \frac{1 - \eta L}{8} \|G(y_t)\|^2 + \left(\frac{1 - \eta L}{4} + \frac{1}{1 - \eta L} \right) \eta^2 \sigma^2 + \frac{1 - \eta L}{4} J^2 q_t(y_t). \end{aligned}$$

Now apply the proximal-PL inequality (Assumption F.6) at y_t : $\frac{1}{2} \|G(y_t)\|^2 \geq \mu_{\text{PL}}(F(y_t) - F^*)$, i.e. $\|G(y_t)\|^2 \geq 2\mu_{\text{PL}}(F(y_t) - F^*)$. Substitute this lower bound into the negative term to obtain the stated contraction. $\square$

Corollary F.32 (Unconditional recursion). *Under the assumptions of Theorem F.31,*

$$\Delta_{t+1} \leq \left(1 - \frac{1 - \eta L}{4} \mu_{\text{PL}}\right) \Delta_t + C_{\text{noise}} \eta^2 \sigma^2 + C_{\text{sign}} J^2 \mathbb{E}[q_t(y_t)].$$

Proof. Take expectation of Theorem F.31 over y_t . $\square$

F.7. Summary Theorem and Interpretations

F.7.1. A CLEAN BOUND UNDER A MARGIN CONDITION

To obtain a fully explicit bound, we now assume a margin lower bound. This is a standard “signal-to-noise” assumption: the deterministic pre-prox gates do not hover near zero.

Assumption F.33 (Uniform margin lower bound). There exist constants $\underline{m}_1, \dots, \underline{m}_d > 0$ such that for all t ,

$$m_{j,t} = |(\bar{\theta}_t)_j| \geq \underline{m}_j \quad \text{almost surely.}$$

Theorem F.34 (Linear convergence to a “noise + sign” neighborhood). Assume Assumptions F.5, F.2, F.3, F.6, F.4, and F.33, and let $\eta \leq 1/L$. Define

$$\rho := 1 - \frac{1 - \eta L}{4} \mu_{\text{PL}} \in (0, 1).$$

Then for all $t \geq 0$,

$$\Delta_t \leq \rho^t \Delta_0 + \frac{1}{1 - \rho} \left(C_{\text{noise}} \eta^2 \sigma^2 + C_{\text{sign}} J^2 \eta^2 \sum_{j=1}^d \frac{\sup_t \nu_{j,t}^2}{\underline{m}_j^2} \right), \quad J = 2R.$$

Proof. By Lemma F.20 and Assumption F.33,

$$q_t(y_t) \leq \sum_{j=1}^d \frac{\eta^2 \nu_{j,t}^2}{\underline{m}_{j,t}^2} \leq \eta^2 \sum_{j=1}^d \frac{\nu_{j,t}^2}{\underline{m}_j^2}.$$

Taking expectation yields the same bound for $\mathbb{E}[q_t(y_t)]$. Plug this into Corollary F.32 to obtain a linear recursion $\Delta_{t+1} \leq \rho \Delta_t + \beta \eta^2$, with β equal to the parenthesized constant. Unrolling the recursion gives the stated formula. $\square$

F.7.2. INTERPRETATIONS

Remark F.35 (When HIER-PROX behaves like standard Prox-SGD). If the margin lower bound holds (gates are not near zero), then the branch-switch probability scales like η^2 and is typically small. In that regime, the last term in Theorem F.34 is of the same order as the usual SGD noise term.

Remark F.36 (When instability is expected). If some margins collapse (gates hover near 0), the bound in Lemma F.20 becomes large, and sign flips can occur frequently. Theorem F.31 then predicts a large additive penalty, which can dominate the dynamics even when σ^2 is moderate. This matches the empirical observation that feature enter/exit events are periods of instability.

F.7.3. VARIANCE TRANSFER VIEWPOINT

The piecewise nonexpansiveness inequality also yields a “variance transfer” message.

Proposition F.37 (Variance transfer: within-branch nonexpansive, across-branch penalty). Let X be a random pre-prox input and let $\bar{X}$ be an independent copy with the same sign pattern (i.e. $s(X) = s(\bar{X})$ almost surely). Then $\text{Var}(\Psi(X)) \leq \text{Var}(X)$ (nonexpansive). In general, for any X ,

$$\mathbb{E} \|\Psi(X) - \mathbb{E}\Psi(X)\|^2 \leq \mathbb{E} \|X - \mathbb{E}X\|^2 + J^2 \mathbb{P}(s(X) \neq s(\mathbb{E}X)),$$

so variance can increase only through a branch-switch-type event.

Proof. The first statement follows from firm nonexpansiveness on a fixed branch. The second statement is a direct application of Lemma F.17 with $x' = \mathbb{E}X$ and Jensen’s inequality. $\square$

F.8. Discussion: Why Variance Reduction Is a Promising Direction

The master recursion (Theorem F.31) isolates two drivers of the final neighborhood size:

1. the classical noise term, scaling like $\eta^2 \sigma^2$;
2. the sign-switch term, scaling like $J^2 \mathbb{E}[q_t(y_t)]$, which itself can be controlled by a margin-to-noise ratio.

This suggests a practical research direction: *stabilize HIER-PROX by reducing gradient noise*, thereby reducing both σ^2 and the switch probability. While full variance-reduction methods (SVRG/SAGA) can be costly in deep learning, lighter-weight options (larger batch size, gradient accumulation, EMA smoothing, etc.) may already reduce sign flips noticeably.

Takeaway. LassoNet’s hierarchical feature-selection mechanism introduces a fundamental discrete instability channel. Our analysis proves that controlling this channel—particularly the variance induced by gate sign transitions—is critical for stable convergence. This theoretical insight directly motivates the exponential moving average (EMA) stabilization introduced in our Seq-Hier-Prox-EMA algorithm, which effectively dampens the branch-switch probability q_t by reducing the effective gradient noise.

F.9. LassoFlexNet Training

Algorithm 5 Update Algorithm for LassoFlexNet

- 1: **Input:** training dataset $X \in \mathbb{R}^{n \times d}$, training labels Y , linear skip module LS_β , per-feature embedding ResNet $NN_\phi(\cdot)$, main neural network $NN_W(\cdot)$ (e.g. Mixer), number of epochs B , hierarchy constrain multiplier M , scalar penalty path $\Lambda = [\lambda_0, \dots, \lambda_N]$ (each $\lambda_i \in \mathbb{R}_+$), learning rate α ,
 - 2: Initialize and train the joint model on the loss $L(\beta, \phi, W)$
 - 3: Initialize the penalty, $\lambda = \lambda_0$, and the number of active features, $k = d$
 - 4: **while** $k > 0$ **do**
 - 5: Get $\lambda \in \Lambda$ iteratively ▷ scalar penalty drawn from the path
 - 6: **for** $b = 1$ to B **do**
 - 7: Compute gradient of the loss w.r.t to (β, ϕ, W) using back-propagation
 - 8: Update $\beta \leftarrow \beta - \alpha \nabla_\beta L$, $\phi \leftarrow \phi - \alpha \nabla_\phi L$, and $W \leftarrow W - \alpha \nabla_W L$
 - 9: Update $(\tilde{\beta}, \tilde{\phi}, \tilde{W}^{(1)}) \leftarrow \text{Coordinate Hier-Prox}(\beta, \phi, W^{(1)}, \alpha \lambda, M)$
 - 10: Update k to be the number of non-zero coordinates of β
-

F.10. Sequential Hierarchical Update: Blockwise Derivation

Our derivation is based on Proposition 1 in (Lemhadri et al., 2021a), which we provide here for completeness and reference:

Proposition 1 (Joint optimization): Fix $v \in \mathbb{R}^k$ and $U \in \mathbb{R}^K$. (Note: the two integers k, K can be different) Let us consider the problem:

$$\begin{aligned} & \underset{(b, W)}{\text{minimize}} && \frac{1}{2} (\|v - b\|_2^2 + \|U - W\|_2^2) + \lambda \|b\|_2 + \bar{\lambda} \|W\|_1 \\ & \text{subject to} && \|W\|_\infty \leq M \cdot \|b\|_2. \end{aligned}$$

(Lemhadri et al., 2021a) derives the sufficient and necessary condition for characterizing the global optimum (b^*, W^*) of the above optimization problem:

1. Let us order the coordinates $\{|U_i|\}_{i \in [K]}$ in decreasing order

$$|U_{(1)}| \geq |U_{(2)}| \geq \dots \geq |U_{(K)}|.$$

and define for each $s \in [K] = \{0, 1, \dots, K\}$ the value b_s by

$$b_s = \frac{1}{1 + sM^2} \left(1 - \frac{a_s}{\|v\|_2} \right)_+ v \text{ where } a_s = \lambda - M \sum_{i=1}^s (|U_{(i)}| - \bar{\lambda}).$$

Then $b^* = b_{s^*}$ where $s^* \in [K]$ is the unique $s \in [K]$ such that

$$M \|b_s\|_2 \in [\mathcal{S}_{\bar{\lambda}}(|U_{(s+1)}|), \mathcal{S}_{\bar{\lambda}}(|U_{(s)}|)].$$

By convention $\mathcal{S}_{\bar{\lambda}}(|U_{(K+1)}|) = 0$ and $\mathcal{S}_{\bar{\lambda}}(|U_{(0)}|) = \infty$.

2. The W^* must satisfy

$$W^* = \text{sign}(U) \min \{M \|b^*\|_2, \mathcal{S}_{\bar{\lambda}}(|U|)\}.$$

Sequential Optimization: In contrast to the joint optimization of Hier-Prox, we propose a sequential (coordinate-wise) proximal-style update, alternating between b and W . We do not claim that this sequential composition is the proximal map of a single joint objective. Rather, each substep solves its own simple proximal or constrained subproblem. While this sequential approach yields different training dynamics and fixed points compared to joint Hier-Prox, it is highly effective in practice. We validate the empirical superiority of this Seq-Hier-Prox approach in our experiments.

For our application, we care about only the standard Lasso, instead of group Lasso, i.e. b is a scalar. Our derivations will follow this case accordingly.

Linear skip layer step for b : We first characterize the following global optimum with respect to b :

$$\begin{aligned} b^* &= \underset{b}{\operatorname{argmin}} \frac{1}{2} ((v - b)^2 + \|U - W\|_2^2) + \lambda|b| + \bar{\lambda}\|W\|_1 \\ &= \underset{b}{\operatorname{argmin}} \frac{1}{2} (v - b)^2 + \lambda|b| \\ &= \mathcal{S}_{\lambda}(v) \end{aligned}$$

This is the standard proximal mapping for Lasso regression (Parikh et al., 2014), whose optimum is given by the soft-thresholding operator $\mathcal{S}_{\lambda}(v)$.

Neural first layer step for W : Next, we solve for the global optimum with respect to W :

$$\begin{aligned} &\underset{W}{\operatorname{minimize}} \quad \frac{1}{2} (|v - b|^2 + \|U - W\|_2^2) + \lambda|b| + \bar{\lambda}\|W\|_1 \\ &\text{subject to} \quad \|W\|_{\infty} \leq M \cdot |b| \end{aligned}$$

Since we are not solving (b, W) jointly:

$$\begin{aligned} &\underset{W}{\operatorname{minimize}} \quad \frac{1}{2} (\|U - W\|_2^2) + \bar{\lambda}\|W\|_1 \\ &\text{subject to} \quad \|W\|_{\infty} \leq M \cdot |b| \end{aligned}$$

Proposition 2 (Closed-form block substeps for the sequential update): Fix $v \in \mathbb{R}^k$ and $U \in \mathbb{R}^K$ (Note: the two integers k, K can be different). The following block substeps admit closed-form solutions:

1. b^* is solved by standard proximal gradient method:

$$\begin{aligned} b^* &= \underset{b}{\operatorname{argmin}} \frac{1}{2} ((v - b)^2 + \|U - W\|_2^2) + \lambda|b| + \bar{\lambda}\|W\|_1 \\ &= \underset{b}{\operatorname{argmin}} \frac{1}{2} (v - b)^2 + \lambda|b| \\ &= \mathcal{S}_{\lambda}(v) \end{aligned}$$

where $\mathcal{S}_{\lambda}$ is the soft-thresholding operator.

2. Given b^* , the W -subproblem has the closed-form constrained shrinkage solution:

$$\begin{aligned} W^* &= \underset{W \text{ s.t. } \|W\|_{\infty} \leq M \cdot |b^*|}{\operatorname{argmin}} \frac{1}{2} ((v - b^*)^2 + \|U - W\|_2^2) + \lambda|b^*| + \bar{\lambda}\|W\|_1 \\ &= \underset{W \text{ s.t. } \|W\|_{\infty} \leq M \cdot |b^*|}{\operatorname{argmin}} \frac{1}{2} (\|U - W\|_2^2) + \bar{\lambda}\|W\|_1 \\ &= \text{sign}(U) \min \{M|b^*|, \mathcal{S}_{\bar{\lambda}}(|U|)\} \end{aligned}$$

where $\mathcal{S}_{\bar{\lambda}}$ is the soft-thresholding operator.

Proof. The first optimization problem is well known and is addressed by the standard proximal gradient and we omit the details. Readers can refer to (Boyd & Vandenberghe, 2004).

The derivation technique for the second problem is identical to the one in LassoNet. The key observation is their derivation for W does not depend on b being jointly optimized. We provide here for clarity and completeness.

We start by proving the claim below: for some $w \in \mathbb{R}_+$

$$\text{Claim : } W^* = \text{sign}(U) \min \{M \cdot |b|, \mathcal{S}_{\bar{\lambda}}(|U|)\}.$$

Denote $w = M |b^*|$, where b^* is solved in prior step above, as a constant for current step. By definition, W^* is the minimum of the below optimization problem:

$$\begin{aligned} & \underset{W}{\text{minimize}} && \frac{1}{2} \|U - W\|_F^2 + \bar{\lambda} \|W\|_1 \\ & \text{subject to} && \|W\|_{\infty} \leq M |b^*| = w. \end{aligned}$$

The key observation is that the constraint $\|W\|_{\infty} \leq M |b^*|$ can be written as separate inequality constraints where each constraint is described by a convex function:

$$\|W\|_{\infty} \leq M |b^*|$$

This is equivalent to:

$$|w_k| \leq M |b^*| \text{ for all } k$$

where $W = [w_1, \dots, w_K]$ and each w_k is a scalar.

Since $\frac{1}{2} \|U - W\|_F^2 + \bar{\lambda} \|W\|_1$ is also a convex function, the strong duality holds since Slater's condition holds (Section 5.2.3 in (Boyd & Vandenberghe, 2004)). Thereby, we know for some dual variable $s \in \mathbb{R}_+^K$, W^* minimizes the Lagrangian function below:

$$W^* = \underset{W \in \mathbb{R}^K}{\text{argmin}} \frac{1}{2} \|U - W\|_2^2 + \sum_{j=1}^K s_j |W_j| + \bar{\lambda} \|W\|_1.$$

Next, let's take subgradient and get that W^* needs to satisfy,

$$\begin{aligned} & W_j^* - U_j + (\bar{\lambda} + s_j) v_j^* = 0 \text{ for some } v_j^* \in \partial (|W_j^*|). \\ & s_j (|W_j^*| - w) = 0. \\ & s_j \geq 0, \text{ and } |W_j^*| \leq w. \end{aligned}$$

Now we divide our discussion into two cases:

1. $s_j = 0$. The KKT condition shows $U_j = W_j^* + \bar{\lambda} v_j^*$ for some $v_j^* \in \partial (|W_j^*|)$. This implies that $W_j^* = \mathcal{S}_{\bar{\lambda}}(U_j)$, which is possible if and only if $|\mathcal{S}_{\bar{\lambda}}(U_j)| \leq w$.
2. $s_j > 0$. The KKT condition gives $|W_j^*| = w$. Since $U_j = W_j^* + (\bar{\lambda} + s_j) v_j^*$ for $v_j^* \in \partial (|W_j^*|)$, it implies $\text{sign}(v_j^*) = \text{sign}(W_j^*) = \text{sign}(U_j)$. Hence $W_j^* = \text{sign}(U_j) w$. Note if $w \neq 0$, then we must have $v_j^* = \text{sign}(W_j^*) = \text{sign}(U_j)$. Thus, having some $s_j > 0$ with $U_j = W_j^* + (\bar{\lambda} + s_j) v_j^*$ is equivalent to $|\mathcal{S}_{\bar{\lambda}}(U_j)| > w$.

Summarizing the above discussion, we see that W^* must satisfy

$$W_j^* = \text{sign}(U_j) \min \{w, \mathcal{S}_{\bar{\lambda}}(|U_j|)\}$$

□

G. Appendix: Convergence of the Convex Hier-Prox Relaxation to a Smaller Noise Neighborhood

G.1. Background

LassoNet’s original hierarchical proximal operator couples a feature-gate θ_j and the first-layer weights $W_{j,\cdot} \in \mathbb{R}^K$ through the hard constraint

$$\|W_{j,\cdot}\|_\infty \leq M|\theta_j|,$$

and computes a per-feature prox by sorting the coordinates of $W_{j,\cdot}$. This induces a *piecewise* proximal map: under stochastic gradients, the update can jump when the active-set/sign pattern changes, and a function-value recursion typically contains an additional “noise + sign-switch” instability term. This is due to the nonconvex nature of the problem: union of two convex cones is not convex.

In this section we propose a *convex, M -scaled, coordinate-wise relaxation* that keeps the intended meaning of the constraint parameter M (the scale linking the gate and the first-layer row), while yielding a globally well-behaved proximal mapping (single-valued, firmly nonexpansive). The resulting stochastic recursion features *noise only* (no sign-switch penalty). This relaxation is not the update used in the final model; it is an analytically tractable proxy used to identify the stable shrinkage geometry that removes the sign-switch term.

Key modeling choice (explicit M via scaled block). For each feature j , define the scaled block

$$x_j := \begin{bmatrix} \theta_j \\ W_{j,\cdot}/M \end{bmatrix} \in \mathbb{R}^{K+1}, \quad (M > 0).$$

We replace the hard hierarchy by the convex surrogate

$$\Omega(\theta, W) := \alpha\|\theta\|_1 + \lambda_w\|W\|_1 + \beta \sum_{j=1}^d \|x_j\|_2, \quad (\alpha, \lambda_w, \beta \geq 0).$$

To retain an iterative *two closed-form proximal implementation*, we apply the proximal step in the scaled coordinate $\widetilde{W} := W/M$ (equivalently, a diagonal preconditioner in the original variables). This yields a per-feature update that is literally “ ℓ_1 -soft-threshold on θ_j ” followed by “group ℓ_2 shrink on $(\theta_j, \widetilde{W}_{j,\cdot})$ ”, in closed form.

We provide a proof of a master function-value recursion under a proximal-PL condition, matching Appendix F.

G.2. Setup: Scaled Coordinates, Stochastic Update, and Lyapunov

G.2.1. ORIGINAL VARIABLES AND M -SCALED COORDINATES

Let $\theta \in \mathbb{R}^d$ denote feature gates and let $W \in \mathbb{R}^{d \times K}$ be first-layer weights (row j is $W_{j,\cdot} \in \mathbb{R}^K$). Let $f_{\text{orig}}(\theta, W)$ be the (possibly nonconvex) smooth training loss as a function of this block.³

Fix $M > 0$ and define the scaled first-layer weights

$$\widetilde{W} := W/M.$$

Define the scaled loss

$$f(\theta, \widetilde{W}) := f_{\text{orig}}(\theta, M\widetilde{W}). \quad (24)$$

We work in the scaled variable

$$x := (\theta, \widetilde{W}) \in \mathbb{R}^d \times \mathbb{R}^{d \times K}.$$

After computing $\widetilde{W}$, we map back to original weights by $W = M\widetilde{W}$.

Remark G.1 (How gradients rescale). By the chain rule, $\nabla_{\widetilde{W}} f(\theta, \widetilde{W}) = M \nabla_W f_{\text{orig}}(\theta, M\widetilde{W})$. Thus implementing SGD on $(\theta, \widetilde{W})$ corresponds to using a diagonal preconditioner on the W -block in the original variables. This is precisely the mechanism that makes the M -scaled prox step admit a clean closed form.

³Other network parameters are treated as frozen.

G.2.2. STOCHASTIC PROXIMAL-GRADIENT UPDATE IN SCALED COORDINATES

Training uses a stochastic gradient g_t (e.g. a minibatch gradient) for the scaled loss f followed by a proximal map:

$$z_t = x_t - \eta g_t, \quad x_{t+1} = \Psi(z_t), \quad (25)$$

where $\eta > 0$ is a constant step size and

$$\Psi(z) \in \arg \min_x \left\{ \Omega(x) + \frac{1}{2} \|x - z\|^2 \right\}. \quad (26)$$

Remark G.2 (Equivalent view as a preconditioned proximal step in original variables). Writing $x = (\theta, \widetilde{W})$ and $W = M\widetilde{W}$, the scaled proximal problem

$$\Psi(z) \in \arg \min_x \left\{ \Omega(x) + \frac{1}{2} \|x - z\|^2 \right\}$$

is equivalent (under the change of variables $\widetilde{W} = W/M$) to a *preconditioned* proximal step in the original variables:

$$(\theta^+, W^+) \in \arg \min_{\theta, W} \left\{ \Omega(\theta, W) + \frac{1}{2} \|\theta - v\|_2^2 + \frac{1}{2M^2} \|W - U\|_F^2 \right\},$$

where (v, U) is the corresponding original-coordinate input to the prox. Thus the algorithm uses the natural M -dependent metric on the W -block; this is exactly what preserves a two-closed-form implementation.

G.2.3. THE LYAPUNOV FUNCTION

Following the existing value-function analysis, define

$$F(x) := \eta f(x) + \Omega(x). \quad (27)$$

The recursion we prove is in $F(x_t)$.

G.3. The M -scaled convex surrogate and the two-step coordinate-wise prox

G.3.1. PENALTY WITH EXPLICIT M

Definition G.3 (M -scaled sparse-group hierarchical surrogate). Fix parameters $\alpha \geq 0$, $\lambda_w \geq 0$, and $\beta \geq 0$, and fix $M > 0$. Define $\Omega : \mathbb{R}^{d+dK} \rightarrow \mathbb{R}$ (in *scaled coordinates* $x = (\theta, \widetilde{W})$) by

$$\Omega(\theta, \widetilde{W}) := \alpha \|\theta\|_1 + \lambda_w M \|\widetilde{W}\|_1 + \beta \sum_{j=1}^d \left\| \begin{bmatrix} \theta_j \\ \widetilde{W}_{j,\cdot} \end{bmatrix} \right\|_2. \quad (28)$$

Equivalently, in original variables (θ, W) ,

$$\Omega(\theta, W) = \alpha \|\theta\|_1 + \lambda_w \|W\|_1 + \beta \sum_{j=1}^d \left\| \begin{bmatrix} \theta_j \\ W_{j,\cdot}/M \end{bmatrix} \right\|_2.$$

Remark G.4 (Prox parameter on the scaled weights). Because $\|W\|_1 = M\|\widetilde{W}\|_1$, the ℓ_1 coefficient acting on $\widetilde{W}$ in the scaled-coordinate prox is aligned with the original parameter λ_w via the shorthand:

$$\bar{\lambda} := \lambda_w M.$$

This is the parameter that appears in Proposition G.9.

Remark G.5 (Interpretation). The group term $\sum_j \|(\theta_j, \widetilde{W}_{j,\cdot})\|_2$ encourages *feature-level coupling*: if θ_j is pushed towards zero, then (in the same group) $\widetilde{W}_{j,\cdot}$ is also shrunk. The parameter M calibrates what it means for the row $W_{j,\cdot}$ to be “large” relative to the gate θ_j .

G.3.2. PROXIMAL MAPPING FACTS (CONVEX CASE)

Lemma G.6 (Firm nonexpansiveness of a convex prox). *If Ω is proper, closed, and convex, then the proximal mapping Ψ in equation 26 is single-valued and firmly nonexpansive:*

$$\|\Psi(z) - \Psi(z')\|^2 \leq \langle \Psi(z) - \Psi(z'), z - z' \rangle \leq \|z - z'\|^2, \quad \forall z, z'.$$

In particular, Ψ is 1-Lipschitz.

Remark G.7. Lemma G.6 is the key structural simplification compared to the original Hier-Prox: here Ω is convex, hence Ψ is globally well-behaved (no branch jumps).

G.3.3. PER-FEATURE PROX AND A CLOSED-FORM OPTIMALITY PROPOSITION

Because Ω in equation 28 is separable across features (Boyd & Vandenberghe, 2004), the proximal problem equation 26 decomposes into d independent $(K + 1)$ -dimensional subproblems.

Definition G.8 (Soft-thresholding). For $\tau \geq 0$, define $\mathcal{S}_\tau : \mathbb{R} \rightarrow \mathbb{R}$ by

$$\mathcal{S}_\tau(u) := \text{sign}(u) (|u| - \tau)_+, \quad (a)_+ := \max\{a, 0\}.$$

For vectors/matrices, apply $\mathcal{S}_\tau$ entrywise.

Proposition G.9 (Closed form for the per-feature prox with explicit M and ℓ_1 on W). *Fix $v \in \mathbb{R}$ and a row vector $\tilde{U} \in \mathbb{R}^K$ (the scaled-row input), and parameters $\alpha \geq 0$, $\bar{\lambda} \geq 0$, and $\beta \geq 0$. Consider the convex proximal subproblem in scaled coordinates:*

$$\min_{b \in \mathbb{R}, \tilde{w} \in \mathbb{R}^K} \frac{1}{2}(b - v)^2 + \frac{1}{2}\|\tilde{w} - \tilde{U}\|_2^2 + \alpha|b| + \bar{\lambda}\|\tilde{w}\|_1 + \beta \left\| \begin{bmatrix} b \\ \tilde{w} \end{bmatrix} \right\|_2. \quad (29)$$

Then the unique minimizer $(b^*, \tilde{w}^*)$ exists and is given by the following two-step closed form:

1. **(Coordinate soft-threshold)** Define

$$\tilde{b} := \mathcal{S}_\alpha(v), \quad \tilde{w} := \mathcal{S}_{\bar{\lambda}}(\tilde{U}) \quad (\text{coordinatewise}).$$

2. **(Group ℓ_2 shrink)** Let

$$r := \sqrt{\tilde{b}^2 + \|\tilde{w}\|_2^2}, \quad \kappa := \left(1 - \frac{\beta}{r}\right)_+ \quad (\kappa = 0 \text{ if } r = 0).$$

Then

$$(b^*, \tilde{w}^*) = \kappa(\tilde{b}, \tilde{w}).$$

Equivalently,

$$(b^*, \tilde{w}^*) = \begin{cases} (0, 0), & r \leq \beta, \\ (1 - \beta/r)(\tilde{b}, \tilde{w}), & r > \beta. \end{cases}$$

Proof. Problem equation 29 is strictly convex because of the quadratic terms, hence has a unique minimizer (Appendix F).

Let $x := (b, \tilde{w}) \in \mathbb{R}^{1+K}$ and $y := (v, \tilde{U}) \in \mathbb{R}^{1+K}$. Define the convex regularizer on $\mathbb{R}^{1+K}$,

$$h(x) := \alpha|b| + \bar{\lambda}\|\tilde{w}\|_1 + \beta\|x\|_2.$$

Then equation 29 is exactly the proximal problem

$$x^* = \text{prox}_h(y) := \arg \min_x \frac{1}{2}\|x - y\|_2^2 + h(x).$$

By standard subgradient optimality for proximal maps (Boyd & Vandenberghe, 2004; Parikh et al., 2014), x^* is the unique point satisfying

$$y - x^* \in \partial h(x^*). \quad (30)$$

Step 1: the coordinatewise ℓ_1 prox. Let

$$\tilde{x} := (\tilde{b}, \tilde{w}) := (\mathcal{S}_\alpha(v), \mathcal{S}_{\bar{\lambda}}(\tilde{U})).$$

The defining property of soft-thresholding is precisely the proximal optimality condition for the separable ℓ_1 term:

$$y - \tilde{x} \in \partial(\alpha|b| + \bar{\lambda}\|\tilde{w}\|_1)|_{x=\tilde{x}}. \quad (31)$$

(Indeed, for each coordinate, $\mathcal{S}$ is the proximal map of the absolute value.)

Step 2: add the group ℓ_2 term. If $\tilde{x} = 0$, then $r = 0$ and the claimed formula gives $x^* = 0$, which satisfies equation 30 because $\partial(\beta\|x\|_2)|_{x=0} = \{u : \|u\|_2 \leq \beta\}$.

Assume now $\tilde{x} \neq 0$, so $r = \|\tilde{x}\|_2 > 0$. If $r \leq \beta$ then $\kappa = 0$ and $x^* = 0$; this is optimal because $y \in \partial h(0)$ can be verified by combining equation 31 at 0 with $\tilde{x} \in \beta \mathbb{B}_2$ (since $\|\tilde{x}\|_2 = r \leq \beta$).

Now consider $r > \beta$, so $\kappa = 1 - \beta/r \in (0, 1)$ and define $x^* := \kappa \tilde{x}$. Then $\|x^*\|_2 = \kappa r > 0$ and therefore

$$\partial(\beta\|x\|_2)|_{x=x^*} = \left\{ \beta \frac{x^*}{\|x^*\|_2} \right\} = \left\{ \beta \frac{\tilde{x}}{r} \right\}.$$

Moreover,

$$y - x^* = (y - \tilde{x}) + (1 - \kappa)\tilde{x} \quad \text{with} \quad 1 - \kappa = \beta/r.$$

By equation 31, the first term belongs to $\partial(\alpha|b| + \bar{\lambda}\|\tilde{w}\|_1)$ evaluated at $\tilde{x}$. Since $x^* = \kappa \tilde{x}$ has the same sign and sparsity pattern as $\tilde{x}$ (scaling by a positive scalar), the same subgradient selection is valid at x^* , i.e.

$$y - \tilde{x} \in \partial(\alpha|b| + \bar{\lambda}\|\tilde{w}\|_1)|_{x=\tilde{x}}.$$

The second term satisfies

$$(1 - \kappa)\tilde{x} = \frac{\beta}{r}\tilde{x} \in \partial(\beta\|x\|_2)|_{x=x^*}.$$

Combining the two pieces yields $y - x^* \in \partial h(x^*)$, proving equation 30 and hence optimality. $\square$

Remark G.10 (Back to original variables and where M appears). At feature j , let $(v_j, \tilde{U}_j)$ be the scaled-coordinate input to the prox (a block of z_t), and let $U_j^{\text{orig}} := M\tilde{U}_j$ denote the corresponding original-coordinate row.

With the shorthand $\bar{\lambda} = \lambda_w M$, Proposition G.9 returns

$$(\theta_j^+, \tilde{W}_{j,\cdot}^+) = \kappa_j (\mathcal{S}_\alpha(v_j), \mathcal{S}_{\bar{\lambda}}(\tilde{U}_j)), \quad \kappa_j = \left(1 - \frac{\beta}{\sqrt{\mathcal{S}_\alpha(v_j)^2 + \|\mathcal{S}_{\bar{\lambda}}(\tilde{U}_j)\|_2^2}} \right)_+.$$

Mapping back yields

$$W_{j,\cdot}^+ = M\tilde{W}_{j,\cdot}^+ = \kappa_j M \mathcal{S}_{\bar{\lambda}}(\tilde{U}_j).$$

Equivalently, since $\mathcal{S}_{\bar{\lambda}}(\tilde{U}_j) = \frac{1}{M}\mathcal{S}_{\lambda_w M^2}(U_j^{\text{orig}})$,

$$\boxed{W_{j,\cdot}^+ = \kappa_j \mathcal{S}_{\lambda_w M^2}(U_j^{\text{orig}})}, \quad \kappa_j = \left(1 - \frac{\beta}{\sqrt{\mathcal{S}_\alpha(v_j)^2 + \|\mathcal{S}_{\lambda_w M^2}(U_j^{\text{orig}})\|_2^2/M^2}} \right)_+.$$

When $\lambda_w = 0$, the intermediate soft-thresholding disappears and we recover the previous special case $W_{j,\cdot}^+ = \kappa_j U_j^{\text{orig}}$.

Coordinate-wise (sequential) implementation. Per feature j , the proximal step is computed by two explicit substeps:

1. soft-threshold the gate coordinate: $\tilde{v}_j = \mathcal{S}_\alpha(v_j)$;
2. shrink the concatenated block $(\tilde{v}_j, \tilde{U}_j)$ by $\kappa_j = (1 - \beta/r_j)_+$ with $r_j = \sqrt{\tilde{v}_j^2 + \|\tilde{U}_j\|_2^2}$.

This is parallel across features.

G.4. Deterministic Descent in Function Values (No Convexity of f Needed)

Assumption G.11 (L -smoothness). The function f is L -smooth: for all x, x' ,

$$f(x') \leq f(x) + \langle \nabla f(x), x' - x \rangle + \frac{L}{2} \|x' - x\|^2.$$

Lemma G.12 (A key proximal inequality). Let $z \in \mathbb{R}^{d+dK}$ and let $x^+ = \Psi(z)$ be defined by equation 26. Then for every $x \in \mathbb{R}^{d+dK}$,

$$\Omega(x^+) + \frac{1}{2} \|x^+ - z\|^2 \leq \Omega(x) + \frac{1}{2} \|x - z\|^2 - \frac{1}{2} \|x - x^+\|^2. \quad (32)$$

Proof. Define $\phi(x) := \Omega(x) + \frac{1}{2} \|x - z\|^2$. Then ϕ is 1-strongly convex. Hence for all x and all $g \in \partial\phi(x^+)$,

$$\phi(x) \geq \phi(x^+) + \langle g, x - x^+ \rangle + \frac{1}{2} \|x - x^+\|^2.$$

At the minimizer x^+ we can take $g = 0$, yielding $\phi(x) \geq \phi(x^+) + \frac{1}{2} \|x - x^+\|^2$, i.e. equation 32. $\square$

Lemma G.13 (Deterministic one-step descent in F). Assume f is L -smooth and $0 < \eta \leq \frac{1}{2L}$. Let $g = \nabla f(x)$ and define $z = x - \eta g$ and $x^+ = \Psi(z)$. Then

$$F(x^+) \leq F(x) - \frac{1}{4} \|x - x^+\|^2. \quad (33)$$

Proof. Apply Lemma G.12 with $z = x - \eta \nabla f(x)$ and comparator x :

$$\Omega(x^+) + \frac{1}{2} \|x^+ - (x - \eta \nabla f(x))\|^2 \leq \Omega(x) + \frac{1}{2} \|x - (x - \eta \nabla f(x))\|^2 - \frac{1}{2} \|x - x^+\|^2.$$

The middle term is $\frac{1}{2} \eta^2 \|\nabla f(x)\|^2$. Expand the left square:

$$\|x^+ - (x - \eta \nabla f(x))\|^2 = \|x^+ - x\|^2 + 2\eta \langle \nabla f(x), x^+ - x \rangle + \eta^2 \|\nabla f(x)\|^2.$$

Cancel the $\eta^2 \|\nabla f(x)\|^2$ terms and rearrange:

$$\Omega(x^+) + \frac{1}{2} \|x^+ - x\|^2 + \eta \langle \nabla f(x), x^+ - x \rangle \leq \Omega(x) - \frac{1}{2} \|x - x^+\|^2.$$

Rearrange:

$$\Omega(x^+) \leq \Omega(x) - \|x - x^+\|^2 - \eta \langle \nabla f(x), x^+ - x \rangle. \quad (34)$$

Now apply L -smoothness (Assumption G.11) with $x' = x^+$:

$$f(x^+) \leq f(x) + \langle \nabla f(x), x^+ - x \rangle + \frac{L}{2} \|x^+ - x\|^2.$$

Multiply by η and add to equation 34:

$$F(x^+) \leq F(x) - \|x - x^+\|^2 + \frac{\eta L}{2} \|x^+ - x\|^2.$$

With $\eta \leq 1/(2L)$, we get $1 - \eta L/2 \geq 3/4$, hence

$$F(x^+) \leq F(x) - \frac{3}{4} \|x - x^+\|^2,$$

which is stronger than equation 33. $\square$

G.5. A Proximal-PL Condition (Function-Value Landscape)

Definition G.14 (Proximal-gradient mapping). Define the (scaled) proximal-gradient mapping for $F(x) = \eta f(x) + \Omega(x)$ as

$$G(x) := x - \Psi(x - \eta \nabla f(x)).$$

Definition G.15 (Proximal-PL condition in function values). We say F satisfies a proximal-PL condition with constant $\mu > 0$ if for all x ,

$$\frac{1}{2} \|G(x)\|^2 \geq \mu (F(x) - F^*), \quad F^* := \inf_x F(x). \quad (35)$$

G.6. Stochastic Recursion: Noise Only (No Sign-Switch Penalty)

Assumption G.16 (Unbiased gradients and bounded conditional variance). The stochastic gradients satisfy $\mathbb{E}[g_t \mid x_t] = \nabla f(x_t)$ and

$$\mathbb{E}[\|\xi_t\|^2 \mid x_t] \leq \sigma^2, \quad \xi_t := g_t - \nabla f(x_t).$$

G.6.1. A CONDITIONAL DESCENT INEQUALITY

Lemma G.17 (Stochastic descent with a noise cross term). Assume Assumption G.11 and Assumption G.16. If $\eta \leq 1/L$, then almost surely

$$F(x_{t+1}) \leq F(x_t) - \frac{1-\eta L}{2} \|x_{t+1} - x_t\|^2 - \eta \langle \xi_t, x_{t+1} - x_t \rangle.$$

Proof. Apply Lemma G.12 to $x_{t+1} = \Psi(z_t)$ with comparator $x = x_t$ and $z_t = x_t - \eta g_t$:

$$\Omega(x_{t+1}) + \frac{1}{2} \|x_{t+1} - z_t\|^2 \leq \Omega(x_t) + \frac{1}{2} \|x_t - z_t\|^2.$$

Expand the squares:

$$\|x_{t+1} - z_t\|^2 = \|x_{t+1} - x_t\|^2 + 2\eta \langle g_t, x_{t+1} - x_t \rangle + \eta^2 \|g_t\|^2, \quad \|x_t - z_t\|^2 = \eta^2 \|g_t\|^2.$$

Cancel the $\eta^2 \|g_t\|^2$ terms and rearrange:

$$\Omega(x_{t+1}) + \frac{1}{2} \|x_{t+1} - x_t\|^2 + \eta \langle g_t, x_{t+1} - x_t \rangle \leq \Omega(x_t).$$

By L -smoothness (Assumption G.11) with $x' = x_{t+1}$:

$$f(x_{t+1}) \leq f(x_t) + \langle \nabla f(x_t), x_{t+1} - x_t \rangle + \frac{L}{2} \|x_{t+1} - x_t\|^2.$$

Multiply by η and add:

$$F(x_{t+1}) \leq F(x_t) - \frac{1-\eta L}{2} \|x_{t+1} - x_t\|^2 - \eta \langle g_t - \nabla f(x_t), x_{t+1} - x_t \rangle.$$

Since $\xi_t = g_t - \nabla f(x_t)$, this is the claim. $\square$

G.6.2. REMOVING THE NOISE CROSS TERM (YOUNG'S INEQUALITY)

Corollary G.18 (Conditional expectation form). Under the assumptions of Lemma G.17, we have

$$\mathbb{E}[F(x_{t+1}) \mid x_t] \leq F(x_t) - \frac{1-\eta L}{4} \mathbb{E}[\|x_{t+1} - x_t\|^2 \mid x_t] + \frac{\eta^2}{1-\eta L} \mathbb{E}[\|\xi_t\|^2 \mid x_t].$$

Proof. Start from Lemma G.17 and take conditional expectation:

$$\mathbb{E}[F(x_{t+1}) \mid x_t] \leq F(x_t) - \frac{1-\eta L}{2} \mathbb{E}[\|x_{t+1} - x_t\|^2 \mid x_t] - \eta \mathbb{E}[\langle \xi_t, x_{t+1} - x_t \rangle \mid x_t].$$

Use Cauchy–Schwarz and Young:

$$-\eta \langle \xi_t, x_{t+1} - x_t \rangle \leq \eta \|\xi_t\| \|x_{t+1} - x_t\| \leq \frac{1-\eta L}{4} \|x_{t+1} - x_t\|^2 + \frac{\eta^2}{1-\eta L} \|\xi_t\|^2.$$

Take conditional expectation and plug in. $\square$

G.6.3. RELATING THE STOCHASTIC STEP LENGTH TO THE DETERMINISTIC PROX-GRADIENT MAPPING

Define the *deterministic* proximal-gradient step at x_t :

$$\bar{z}_t := x_t - \eta \nabla f(x_t), \quad \bar{x}_{t+1} := \Psi(\bar{z}_t).$$

Lemma G.19 (Lower bound on the stochastic step length (convex prox case)). *For any t ,*

$$\mathbb{E}[\|x_{t+1} - x_t\|^2 \mid x_t] \geq \frac{1}{2} \|G(x_t)\|^2 - \mathbb{E}[\|x_{t+1} - \bar{x}_{t+1}\|^2 \mid x_t],$$

and moreover

$$\mathbb{E}[\|x_{t+1} - \bar{x}_{t+1}\|^2 \mid x_t] \leq \eta^2 \mathbb{E}[\|\xi_t\|^2 \mid x_t].$$

Proof. Note that $x_{t+1} - x_t = (x_{t+1} - \bar{x}_{t+1}) + (\bar{x}_{t+1} - x_t)$ and $\bar{x}_{t+1} - x_t = -G(x_t)$ by definition. Hence

$$\|x_{t+1} - x_t\|^2 = \|(x_{t+1} - \bar{x}_{t+1}) - G(x_t)\|^2 \geq \frac{1}{2} \|G(x_t)\|^2 - \|x_{t+1} - \bar{x}_{t+1}\|^2.$$

Take conditional expectation to get the first inequality.

For the second inequality, note that $z_t = \bar{z}_t - \eta \xi_t$ and by Lemma G.6, Ψ is 1-Lipschitz:

$$\|x_{t+1} - \bar{x}_{t+1}\| = \|\Psi(z_t) - \Psi(\bar{z}_t)\| \leq \|z_t - \bar{z}_t\| = \eta \|\xi_t\|.$$

Square and take conditional expectation. □

G.6.4. MASTER RECURSION IN FUNCTION VALUES

Theorem G.20 (Master recursion under proximal-PL: linear-to-noise-ball). *Assume f is L -smooth (Assumption G.11) and $0 < \eta \leq \frac{1}{2L}$. Assume the stochastic gradient model (Assumption G.16). Assume $F(x) = \eta f(x) + \Omega(x)$ satisfies proximal-PL with constant $\mu > 0$ (Definition G.15).*

Let $\Delta_t := \mathbb{E}[F(x_t) - F^*]$, where $F^* := \inf_x F(x)$. Then the iterates equation 25 satisfy

$$\Delta_{t+1} \leq \left(1 - \frac{\mu(1-\eta L)}{4}\right) \Delta_t + \eta^2 \sigma^2 \left(\frac{1-\eta L}{4} + \frac{1}{1-\eta L}\right). \quad (36)$$

In particular, Δ_t converges linearly to a neighborhood of radius $O\left(\frac{\eta^2 \sigma^2}{\mu}\right)$, and if $\sigma = 0$ then Δ_t converges linearly to 0.

Proof. Start from Corollary G.18 and bound the conditional step length using Lemma G.19:

$$\begin{aligned} \mathbb{E}[F(x_{t+1}) \mid x_t] &\leq F(x_t) - \frac{1-\eta L}{4} \mathbb{E}[\|x_{t+1} - x_t\|^2 \mid x_t] + \frac{\eta^2}{1-\eta L} \mathbb{E}[\|\xi_t\|^2 \mid x_t] \\ &\leq F(x_t) - \frac{1-\eta L}{4} \left(\frac{1}{2} \|G(x_t)\|^2 - \mathbb{E}[\|x_{t+1} - \bar{x}_{t+1}\|^2 \mid x_t]\right) + \frac{\eta^2}{1-\eta L} \mathbb{E}[\|\xi_t\|^2 \mid x_t]. \end{aligned}$$

Using the second bound in Lemma G.19, we have $\mathbb{E}[\|x_{t+1} - \bar{x}_{t+1}\|^2 \mid x_t] \leq \eta^2 \mathbb{E}[\|\xi_t\|^2 \mid x_t]$. Substitute this and simplify:

$$\mathbb{E}[F(x_{t+1}) \mid x_t] \leq F(x_t) - \frac{1-\eta L}{8} \|G(x_t)\|^2 + \eta^2 \mathbb{E}[\|\xi_t\|^2 \mid x_t] \left(\frac{1-\eta L}{4} + \frac{1}{1-\eta L}\right).$$

Now apply proximal-PL (Definition G.15):

$$\frac{1}{2} \|G(x_t)\|^2 \geq \mu(F(x_t) - F^*) \implies \|G(x_t)\|^2 \geq 2\mu(F(x_t) - F^*).$$

Therefore,

$$\mathbb{E}[F(x_{t+1}) - F^* \mid x_t] \leq \left(1 - \frac{\mu(1-\eta L)}{4}\right) (F(x_t) - F^*) + \eta^2 \mathbb{E}[\|\xi_t\|^2 \mid x_t] \left(\frac{1-\eta L}{4} + \frac{1}{1-\eta L}\right).$$

Finally, take total expectation and use $\mathbb{E}[\|\xi_t\|^2 \mid x_t] \leq \sigma^2$ to obtain equation 36. □

G.7. Interpretations and Implementation Notes

Remark G.21 (What improves relative to Hier-Prox). The original Hier-Prox mapping is piecewise-defined and can jump when active sets/sign patterns change; this creates an additional instability term (e.g. a “jump size” squared times a switch probability) in a function-value recursion. With the convex surrogate equation 28, the prox map is globally firmly nonexpansive (Lemma G.6), so the recursion equation 36 has *noise only*.

Remark G.22 (What we give up). We no longer enforce the hard hierarchy $\|W_{j,\cdot}\|_\infty \leq M|\theta_j|$. Instead, we encourage the gate and the row weights to shrink *jointly* via the convex sparse-group penalty on $(\theta_j, W_{j,\cdot}/M)$. This is often favorable when stability and simple proximal geometry are prioritized.

Closed-form shrinkage structure (featurewise). Given the pre-prox iterate in scaled coordinates for feature j , i.e. $(v_j, \tilde{U}_j)$, do:

1. Soft-threshold the gate: $\tilde{v}_j = \mathcal{S}_\alpha(v_j)$.
2. Group shrink: $r_j = \sqrt{\tilde{v}_j^2 + \|\tilde{U}_j\|_2^2}$ and $\kappa_j = (1 - \beta/r_j)_+$. Return $(\theta_j^+, \tilde{W}_{j,\cdot}^+) = \kappa_j(\tilde{v}_j, \tilde{U}_j)$ and map back $W_{j,\cdot}^+ = M\tilde{W}_{j,\cdot}^+$.

The relaxation remains coupled through the shared group-shrinkage factor κ_j . Its relevance is that the coupling follows an explicit gate-first shrinkage structure rather than the sorted hard projection in which the row weights can dominate the gate magnitude.

M -tuning intuition. In the group-shrink step, $\|\tilde{U}_j\|_2 = \|U_j^{\text{orig}}\|_2/M$. Thus increasing M makes $\|\tilde{U}_j\|_2$ smaller and tends to *increase* shrinkage (smaller κ_j), which matches the intuition that large M enforces a tighter coupling between the gate and the row weights.

Remark G.23. This convex relaxation provides theoretical evidence that removing sign-branching and using stable gate-first shrinkage geometry can improve stochastic dynamics. It motivates, but does not formally justify, the hard-constrained Seq-Hier-Prox update used in the main algorithm.